

Veri Seti Hakkında Bilgi

Problem: Şirketi terk edecek müşterileri (müşteri kaybı) tahmin edebilecek bir makine öğrenmesi modeli geliştirilmesi istenmektedir.

Değişkenler:

1. **CreditScore (Kredi Skoru):** Müşterilerin kredi puanını verir. Yüksek puan, müşterinin borcunun yüksek geri ödeme kapasitesini gösterir.
2. **Geography (Ülke):** Müşterinin ülkesi.
3. **Gender (Cinsiyet):** Müşterinin cinsiyeti.
4. **Age (Yaş):** Müşterinin yaşı.
5. **Tenure:** Müşterinin bankada kaldığı ay sayısı.
6. **Balance (Bakiye):** Müşterinin hesabında bulunan tutar (bakiye).
7. **NumOfProducts:** Müşterinin sahip olduğu ürün sayısı.
8. **HasCrCard:** Müşterinin kredi kartı olup olmadığı bilgisi.
9. **IsActiveMember:** Müşterinin aktif üye olup olmadığı bilgisi.
10. **EstimatedSalary:** Müşterinin tahmini maaşı.
11. **Exited (Churn):** Müşterinin bankadan ayrılıp ayrılmadığı bilgisi.

Gerekli kütüphaneleri yükleyelim:

```
"""Veri hazırlık kütüphaneleri"""
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

"""Modelleme Kütüphaneleri"""
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.naive_bayes import GaussianNB
import xgboost as xgb
from xgboost import XGBClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

"""Model Eleme"""
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV

"""Diğer"""
import os
import warnings
#from sklearn.utils.testing import ignore_warnings
from sklearn.exceptions import ConvergenceWarning
%matplotlib inline
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=ConvergenceWarning)
import warnings
warnings.simplefilter(action="ignore")

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 170)
pd.set_option('display.max_rows', None)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
```

Veriyi okutalım:

```
: df = pd.read_csv(r"C:\Users\HANDENUR\Downloads\Churn.csv")
```

```
df.shape
```

```
(10000, 11)
```

Verimiz 10.000 satır, 11 sütundan (değişken) oluşuyor.

İlk 10 satıra bakalım:

```
df.head(10)
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	619	France	Female	42	2	0.000	1	1	1	101348.880	1
1	608	Spain	Female	41	1	83807.860	1	0	1	112542.580	0
2	502	France	Female	42	8	159660.800	3	1	0	113931.570	1
3	699	France	Female	39	1	0.000	2	0	0	93826.630	0
4	850	Spain	Female	43	2	125510.820	1	1	1	79084.100	0
5	645	Spain	Male	44	8	113755.780	2	1	0	149756.710	1
6	822	France	Male	50	7	0.000	2	1	1	10062.800	0
7	376	Germany	Female	29	4	115046.740	4	1	0	119346.880	1
8	501	France	Male	44	4	142051.070	2	0	1	74940.500	0
9	684	France	Male	27	2	134603.880	1	1	1	71725.730	0

Son 10 satıra bakalım:

```
df.tail(10)
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
9990	714	Germany	Male	33	3	35016.600	1	1	0	53667.080	0
9991	597	France	Female	53	4	88381.210	1	1	0	69384.710	1
9992	726	Spain	Male	36	2	0.000	1	1	0	195192.400	0
9993	644	France	Male	28	7	155060.410	1	1	0	29179.520	0
9994	800	France	Female	29	2	0.000	2	0	0	167773.550	0
9995	771	France	Male	39	5	0.000	2	1	0	96270.640	0
9996	516	France	Male	35	10	57369.610	1	1	1	101699.770	0
9997	709	France	Female	36	7	0.000	1	0	1	42085.580	1
9998	772	Germany	Male	42	3	75075.310	2	1	0	92888.520	1
9999	792	France	Female	28	4	130142.790	1	1	0	38190.780	0

Değişken isimlerini Türkçeye çevirelim.

```
# Sütun isimlerini Türkçeye çevirme
df.rename(columns={
    "CreditScore": "kredi_skoru",
    "Geography": "ulke",
    "Gender": "cinsiyet",
    "Age": "yas",
    "Tenure": "sirkette_kaldığı_ay_sayısı",
    "Balance": "bakiye",
    "NumOfProducts": "urun_sayısı",
    "HasCrCard": "kredi_kartı_var_mı",
    "IsActiveMember": "aktif_uye_mi",
    "EstimatedSalary": "tahmini_maas",
    "Exited": "ayrıldı_mı(churn)"
}, inplace=True)

# Yeni sütun isimlerini görüntüle
df.head()
```

	kredi_skoru	ulke	cinsiyet	yas	sirkette_kaldigi_ay_sayisi	bakiye	urun_sayisi	kredi_karti_var_mi	aktif_uye_mi	tahmini_maas	ayrildi_mi(churn)
0	619	France	Female	42	2	0.000	1	1	1	101348.880	1
1	608	Spain	Female	41	1	83807.860	1	0	1	112542.580	0
2	502	France	Female	42	8	159660.800	3	1	0	113931.570	1
3	699	France	Female	39	1	0.000	2	0	0	93826.630	0
4	850	Spain	Female	43	2	125510.820	1	1	1	79084.100	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   kredi_skoru                          10000 non-null   int64
1   ulke                                 10000 non-null   object
2   cinsiyet                             10000 non-null   object
3   yas                                  10000 non-null   int64
4   sirkette_kaldigi_ay_sayisi           10000 non-null   int64
5   bakiye                               10000 non-null   float64
6   urun_sayisi                          10000 non-null   int64
7   kredi_karti_var_mi                  10000 non-null   int64
8   aktif_uye_mi                        10000 non-null   int64
9   tahmini_maas                        10000 non-null   float64
10  ayrildi_mi(churn)                   10000 non-null   int64
dtypes: float64(2), int64(7), object(2)
memory usage: 859.5+ KB
```

Veri tiplerine baktığımızda bazı değişkenlerde hata olduğunu görüyoruz. Değişken tiplerini kontrol edelim.

```
def grab_col_names(dataframe, cat_th=10, car_th=15):

    # cat_cols, cat_but_car
    cat_cols = [col for col in dataframe.columns if dataframe[col].dtypes == "O"]
    num_but_cat = [col for col in dataframe.columns if dataframe[col].nunique() < cat_th and dataframe[col].dtypes != "O"]
    cat_but_car = [col for col in dataframe.columns if dataframe[col].nunique() > car_th and dataframe[col].dtypes == "O"]
    cat_cols = cat_cols + num_but_cat
    cat_cols = [col for col in cat_cols if col not in cat_but_car]

    # num_cols
    num_cols = [col for col in dataframe.columns if dataframe[col].dtypes != "O"]
    num_cols = [col for col in num_cols if col not in num_but_cat]

    print(f"Observations: {dataframe.shape[0]}")
    print(f"Variables: {dataframe.shape[1]}")
    print(f'cat_cols: {len(cat_cols)}')
    print(f'num_cols: {len(num_cols)}')
    print(f'cat_but_car: {len(cat_but_car)}')
    print(f'num_but_cat: {len(num_but_cat)}')

    return cat_cols, num_cols, cat_but_car
```

Bu fonksiyon veri setindeki kategorik, numerik ve kategorik fakat kardinal değişkenlerin isimlerini verir.

Not: Kategorik değişkenlerin içerisine numerik görünümlü kategorik değişkenler de dahildir.

Parametreleri:

Dataframe: Değişken isimleri alınmak istenilen dataframe

cat_th:((int, optional) numerik fakat kategorik olan değişkenler için sınıf eşik değeri

car_th: (int, optional)kategorik fakat kardinal değişkenler için sınıf eşik değeri

Returns:

cat_cols:Kategorik değişken listesi

num_cols:Numerik değişken listesi

cat_but_car: Kategorik görünümlü kardinal değişken listesi

Not:

cat_cols + num_cols + cat_but_car = toplam değişken sayısı

num_but_cat,cat_cols'un içerisinde.

```
cat_cols, num_cols, cat_but_car = grab_col_names(df)

cat_cols
num_cols
cat_but_car
```

```
Observations: 10000
Variables: 11
cat_cols: 6
num_cols: 5
cat_but_car: 0
num_but_cat: 4
```

```
[]
```

```
cat_cols
```

```
['ulke',
 'cinsiyet',
 'urun_sayısı',
 'kredi_kartı_var_mı',
 'aktif_uye_mi',
 'ayrıldı_mı(churn)']
```

```
num_cols
```

```
['kredi_skoru', 'yas', 'sirkette_kaldığı_ay_sayısı', 'bakiye', 'tahmini_maas']
```

‘Ülke’ ve ‘urun_sayısı’ değişkenleri kategorik gözüküyor. Diğer değişkenler doğru listelenmiş. Ülke ve ürün sayısına detaylı bakalım.

```
df["ulke"].unique()
array(['France', 'Spain', 'Germany'], dtype=object)
```

Ülkeler sadece 3 türden oluşuyor. String değil kategorik tipinde kabul ederiz.

```
df["urun_sayısı"].unique()
array([1, 3, 2, 4], dtype=int64)
```

Ürün sayısı sadece 4 değerden oluşuyor. Kategorik tipinde kabul ederiz.

Grab_col_names fonksiyonumuzun kategorik bulduğu değişkenler gerçekten de kategorik.

Eksik değer kontrolü:

```
df.isnull().sum()
kredi_skoru      0
ulke             0
cinsiyet         0
yas             0
sirkette_kaldığı_ay_sayısı  0
bakiye          0
urun_sayısı      0
kredi_kartı_var_mı  0
aktif_uye_mi     0
tahmini_maas     0
ayrıldı_mı(churn)  0
kredi_segmenti   0
dtype: int64
```

Eksik değer yoktur.

Sayısal değişkenler için betimleyici istatistikler:

```
df.describe()
```

	kredi_skoru	yas	sirkette_kaldığı_ay_sayısı	bakiye	tahmini_maas
count	10000.000	10000.000	10000.000	10000.000	10000.000
mean	650.529	38.922	5.013	76485.889	100090.240
std	96.653	10.488	2.892	62397.405	57510.493
min	350.000	18.000	0.000	0.000	11.580
25%	584.000	32.000	3.000	0.000	51002.110
50%	652.000	37.000	5.000	97198.540	100193.915
75%	718.000	44.000	7.000	127644.240	149388.247
max	850.000	92.000	10.000	250898.090	199992.480

Kredi Skoru için:

- Ortalama (mean): Müşterilerin kredi skoru ortalama 650.529'dur.
- Standart Sapma (std): Müşterilerin kredi skoru 96.653 civarında bir yayılıma sahiptir.
- Min: En düşük kredi skoru 350.000'dir.
- 25%: %25'lik kesimin kredi skoru 584.000 ve altındadır.
- 50%: Ortanca kredi skoru 652.000'dir.
- 75%: %75'lik kesimin kredi skoru 718.000 ve altındadır.
- Max: En yüksek kredi skoru 850.000'dir.

Yaş için:

- Ortalama (mean): Müşterilerin yaş ortalaması yaklaşık 39'dur.
- Standart Sapma (std): Müşterilerin yaşı 10.488 civarında bir yayılıma sahiptir.
- Min: En küçük yaş 18'dir.
- 25%: %25'lik kesimin yaşı 32 ve altındadır.
- 50%: Ortanca yaş 37'dir.
- 75%: %75'lik kesimin yaşı 44 ve altındadır.
- Max: En yüksek yaş 92'dir.

Şirkette kaldığı ay sayısı için:

- Ortalama (mean): Müşterilerin şirkette kalma süresi ortalama 5 aydır.
- Standart Sapma (std): Müşterilerin şirkette kalma süresi 2,89 ay civarında bir yayılıma sahiptir.
- Min: Şirkette en kısa süre kalan müşteri 0 ay kalmıştır.
- 25%: %25'lik kesimin şirkette kalma süresi 3 ay ve altındadır.
- 50%: Ortanca değer 5 aydır.
- 75%: %75'lik kesimin şirkette kalma süresi 7 ay ve altındadır.
- Max: Şirkette en uzun kalan müşteri 10 ay kalmıştır.

Bakiye için:

- Ortalama (mean): Müşterilerin ortalama bakiyesi 76,485.889 birimdir.
- Standart Sapma (std): Bakiye miktarı arasında 62,397.405 birimlik büyük bir değişkenlik vardır.
- Min: En düşük bakiye 0 birim, yani bu müşterilerin hiç bakiyesi yoktur.
- 25%: %25'lik dilimdeki müşterilerin bakiyesi 0 birimdir.
- 50%: Ortanca bakiye 97,198.540 birimdir.
- 75%: %75'lik dilimdeki müşterilerin bakiyesi 127,644.240 birim veya daha azdır.
- Max: En yüksek bakiye 250,898.090 birimdir.

Tahmini Maaş İçin:

- Ortalama (mean): Müşterilerin tahmini maaşı ortalama 100,090.240 birimdir.
- Standart Sapma (std): Maaşlar arasında 57,510.493 birimlik bir yayılma vardır.
- Min: En düşük maaş 11.580 birimdir.
- 25%: %25'lik dilimdeki müşterilerin maaşı 51,002.110 birim veya daha azdır.
- 50%: Ortanca değer 100,193.915 birimdir, yani müşterilerin yarısı bu maaşın altındadır.
- 75%: %75'lik dilimdeki müşterilerin maaşı 149,388.247 birim veya daha azdır.
- Max: En yüksek maaş 199,992.480 birimdir.

VERİ ANALİZİ VE GÖRSELLEŞTİRME

Kategorik Değişkenler İçin Tek Değişkenli Analiz

Kategorik değişkenlerin özetini çıkaralım ve her bir değişkenin dağılımını görselleştirelim.

```
def cat_summary(dataframe, col_name, plot=False):
    summary = pd.DataFrame({col_name: dataframe[col_name].value_counts(),
                            "Ratio": 100 * dataframe[col_name].value_counts() / len(dataframe)})

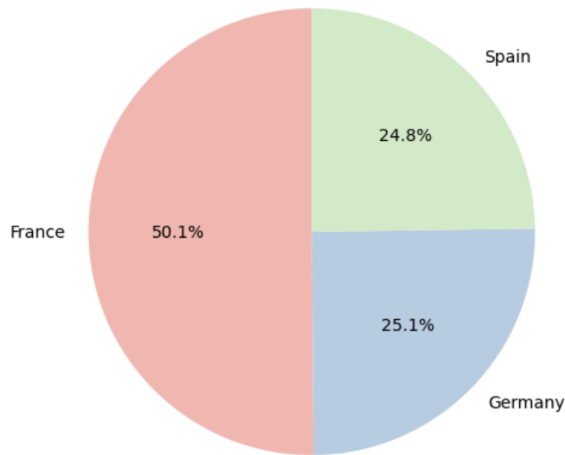
    print(summary)
    print("#####")

    if plot:
        plt.figure(figsize=(6, 6))
        # Pasta grafiği
        sizes = summary[col_name] # Her kategorinin frekansı
        labels = summary.index # Kategoriler
        plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90, colors=plt.cm.Pastel1.colors)
        plt.title(f"{col_name} Dağılımı")
        plt.show()

for col in cat_cols:
    cat_summary(df, col, plot=True)
```

```
ulke Ratio
France  5014 50.140
Germany 2509 25.090
Spain   2477 24.770
#####
```

ulke Dağılımı



Müşterilerin yaşadıkları ülkelere göre dağılımı:

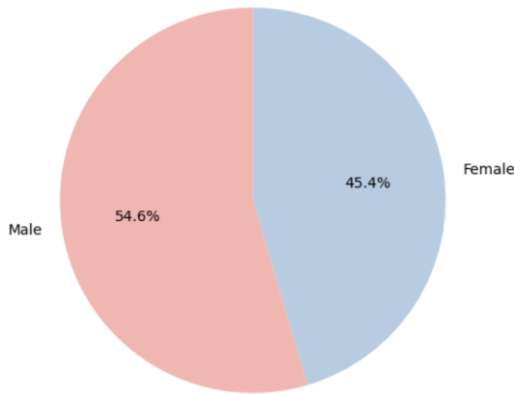
- **France (Fransa):** %50,14 oranıyla en büyük müşteri kitlesini oluşturuyor (**5014 müşteri**).
- **Germany (Almanya):** %25.09 oranıyla ikinci sırada (**2509 müşteri**).
- **Spain (İspanya):** %24,77 oranıyla en az müşteri sayısına sahip (**2477 müşteri**).

Kısaca:

Fransa, müşterilerin yarısını oluşturduğu için veri seti içinde ağırlıklı bir öneme sahiptir.

Almanya ve İspanya'daki müşteriler yaklaşık olarak eşit orana sahip, ancak Fransa'ya kıyasla daha az katkıda bulunuyorlar.

```
cinsiyet  Ratio
cinsiyet
Male      5457 54.570
Female    4543 45.430
#####
cinsiyet Dağılımı
```



Müşterilerin cinsiyetlere göre dağılımı:

- **Male (Erkek):** %54,57 oranıyla çoğunluğu oluşturuyor (**5457 müşteri**).
- **Female (Kadın):** %45,43 oranında müşteriler arasında yer alıyor (**4543 müşteri**).

Kısaca:

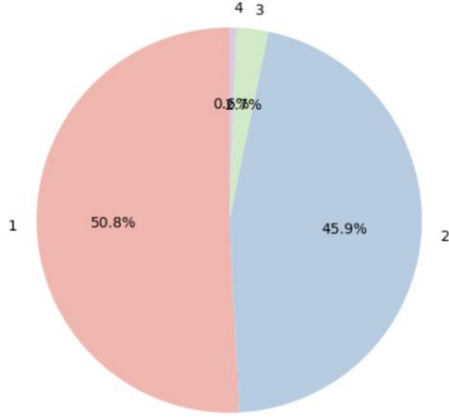
Erkek müşteriler kadın müşterilere göre biraz daha fazladır.

Bu dağılım dengeli bir veri seti sunduğundan dolayı modelleme sırasında cinsiyet değişkeninin etkisi net bir şekilde değerlendirilebilir.

urun_sayısı	urun_sayısı	Ratio
1	5084	50.840
2	4590	45.900
3	266	2.660
4	60	0.600

#####

urun_sayısı Dağılımı



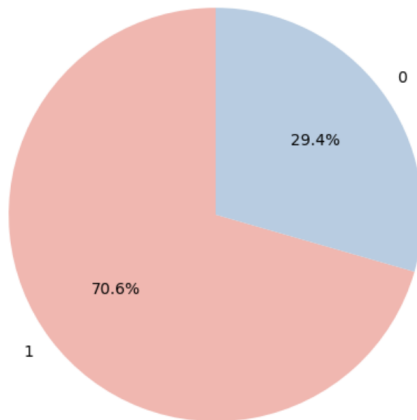
Müşterilerin ürün sayısına göre dağılımı:

- **1 Ürün Kullananlar:** Müşterilerin %50,84'ü (**5084 kişi**) sadece bir ürün kullanıyor. Bu grup, toplam müşterilerin yarısından fazlasını oluşturuyor.
- **2 Ürün Kullananlar:** %45,90 oranıyla (**4590 kişi**) ikinci büyük grubu oluşturuyor. 1 ve 2 ürün kullanan müşteriler, toplam müşterilerin yaklaşık %96,74'ünü kapsıyor.
- **3 Ürün Kullananlar:** %2,66 oranıyla (**266 kişi**) oldukça az bir müşteri kitlesi bu gruba dahil.
- **4 Ürün Kullananlar:** Sadece %0,6 oranında (**60 kişi**) bu gruba dahil. Bu grup en küçük müşteri kitlesini oluşturuyor.

kredi_kartı_var_mı	kredi_kartı_var_mı	Ratio
1	7055	70.550
0	2945	29.450

#####

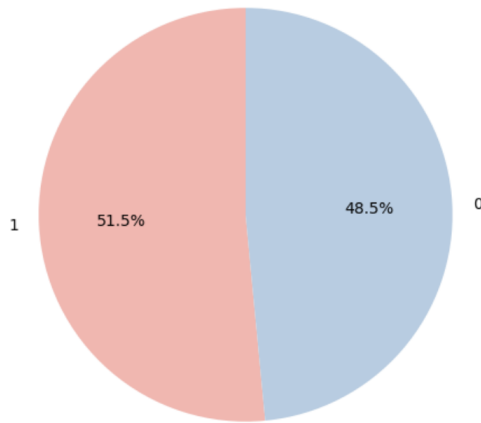
kredi_kartı_var_mı Dağılımı



Müşterilerin kredi kartlarına sahip olup olmadıklarına göre dağılım:

- **Kredi Kartı Sahip Olanlar (1):** Müşterilerin %70,55'i (**7055 kişi**) kredi kartına sahiptir. Bu grup, toplam müşterilerin büyük çoğunluğunu oluşturmaktadır.
- **Kredi Kartı Sahip Olmayanlar (0):** %29,45 oranıyla (**2945 kişi**) kredi kartına sahip değildir.

```
aktif_uye_mi  Ratio
aktif_uye_mi
1             5151 51.510
0             4849 48.490
#####
aktif_uye_mi Dağılımı
```



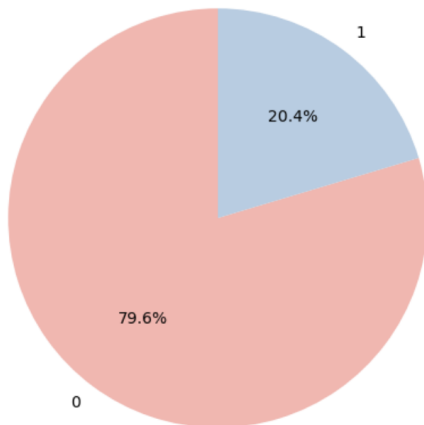
Müşterilerin aktif üye olup olmadıklarına göre dağılım:

- **Aktif Üyeler (1):** Müşterilerin %51,51'i (**5151 kişi**) aktif üye durumundadır.
- **Aktif Olmayan Üyeler (0):** %48,49 oranında (**4849 kişi**) aktif olmayan üyeler bulunmaktadır.

Aktif ve aktif olmayan üyeler arasındaki dağılım dengelidir.

Aktif olmayan üyelerin churn (müşteri kaybı) etme olasılığı daha yüksek olabilir.

```
ayrıldı_mı(churn)  Ratio
ayrıldı_mı(churn)
0                 7963 79.630
1                 2037 20.370
#####
ayrıldı_mı(churn) Dağılımı
```



Müşterilerin ayrılıp ayrılmadıklarına göre dağılım:

- **Ayrılmayan Müşteriler (0):** Müşterilerin %79,63'ü (**7963 kişi**) bankada kalmaya devam etmektedir. Bu grup, müşterilerin büyük çoğunluğunu oluşturmaktadır.
- **Ayrılan Müşteriler (1):** %20,37 oranında (**2037 kişi**) müşteri bankadan ayrılmıştır. Ayrılan müşteri oranı azınlıkta olmakla birlikte, dikkate alınması gereken önemli bir gruptur.

Veri setimiz dengesiz dağılıyor.

Sayısal Değişkenler İçin Tek Değişkenli Analiz

Sayısal değişkenlerin özetini çıkaralım ve her bir değişkenin dağılımını görselleştirelim.

```
### numerik değişken analizi:
def num_summary(dataframe, numerical_col, plot=False):
    quantiles = [0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 0.95, 0.99]
    print(dataframe[numerical_col].describe(quantiles).T)

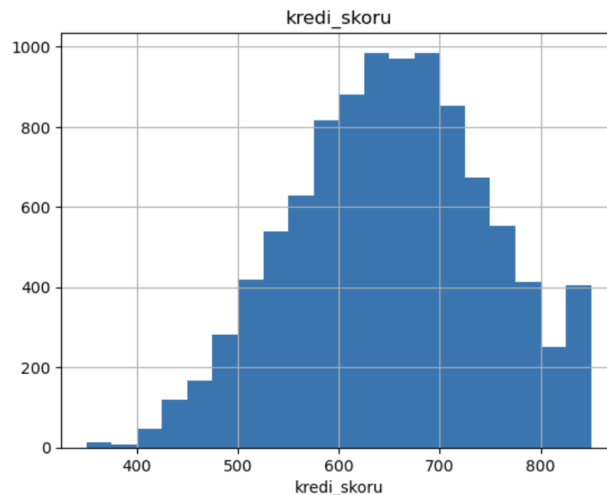
    if plot:
        dataframe[numerical_col].hist(bins=20)
        plt.xlabel(numerical_col)
        plt.title(numerical_col)
        plt.show()

for col in num_cols:
    num_summary(df, col, plot=True)
```

Kredi skoru için:

count	10000.000
mean	650.529
std	96.653
min	350.000
5%	489.000
10%	521.000
20%	566.000
30%	598.700
40%	627.000
50%	652.000
60%	678.000
70%	704.000
80%	735.000
90%	778.000
95%	812.000
99%	850.000
max	850.000

Name: kredi_skoru, dtype: float64

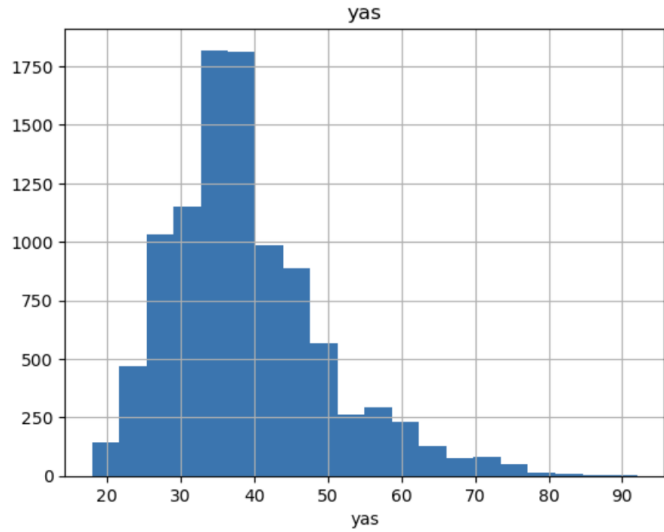


- **Ortalama (mean):** Kredi skoru ortalama **650.53** olup, müşterilerin büyük çoğunluğu bu değere yakın bir kredi skoruna sahiptir.
- **Standart Sapma (std):** Müşterilerin kredi skoru **96.653** civarında bir yayılıma sahiptir.
- **Min-Max:** Kredi skorları 350 ile 850 arasında değişmektedir.

Kredi skoru yaklaşık olarak normal bir dağılıma sahiptir.

Yaş için:

```
count    10000.000
mean      38.922
std       10.488
min       18.000
5%        25.000
10%       27.000
20%       31.000
30%       33.000
40%       35.000
50%       37.000
60%       40.000
70%       42.000
80%       46.000
90%       53.000
95%       60.000
99%       72.000
max       92.000
Name: yas, dtype: float64
```



- **Ortalama (mean):** Müşteri yaş ortalaması yaklaşık **39**'dur.
- **Standart Sapma (std):** Müşterilerin yaşları **10.488** civarında bir yayılıma sahiptir
- **Min-Max:** En küçük yaş **18**, en büyük yaş **92**'dir.

Yaşlar yaklaşık olarak **30-40** yaş aralığında toplanmıştır.

Dağılım hafif sağa çarpıktır. Yaşlı müşteriler daha az.

Müşterilerin şirkette kaldığı ay sayısı için:

```
count    10000.000
mean      5.013
std       2.892
min       0.000
5%        1.000
10%       1.000
20%       2.000
30%       3.000
40%       4.000
50%       5.000
60%       6.000
70%       7.000
80%       8.000
90%       9.000
95%       9.000
99%      10.000
max       10.000
Name: sirkette_kaldigi_ay_sayisi, dtype: float64
```

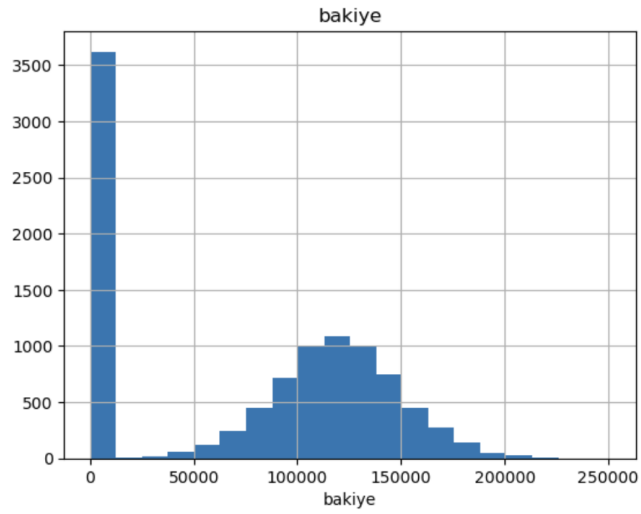


- **Ortalama (mean):** Müşterilerin şirkette kalma süresi ortalama **5** aydır.
- **Standart Sapma (std):** Müşterilerin şirkette kalma süresi **2,89** ay civarında bir yayılıma sahiptir.
- **Min-Max:** En kısa şirkette kalma süresi **0** ay, en uzun süre ise **10** aydır.

Müşteri sayısı kalma süresi boyunca dengeli bir dağılım göstermiştir.

Müşterilerin bakiyesi için:

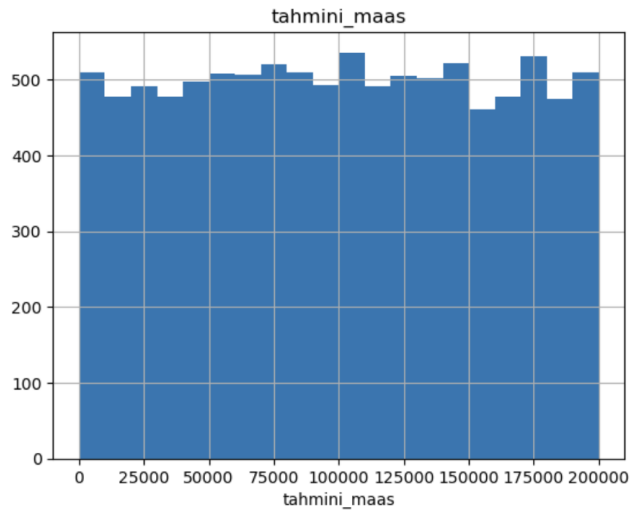
```
count    10000.000
mean      76485.889
std       62397.405
min        0.000
5%         0.000
10%        0.000
20%        0.000
30%        0.000
40%       73080.908
50%       97198.540
60%      110138.926
70%      122029.870
80%      133710.358
90%      149244.792
95%      162711.669
99%      185967.985
max       250898.090
Name: bakiye, dtype: float64
```



- **Ortalama (mean):** Müşteri ortalama bakiyesi yaklaşık **76,485.889**'dur.
- **Standart Sapma (std):** Bakiye miktarı arasında **62,397.405** birimlik bir değişkenlik vardır.
- **Min-Max:** En düşük bakiye **0** birim, en yüksek bakiye ise **250,898.09** birimdir.

Müşterilerin tahmini maaşı için:

```
count    10000.000
mean     100090.240
std       57510.493
min        11.580
5%        9851.819
10%       20273.580
20%       41050.736
30%       60736.079
40%       80238.340
50%      100193.915
60%      119710.038
70%      139432.236
80%      159836.726
90%      179674.704
95%      190155.375
99%      198069.734
max      199992.480
Name: tahmini_maas, dtype: float64
```



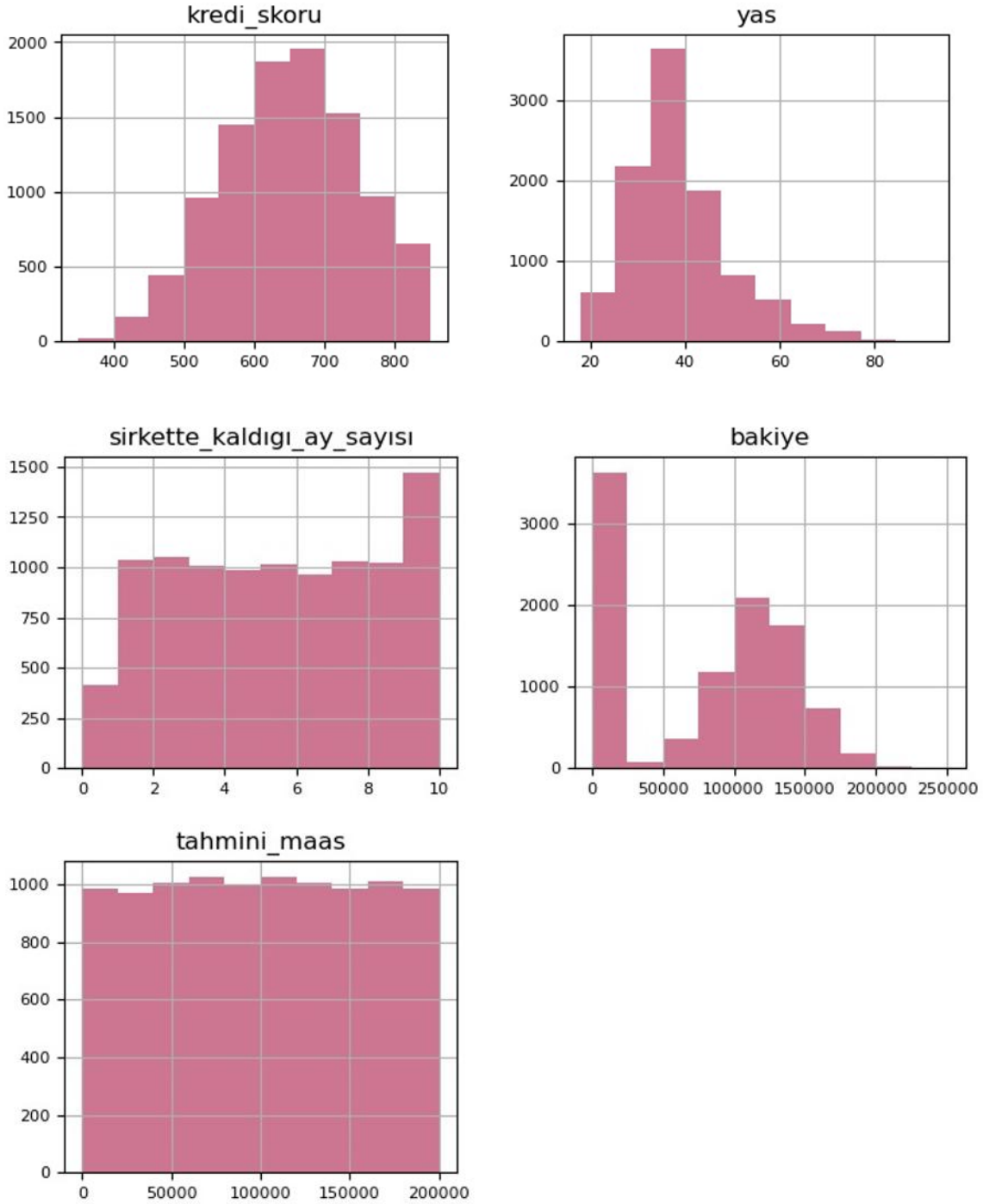
- **Ortalama (mean):** Müşteri ortalama maaş tahmini yaklaşık **100,090.240**'dır.
- **Standart Sapma (std):** Maaş tahmin miktarı arasında **57,510.493** birimlik bir değişkenlik vardır.
- **Min-Max:** En düşük maaş tahmini **11.580** birim, en yüksek bakiye ise **199.992,48** birimdir.

Maaşlar genelde eşit aralıklarla dağılmış gibi görünüyor. Ancak bu dağılımın “uniform” bir yapıya yakın olduğunu gösteriyor.

Çoğu müşteri orta maaş aralıklarında yoğunlaşmış olsa da uç değerler (çok düşük veya çok yüksek maaşlar) vardır.

Bir de grafiklere bir arada bakalım.

```
df[num_cols].hist(figsize=(8, 10), bins=10, xlabelsize=8, ylabelsize=8, color="palevioletred")
```



Kredi skorunu segmentlere ayırarak (düşük, orta, yüksek) müşterilerin finansal durumlarının analizini ve kredi segmentlerine göre bakiye dağılımını görselleştirelim.

```
# Kredi skorunu segmentlere ayır
kopya=df.copy()
bins = [0, 500, 700, 850] # Aralıklar: 0-500, 501-700, 701-850
labels = ['Düşük', 'Orta', 'Yüksek']
kopya['kredi_segmenti'] = pd.cut(kopya['kredi_skoru'], bins=bins, labels=labels)

# Segmentlere göre analiz
segment_analysis = kopya.groupby('kredi_segmenti').mean()

print("\nSegmentlere Göre Analiz:")
segment_analysis
```

Segmentlere Göre Analiz:

	kredi_skoru	yas	sirkette_kaldigi_ay_sayisi	bakiye	tahmini_maas
kredi_segmenti					
Düşük	463.580	38.950	5.075	80241.272	103890.178
Orta	614.641	38.959	4.995	75932.588	99836.883
Yüksek	760.989	38.841	5.035	76819.154	100255.441

Segmentasyon:

Kredi skorları 3 gruba ayrılmıştır:

- Düşük: 0-500
- Orta: 501-700
- Yüksek: 701-850

Tablonun içeriği:

Segmentlere göre ortalama değerler hesaplanmıştır.

- **Kredi Skoru:** Her segmentin ortalama kredi skorunu temsil eder.
- **Yaş:** Segmentlerdeki müşterilerin ortalama yaşı benzerdir. (38-39)
- **Şirkette Kaldığı Ay Sayısı:** Tüm segmentlerde ortalama şirket bağlılığı 5 aydır.
- **Bakiye:** Düşük kredi skoru segmentinde ortalama bakiye 80,241.27 birim ile daha yüksektir. Orta ve yüksek kredi segmentlerinde bu değer daha düşüktür.
- **Tahmini Maaş:** Düşük kredi skoruna sahip segmentte maaşlar daha yüksektir.

Kısaca, bu değişkenlerin kredi skoru ile doğrusal bir ilişkisi bulunmamaktadır.

Daha detaylı incelemek için pasta grafiği çizelim.


```
# Segmentlerin oranlarını hesaplama
segment_counts = kopya['kredi_segmenti'].value_counts()

# Pasta grafiği çizimi
segment_counts.plot.pie(
    autopct="%.1f%%", # Yüzde değerlerini göster
    startangle=90, # Grafiği 90 dereceden başlat
    colors=sns.color_palette("pastel"), # Pastel renkler
    ylabel="", # Y eksen etiketi olmadan
    title="Kredi Skoru Segmentlerinin Dağılımı"
)
plt.show()
```



Orta Segment (501-700):

En büyük segmenttir, müşterilerin **%62,4'ü** bu grupta yer almaktadır.

Orta düzey kredi skoru olan müşteriler, bankanın müşteri portföyünün büyük bir kısmını oluşturur.

Yüksek Segment (701-850):

İkinci büyük segmenttir, müşterilerin **%31,2'si** bu gruba dahildir.

Yüksek kredi skoru olan müşteriler, daha güvenilir ve potansiyel olarak daha karlı bir müşteri kitlesini temsil eder.

Düşük Segment (0-500):

En küçük segmenttir, müşterilerin yalnızca **%6,4'ü** bu gruba dahildir.

Bu grup, finansal açıdan daha riskli olabilir ve bankanın dikkat etmesi gereken bir segmenttir.

HEDEF DEĞİŞKEN ANALİZİ

Kategorik Değişkenlerin Churn'e Göre Analizi

```
import pandas as pd
import matplotlib.pyplot as plt

def target_summary_with_cat(dataframe, target, categorical_col, plot=False):
    """
    Bağımlı değişken kategorik olduğunda, bağımsız değişkenin her bir kategorisine göre bağımlı değişkenin
    dağılımını gösteren bir özet çıkarır ve isteğe bağlı olarak grafik çizer.

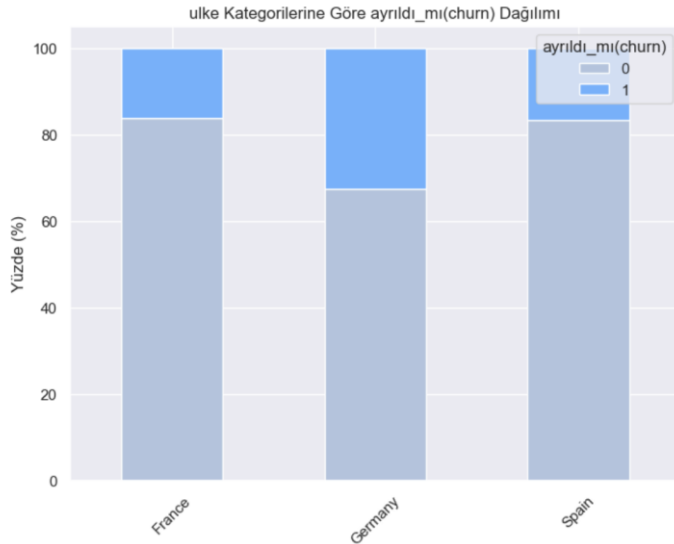
    :param dataframe: pandas DataFrame
    :param target: kategorik bağımlı değişken
    :param categorical_col: analiz edilecek kategorik bağımsız değişken
    :param plot: bool, True ise grafik çizer
    """
    # Çapraz tablo ile bağımsız ve bağımlı değişken ilişkisini inceleme
    cross_tab = pd.crosstab(dataframe[categorical_col], dataframe[target], normalize="index") * 100
    print(f"--- {categorical_col} için Churn Dağılımı ---")
    print(cross_tab)
    print("\n")

    if plot:
        cross_tab.plot(kind="bar", stacked=True, figsize=(8, 6), color=["#b0c4de", "#66b2ff"])
        plt.title(f"{categorical_col} Kategorilerine Göre {target} Dağılımı")
        plt.ylabel("Yüzde (%)")
        plt.xlabel(categorical_col)
        plt.legend(title=target)
        plt.xticks(rotation=45)
        plt.show()

# Tüm kategorik değişkenler için analiz ve grafik
for col in cat_cols:
    target_summary_with_cat(df, "ayrıldı_mı(churn)", col, plot=True)
```

Ülkeler bazında müşteri ayrılma oranlarını (churn) yüzdesel olarak gösterme:

```
--- ülke için Churn Dağılımı ---
ayrıldı_mı(churn)    0    1
ülke
France             83.845 16.155
Germany            67.557 32.443
Spain              83.327 16.673
```



1. France (Fransa):

- Ayrılmayan müşteriler: %83,85
- Ayrılan müşteriler: %16,15

2. Germany (Almanya):

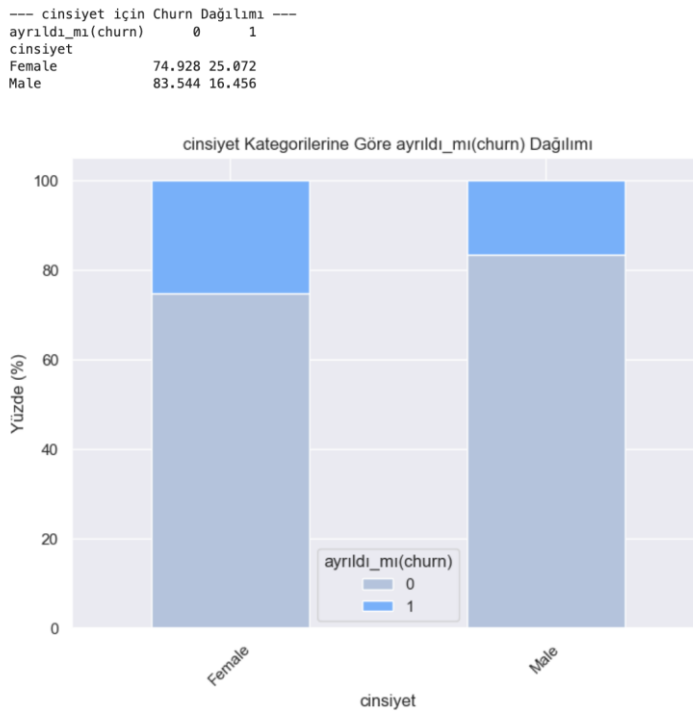
- Ayrılmayan müşteriler: %67,56
- Ayrılan müşteriler: %32,44
- En yüksek ayrılma oranı bu ülkede.

3. Spain (İspanya):

- Ayrılmayan müşteriler: %83,33
- Ayrılan müşteriler: %16,67

Germany (Almanya), diğer ülkelere kıyasla ayrılma oranı en yüksek ülkedir.

Cinsiyet bazında müşteri ayrılma oranlarını (churn) yüzdesel olarak gösterme:



1. Female (Kadın):

- Ayrılmayan müşteriler: %74,93
- Ayrılan müşteriler: %25,07
- Kadın müşterilerin ayrılma oranı erkeklere göre daha yüksek.

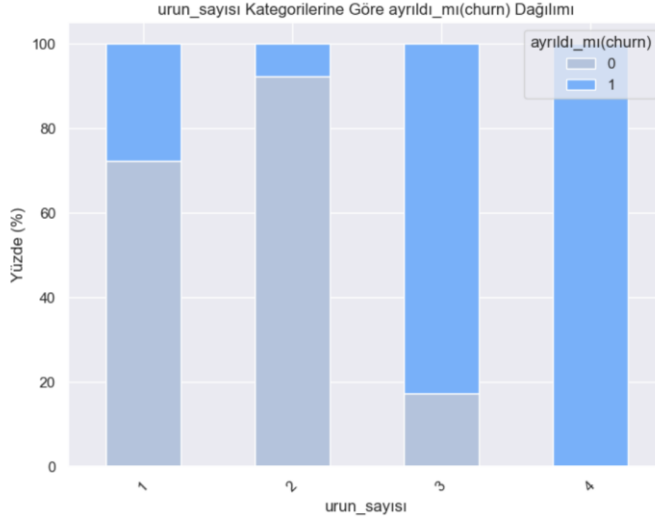
2. Male (Erkek):

- Ayrılmayan müşteriler: %83,54
- Ayrılan müşteriler: %16,46

Kadın müşteriler, erkeklere kıyasla daha yüksek bir ayrılma oranına sahiptir (%25.07).

Müşteri başına ürün sayısına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

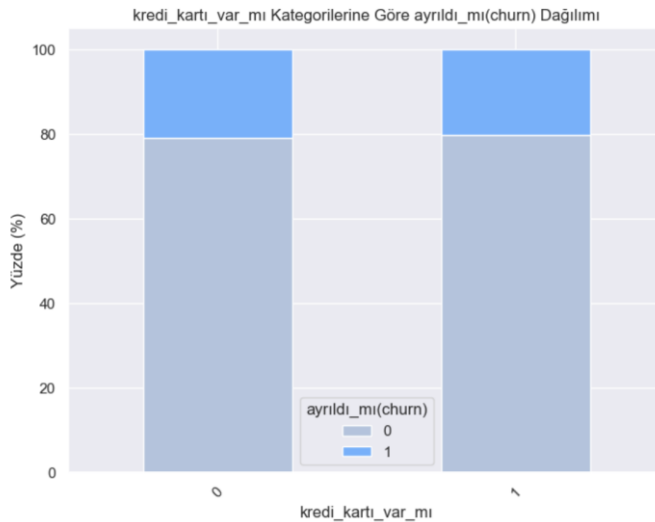
```
--- urun_sayisi için Churn Dağılımı ---  
ayrildi_mi(churn) 0 1  
urun_sayisi  
1 72.286 27.714  
2 92.418 7.582  
3 17.293 82.707  
4 0.000 100.000
```



- 2 ürün sahibi müşteriler, en düşük ayrılma oranına sahiptir.
- 3 ve 4 ürün sahibi müşteriler, ayrılma riski en yüksek olan gruplardır.

Müşterilerin kredi kartı olup olmadığına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

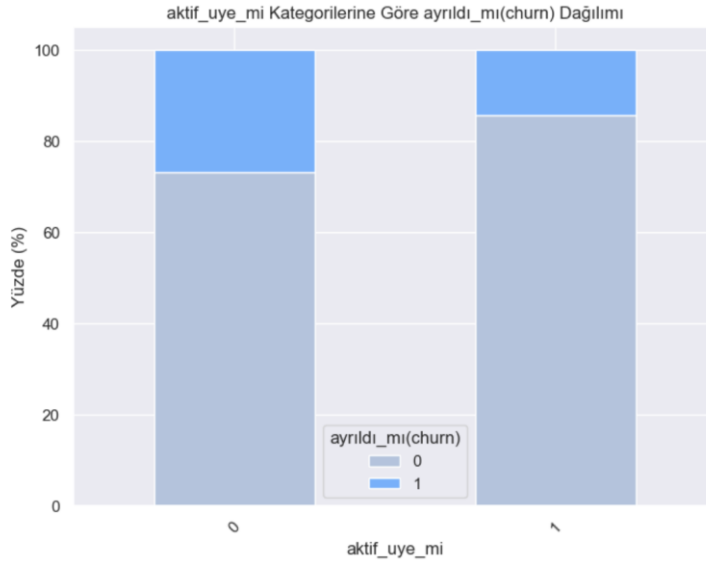
```
--- kredi_karti_var_mi için Churn Dağılımı ---  
ayrildi_mi(churn) 0 1  
kredi_karti_var_mi  
0 79.185 20.815  
1 79.816 20.184
```



- Kredi kartı sahipliği ile müşteri ayrılma oranı arasında anlamlı bir fark yoktur.
- Her iki grup da yaklaşık %20 oranında ayrılmaktadır.
- Kredi kartı sahipliği, ayrılma eğilimini doğrudan etkilemeyen bir faktör gibi görünüyor.

Müşterilerin aktif üye olup olmamasına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

```
--- aktif_uye_mi için Churn Dağılımı ---
ayrıldı_mi(churn)    0    1
aktif_uye_mi
0      73.149 26.851
1      85.731 14.269
```



- Aktif üyeler (1), daha düşük bir ayrılma oranına sahiptir (%14,27).
- Aktif olmayan üyeler (0), daha yüksek bir ayrılma oranına sahiptir (%26,85).

Sayısal Değişkenlerin Churn'e Göre Analizi

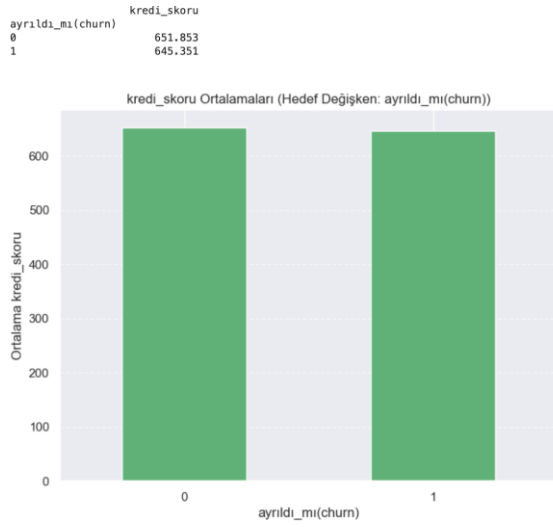
```
def target_summary_with_num(dataframe, target, numerical_col, plot=False):
    """
    Sayısal bir değişkenin hedef değişkene göre özetini çıkarır ve opsiyonel olarak grafik çizer.

    :param dataframe: pandas DataFrame
    :param target: kategorik hedef değişken
    :param numerical_col: analiz edilecek sayısal bağımsız değişken
    :param plot: bool, True ise grafik çizer
    """
    # Hedef değişkene göre grupta ve ortalama hesaplama
    summary = dataframe.groupby(target).agg({numerical_col: "mean"})
    print(summary, end="\n\n")

    # Grafik çizimi
    if plot:
        summary.plot(kind="bar", legend=False, color="mediumseagreen", figsize=(8, 6))
        plt.title(f"{numerical_col} Ortalamaları (Hedef Değişken: {target})")
        plt.ylabel(f"Ortalama {numerical_col}")
        plt.xlabel(target)
        plt.xticks(rotation=0) # X eksen etiketlerini düz yaz
        plt.grid(axis="y", linestyle="--", alpha=0.7)
        plt.show()

    # Tüm sayısal sütunlar için analiz ve grafik
    for col in num_cols:
        target_summary_with_num(df, "ayrıldı_mi(churn)", col, plot=True)
```

Müşterilerin kredi skorları ortalamalarına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

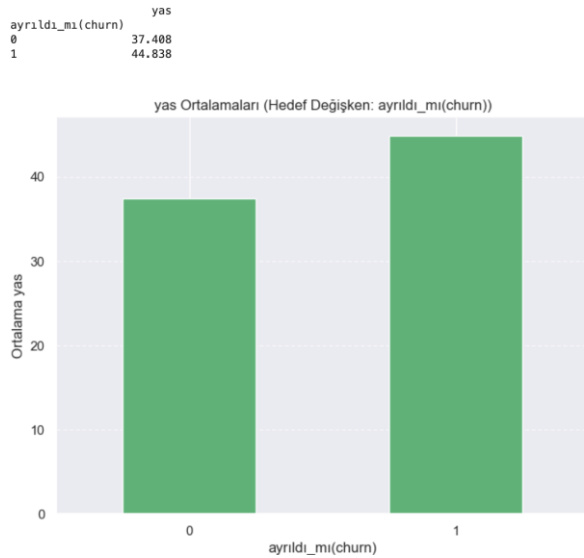


Ayrılmayan Müşteriler (0): Ortalama kredi skoru: 651.85

Ayrılan Müşteriler (1): Ortalama kredi skoru: 645.35

- Ayrılan müşterilerin ortalama kredi skoru, ayrılmayan müşterilere göre biraz daha düşüktür. Fakat fark oldukça küçük ve kredi skoru ile ayrılma arasında güçlü bir ilişki olmadığını düşündürmektedir.

Müşterilerin yaş ortalamalarına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

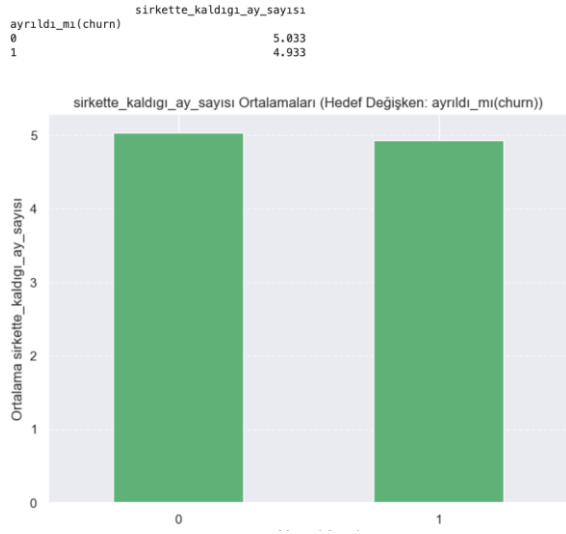


Ayrılmayan Müşteriler (0): Ortalama yaş: 37.41

Ayrılan Müşteriler (1): Ortalama yaş: 44.84

- Ayrılan müşterilerin yaş ortalaması ayrılmayan müşterilere göre daha yüksektir.
- Yaş ilerledikçe müşteri ayrılma riski artmaktadır.

Müşterilerin şirkette kaldığı ay sayısı ortalamalarına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

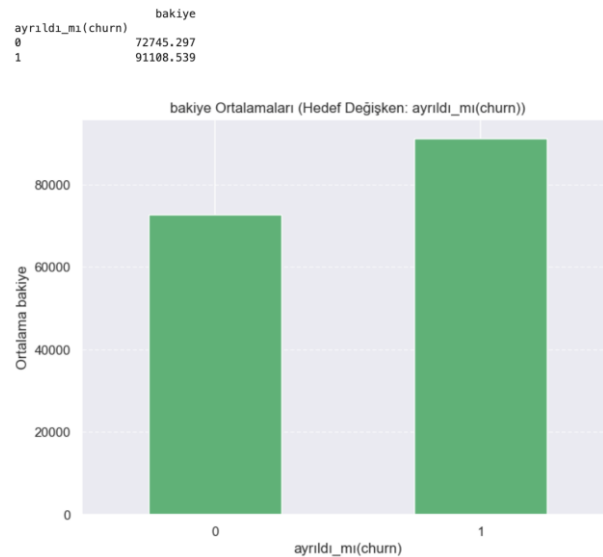


Ayrılmayan Müşteriler (0): Ortalama şirkette kalma süresi: 5.033 ay

Ayrılan Müşteriler (1): Ortalama şirkette kalma süresi: yaklaşık 5 ay

- Ayrılmayan ve ayrılan müşteriler arasında şirket içinde kalma süresi bakımından çok küçük bir fark vardır.
- Ayrılma riski, müşterilerin şirkette kaldığı süreyle güçlü bir ilişki göstermemektedir.

Müşterilerin bakiye ortalamalarına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:



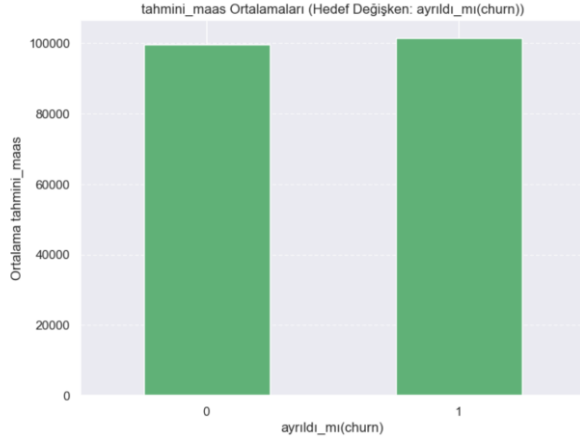
Ayrılmayan Müşteriler (0): Ortalama bakiye: 72,745.30

Ayrılan Müşteriler (1): Ortalama bakiye: 91,108.54

- Ayrılan müşteriler, ayrılmayanlara göre daha yüksek bir ortalama bakiyeye sahiptir.

Müşterilerin tahmini maaş ortalamalarına göre ayrılma oranlarını (churn) yüzdesel olarak gösterme:

```
ayrildi_mi(churn)    tahmini_maas
0                    99738.392
1                   101465.678
```



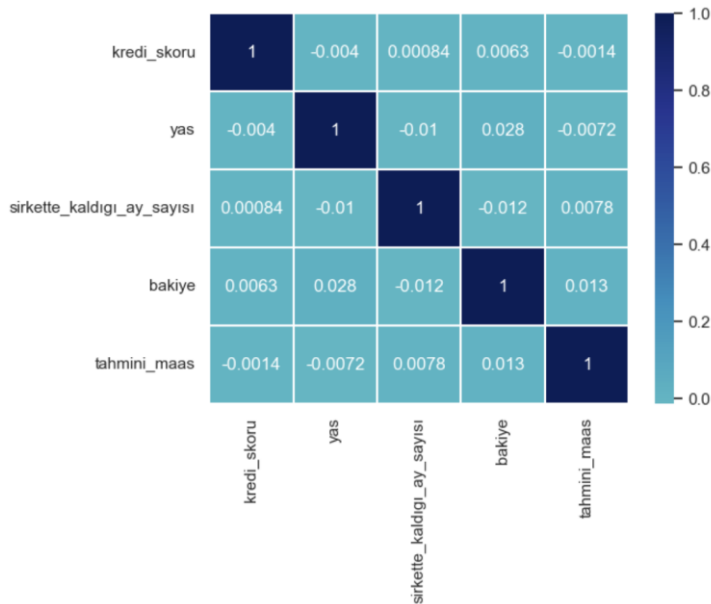
Ayrılmayan Müşteriler (0): Ortalama tahmini maaş: 99,738.40

Ayrılan Müşteriler (1): Ortalama tahmini maaş: 101,465.678

- Ayrılan müşteriler, ayrılmayanlara göre daha yüksek bir ortalama tahmini maaşa sahiptir. Ancak fark oldukça küçük olduğu için maaşın müşteri ayrılma üzerinde güçlü bir etkisi olduğu söylenemez.

KORELASYON

```
sns.set()
corr = df[num_cols].corr()
ax = sns.heatmap(corr,
                  center=0,
                  annot=True,
                  linewidths=.2,
                  cmap="YlGnBu")
plt.show()
```



Değişkeler arasındaki korelasyon değerleri oldukça düşük diyebiliriz.

Örneğin:

- Kredi skoru ile diğer değişkenler arasında neredeyse hiç ilişki yok.
- Yaş ve bakiye arasında çok hafif bir pozitif korelasyon vardır.
- Tahmini maaş ve diğer değişkenler arasında sıfıra yakın bir ilişki bulunuyor.

✓ Sayısal değişkenler arasında güçlü bir ilişki görünmüyor. Bu değişkenler birbirini fazla etkilemiyor ve her birinin modelleme sürecinde bağımsız olarak değerlendirilebileceğini gösteriyor.

Şimdi aralarında en düşük ilişki bulunan iki değişkene bakalım:

```
df.corr(method = 'pearson', numeric_only = True).unstack().idxmin()  
( 'sirkette_kaldığı_ay_sayısı', 'bakiye' )
```

- Şirkette kaldığı ay sayısı ve bakiye arasında çok zayıf bir ilişki olduğunu söyleyebiliriz.

Sayısal değişkenlerin hedef değişkenle (churn) olan korelasyonlarını en çoktan en aza doğru görelim:

```
df[num_cols].corrwith(df["ayrıldı_mı(churn)"]).sort_values(ascending=False)  
  
yas                0.285  
bakiye             0.119  
tahmini_maas       0.012  
sirkette_kaldığı_ay_sayısı -0.014  
kredi_skoru        -0.027  
dtype: float64
```

- **Yaş:** En yüksek pozitif korelasyona sahip değişkendir (0.285). Bu, yaş arttıkça müşterinin churn etme olasılığının da arttığını gösterebilir.
- **Bakiye:** Pozitif bir korelasyona sahiptir (0.119). Burada bakiyenin artmasının churn ile zayıf bir şekilde ilişkili olduğunu söyleyebiliriz.
- **Tahmini Maaş:** Korelasyonu düşüktür (0.012). Bu değişkenin churn üzerinde anlamı bir etkiye sahip olmadığını söyleyebiliriz.
- **Şirkette Kalma Süresi:** Negatif korelasyona sahiptir (-0.014). Bu şirkette daha uzun süre kalan müşterilerin churn etme olasılığının biraz daha düşük olabileceğini gösteriyor.
- **Kredi Skoru:** Negatif korelasyona sahiptir (-0.027). Etkisi oldukça zayıftır.

Kısaca, yaş ve bakiye değişkenleri, diğer ilişkilere göre, churn ile daha güçlü bir ilişkiye sahiptir. Bu model oluşturma sırasında önemli bir özellik olabilir.

Detaylı Görselleştirme

Scatter plot

```
from itertools import combinations

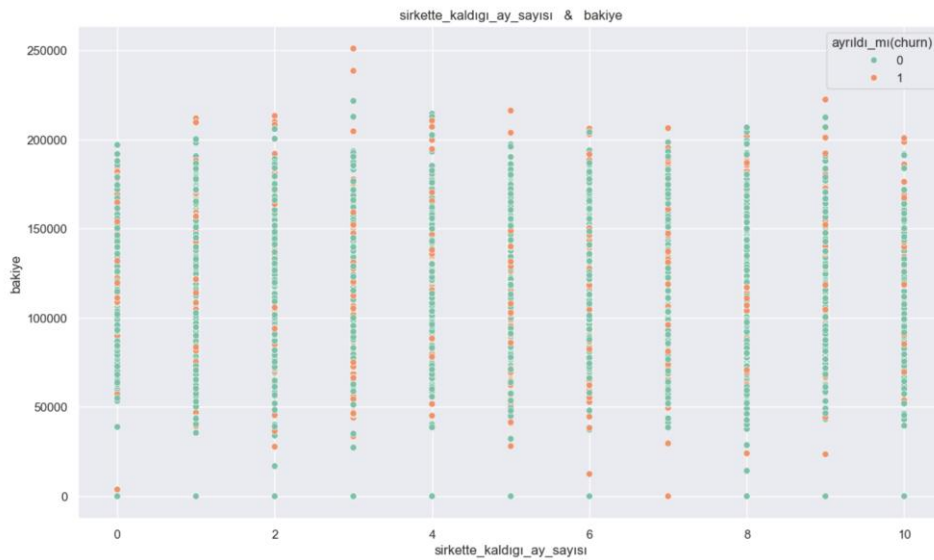
def numcols_target_corr(df, target="ayrıldı_m1(churn)":

    # Sayısal sütunların ikili kombinasyonlarını oluştur
    numvar_combinations = list(combinations(num_cols, 2))

    # Her kombinasyon için scatter plot oluştur
    for item in numvar_combinations:
        plt.subplots(figsize=(14, 8))
        sns.scatterplot(x=df[item[0]], y=df[item[1]], hue=df[target], palette="Set2")\
            .set_title(f'{item[0]} & {item[1]}')
        plt.grid(True)
        plt.show()

# Fonksiyonu çağır
numcols_target_corr(df, target="ayrıldı_m1(churn)")
```

Oluşturulan grafiklerden birkaçını yorumlayalım.



- Şirkette kaldığı ay sayısı değişkeni boyunca ayrılan (turuncu) ve ayrılmayan (yeşil) müşterilerin dağılımı benzerdir. Şirkette kaldığı ay sayısı değişkeninin churn üzerinde güçlü bir etkiye sahip olmadığını söyleyebiliriz.
- Yüksek bakiye seviyelerinde turuncu noktalar daha fazladır bu nedenle, daha yüksek bakiyelere sahip müşterilerde ayrılma (churn etme) olasılığı daha fazla görünüyor diyebiliriz.

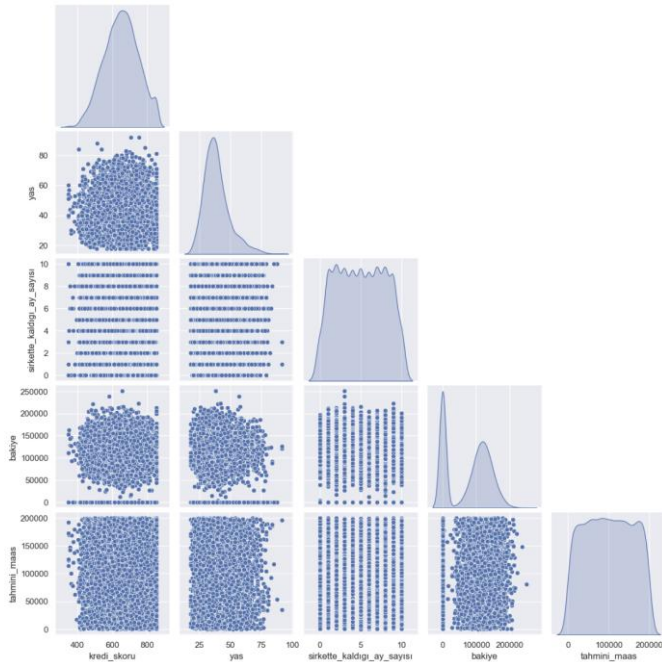


- Yaş büyüdükçe, ayrılan müşterilerin sayısında artış gözlemleniyor. Özellikle 40-60 yaş aralığındaki müşterilerin ayrılma oranı daha yüksek görünüyor.
- Yüksek bakiyeye sahip müşteriler arasında ayrılan müşteriler biraz daha fazla gözüküyor. Yüksek bakiyeye sahip müşterilerin ayrılma olasılığının daha yüksek olabileceğini söyleyebiliriz.
- Ayrılmayan (churn:0) müşteriler hem düşük hem yüksek bakiyelerde yoğunlaşırken, ayrılan müşteriler genellikle yüksek yaş ve bakiye kategorilerinde yer alıyor.

Pair Plot

Bu kod tüm numerik değişkenlerin kendi aralarındaki ilişkilerini ve dağılımlarını göstermektedir.

```
# Sayısal - Sayısal İlişki: Scatterplot
sns.pairplot(df[num_cols], diag_kind='kde', corner=True)
plt.show()
```

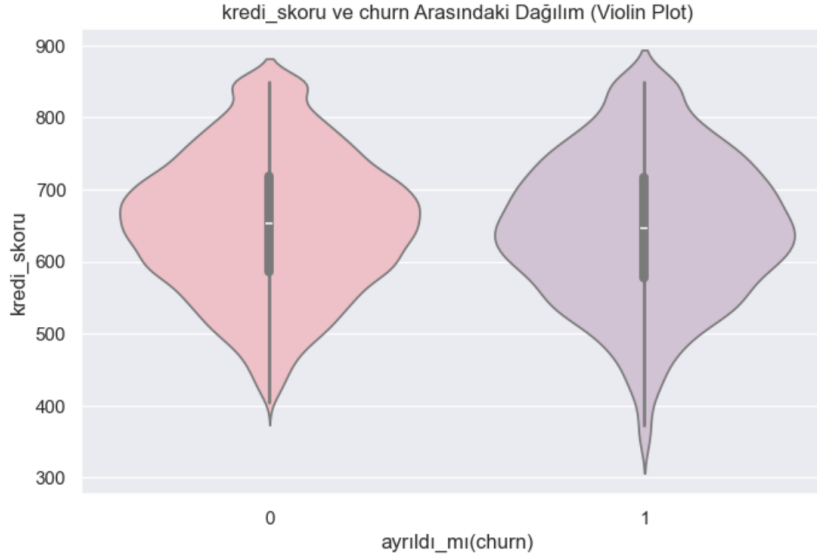


Bu grafikteki ilişkiler, numerik değişkenlerin çoğunlukla birbirinden bağımsız olduğunu göstermektedir.

Violin Plot

```
# Violin plot çizdirme

for degisken in num_cols:
    plt.figure(figsize=(8, 5))
    sns.violinplot(x="ayrıldı_mı(churn)", y=degisken, data=df, hue="ayrıldı_mı(churn)", palette=["lightpink", "thistle"], legend=False)
    plt.title(f"{degisken} ve churn Arasındaki Dağılım (Violin Plot)")
    plt.show()
```



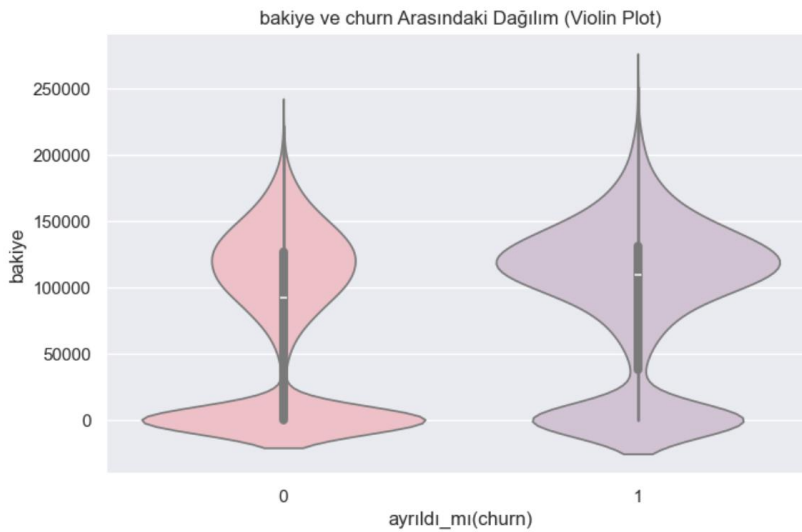
1. Churn etmeyenler (0):

- Kredi skoru dağılımı geniştir.

2. Churn edenler (1):

- Kredi skoru daha dar ve simetrik gözükmemekte.
- Her iki grup için de medyan (ortanca değer) oldukça benzerdir. Bu, churn edenler ve etmeyenler arasında kredi skoru açısından belirgin bir fark olmadığını göstermektedir.

Kısaca: Kredi skoru ile churn arasında belirgin bir ilişki gözükmemektedir.



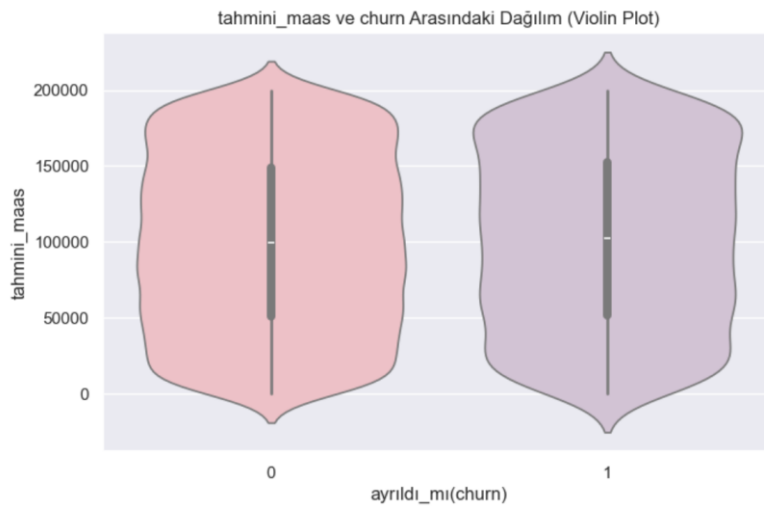
3. Churn etmeyenler (0):

- Dağılımın en alt kısmında belirgin bir yoğunluk bulunmaktadır.

4. Churn edenler (1):

- Dağılım daha geniştir.
- Churn etmeyenlerin (Churn 0) medyanı churn edenlere göre daha düşük bir noktadadır. Bu, daha yüksek bakiyeye sahip müşterilerin churn etme eğiliminde olabileceğini gösterebilir.

Kısaca: Bakiye ile churn arasında bir ilişki olabileceği söylenebilir.



- Tahmini maaş ile churn arasında belirgin bir fark bulunmamaktadır. Ancak churn edenlerin maaşları biraz daha yüksek gibi görünmektedir.
- Her iki grubun medianı birbirine çok yakındır. Bu, maaş değişkeninin churn üzerinde belirgin bir etkiye sahip olmadığını gösterebilir.

FEATURE ENGINEERING

Base Model:

Buraya kadar yaptığımız işlemlerin modele katkısını gözlemlemek için bir base model kuralım. Analiz gidişatımıza karar vermemizde yardımcı olacaktır. Verimizin kopyası üzerinde gidelim.

```
dff = df.copy()
cat_cols = [col for col in cat_cols if col not in ["ayrıldı_m1(churn)"]]
cat_cols

def one_hot_encoder(dataframe, categorical_cols, drop_first=False):
    dataframe = pd.get_dummies(dataframe, columns=categorical_cols, drop_first=drop_first)
    return dataframe
dff = one_hot_encoder(dff, cat_cols, drop_first=True)

y = dff["ayrıldı_m1(churn)"]
X = dff.drop(["ayrıldı_m1(churn)"], axis=1)

models = [('LR', LogisticRegression(random_state=12345)),
          ('KNN', KNeighborsClassifier()),
          ('CART', DecisionTreeClassifier(random_state=12345)),
          ('RF', RandomForestClassifier(random_state=12345)),
          ('SVM', SVC(gamma='auto', random_state=12345)),
          ('XGB', XGBClassifier(random_state=12345)),
          ('LightGBM', LGBMClassifier(random_state=12345)),
          ('CatBoost', CatBoostClassifier(verbose=False, random_state=12345))]

for name, model in models:
    cv_results = cross_validate(model, X, y, cv=10, scoring=["accuracy", "f1", "roc_auc", "precision", "recall"])
    print(f"##### {name} #####")
    print(f"Accuracy: {round(cv_results['test_accuracy'].mean(), 4)}")
    print(f"Auc: {round(cv_results['test_roc_auc'].mean(), 4)}")
    print(f"Recall: {round(cv_results['test_recall'].mean(), 4)}")
    print(f"Precision: {round(cv_results['test_precision'].mean(), 4)}")
    print(f"F1: {round(cv_results['test_f1'].mean(), 4)}")
```

LR

Accuracy: 0.7902

Auc: 0.6705

Recall: 0.0579

Precision: 0.3939

F1: 0.1009

KNN

Accuracy: 0.7664

Auc: 0.5297

Recall: 0.0903

Precision: 0.2765

F1: 0.1361

CART

Accuracy: 0.7929

Auc: 0.6871

Recall: 0.5086

Precision: 0.4916

F1: 0.4996

LightGBM

Accuracy: 0.8619

Auc: 0.8616

Recall: 0.4993

Precision: 0.7397

F1: 0.5947

CatBoost

Accuracy: 0.8648

Auc: 0.8675

Recall: 0.4978

Precision: 0.7576

F1: 0.5993

RF

Accuracy: 0.8619

Auc: 0.8559

Recall: 0.4742

Precision: 0.7584

F1: 0.5821

SVM

Accuracy: 0.7963

Auc: 0.5042

Recall: 0.0

Precision: 0.0

F1: 0.0

XGB

Accuracy: 0.8523

Auc: 0.8466

Recall: 0.4933

Precision: 0.695

F1: 0.5749

Model performansları bu şekilde. Aykırı değer analizi yapalım. Eğer aykırılıklar varsa onları belirli bir aralıkta baskılayıp model performanslarına tekrar bakalım.

Box-plot ile aykırı değer incelemesi:

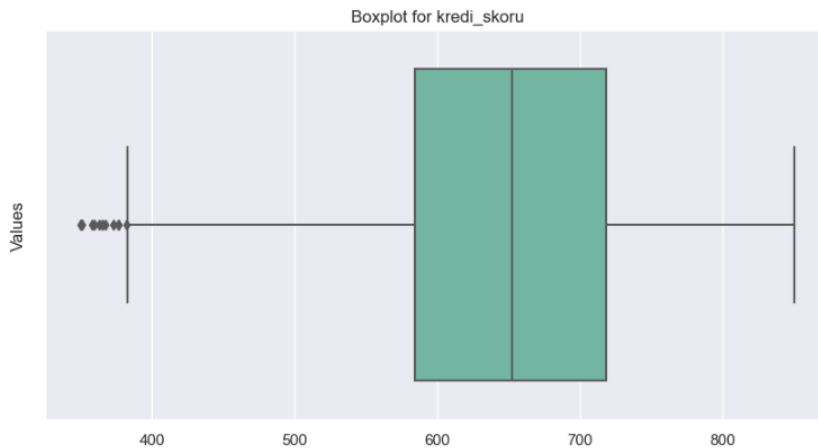
```
def draw_boxplots_for_numeric_columns(df, figsize=(10, 5), palette="Set2"):
    """
    Tüm numerik değişkenler için kutu grafiği çizer.

    Args:
        df (pd.DataFrame): Veri çerçevesi (DataFrame).
        figsize (tuple): Grafik boyutu (default: (10, 5)).
        palette (str): Grafik renk paleti (default: "Set2").
    """

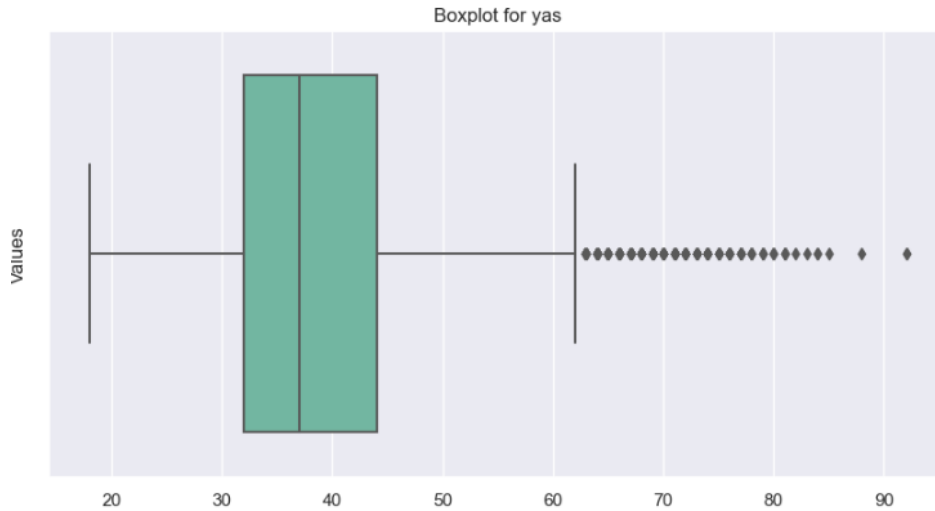
    for col in num_cols:
        plt.figure(figsize=figsize)
        sns.boxplot(data=df, x=col, palette=palette)
        plt.title(f'Boxplot for {col}')
        plt.xlabel('')
        plt.ylabel('Values')
        plt.show()

draw_boxplots_for_numeric_columns(df)
```

Sadece 'kredi_skoru' ve 'yas' değişkenlerimizde aykırılıklar var, inceleyelim.



1. Çoğu kredi skoru, 600-700 aralığında yoğunlaşmıştır. Bu, genel olarak ortalama veya iyi bir kredi geçmişine sahip bireylerin bulunduğu bir veri setini işaret edebilir.
2. 400'ün altındaki aykırı değerler, finansal zorluk çeken veya kredi notu çok düşük olan bireyleri temsil edebilir.
3. Veri setinin üst sınırında aykırı değer bulunmaması, yüksek kredi skorlarının veride daha istikrarlı olduğunu gösteriyor.



- 60 yaşın üstünde birçok aykırı değer var, bunlar 70 ile 90 arasında dağılmış durumda.
- Bu aykırı değerler, veri setinde daha az sıklıkla görülen yaşları temsil eder.
- Veri seti, yaş açısından yukarı doğru çarpık (pozitif çarpıklık) bir dağılıma sahip gibi görünmektedir, çünkü üst uçta daha fazla aykırı değer bulunmaktadır.

Fonksiyon ile aykırı değer analizi:

```
def outlier_thresholds(dataframe, col_name, q1=0.25, q3=0.75):
    quartile1 = dataframe[col_name].quantile(q1)
    quartile3 = dataframe[col_name].quantile(q3)
    interquartile_range = quartile3 - quartile1
    up_limit = quartile3 + 1.5 * interquartile_range
    low_limit = quartile1 - 1.5 * interquartile_range
    return low_limit, up_limit

def check_outlier(dataframe, col_name):
    low_limit, up_limit = outlier_thresholds(dataframe, col_name)
    if dataframe[(dataframe[col_name] > up_limit) | (dataframe[col_name] < low_limit)].any(axis=None):
        return True
    else:
        return False

for col in num_cols:
    print(col, check_outlier(dff, col))
```

```
kredi_skoru True
yas True
sirkette_kaldigi_ay_sayisi False
bakiye False
tahmini_maas False
```

Sadece 'kredi_skoru' ve 'yas' değişkenlerimizde aykırılıklar var.


```
def replace_with_thresholds(dataframe, variable, q1=0.25, q3=0.75):
    low_limit, up_limit = outlier_thresholds(dataframe, variable, q1=0.25, q3=0.75)
    dataframe.loc[(dataframe[variable] < low_limit), variable] = low_limit
    dataframe.loc[(dataframe[variable] > up_limit), variable] = up_limit

# Aykırı Değer Analizi ve Baskılama İşlemi
for col in num_cols:
    print(col, check_outlier(dff, col))
    if check_outlier(dff, col):
        replace_with_thresholds(dff, col) #2 kere çalıştırın

kredi_skoru False
yas False
sirkette_kaldigi_ay_sayısı False
bakiye False
tahmini_maas False
```

Aykırı değerleri baskıladık ve aykırılık gözüküyor. Şimdi model performansını tekrar inceleyelim.

```
##### LR #####
Accuracy: 0.7902
Auc: 0.6705
Recall: 0.0579
Precision: 0.3939
F1: 0.1009
##### KNN #####
Accuracy: 0.7664
Auc: 0.5297
Recall: 0.0903
Precision: 0.2765
F1: 0.1361
##### CART #####
Accuracy: 0.7929
Auc: 0.6871
Recall: 0.5086
Precision: 0.4916
F1: 0.4996
##### RF #####
Accuracy: 0.8619
Auc: 0.8559
Recall: 0.4742
Precision: 0.7584
F1: 0.5821
##### SVM #####
Accuracy: 0.7963
Auc: 0.5042
Recall: 0.0
Precision: 0.0
F1: 0.0
##### XGB #####
Accuracy: 0.8523
Auc: 0.8466
Recall: 0.4933
Precision: 0.695
F1: 0.5749
```

```

##### LightGBM #####
Accuracy: 0.8619
Auc: 0.8616
Recall: 0.4993
Precision: 0.7397
F1: 0.5947
##### CatBoost #####
Accuracy: 0.8648
Auc: 0.8675
Recall: 0.4978
Precision: 0.7576
F1: 0.5993

```

Baskılama sonrası model performansları değişmedi. Aykırı değerlerin oranı düşük olabilir. Oranlarına bakalım.

```

import pandas as pd

def calculate_outlier_ratio(dataframe, columns=None):
    """
    Verilen bir veri çerçevesindeki sayısal sütunlar için aykırı değer oranlarını hesaplar.

    Parameters:
        dataframe (pd.DataFrame): Veri çerçevesi.
        columns (list, optional): Kontrol edilecek sütunların listesi. None ise tüm sayısal sütunlar kullanılır.

    Returns:
        pd.DataFrame: Her bir sütun için aykırı değer oranını içeren bir veri çerçevesi.
    """

    outlier_ratios = {}
    for col in num_cols:
        Q1 = dataframe[col].quantile(0.25)
        Q3 = dataframe[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = ((dataframe[col] < lower_bound) | (dataframe[col] > upper_bound)).sum()
        outlier_ratios[col] = outliers / len(dataframe) * 100 # Aykırı değer oranı (%)

    return pd.DataFrame(outlier_ratios.items(), columns=["Column", "Outlier Ratio (%)"])

outlier_ratios = calculate_outlier_ratio(df)
print(outlier_ratios)

```

	Column	Outlier Ratio (%)
0	kredi_skoru	0.150
1	yas	3.590
2	sirkette_kaldigi_ay_sayisi	0.000
3	bakiye	0.000
4	tahmini_maas	0.000

Görüldüğü gibi aykırı değerlerin veri kümesindeki oranı düşük. Yaptığımız aykırı değer analizi sonuçlarına bakarak kredi skoru ve yaş değişkenindeki aykırı değerlerin anlamlı olduğu ve model performanslarını etkilemediğini gördük. O yüzden modellemeye aykırı değerleri baskılamadan devam edeceğiz.

YENİ DEĞİŞKEN ÜRETMEK

İş bilgisine dayanarak ve mantıklı çıkarımlar yaparak model performansını iyileştirmek için yeni değişkenler üretelim.

```
df_copy = df.copy()

# Yaşa göre yaşam dönemi kategorisi
df_copy['yas_donemi'] = pd.cut(df_copy['yas'], bins=[0, 25, 40, 60, 100], labels=['Genç', 'Orta Yaş Altı', 'Orta Yaş', 'Yaşlı'])
# Bakiye durumunu sınıflandırma
df_copy['bakiye_durumu'] = pd.cut(df_copy['bakiye'], bins=[-1, 0, 50000, 100000, 200000, float('inf')],
                                  labels=['Hiç Para Yok', 'Düşük', 'Orta', 'Yüksek', 'Çok Yüksek'])

# Aylık ortalama gelir
df_copy['ortalama_gelir'] = df_copy['tahmini_maas'] / (df_copy['sirkette_kaldigi_ay_sayisi'] + 1) # 0'a bölmeyi önlemek için +1

# Kredi skoru durumunu sınıflandırma
df_copy['kredi_skoru_durumu'] = pd.cut(df_copy['kredi_skoru'], bins=[0, 500, 700, 850],
                                       labels=['Düşük', 'Orta', 'Yüksek'])

def musteri_kategorisi(row):
    if row['bakiye_durumu'] == 'Çok Yüksek' and row['kredi_skoru_durumu'] == 'Yüksek':
        return 'Premium'
    elif row['bakiye_durumu'] in ['Yüksek', 'Orta'] and row['kredi_skoru_durumu'] == 'Orta':
        return 'Standart'
    else:
        return 'Riskli'

df_copy['musteri_kategorisi'] = df_copy.apply(musteri_kategorisi, axis=1)

df_copy['musteri_kidem_kategorisi'] = pd.cut(
    df_copy['sirkette_kaldigi_ay_sayisi'],
    bins=[0, 3, 6, 10], # Kısa, Orta ve Uzun vadeli için eşik değerler
    labels=['Kısa Vadeli', 'Orta Vadeli', 'Uzun Vadeli'],
    right=True, # Sağ kapsayıcılığı aç
    include_lowest=True # En düşük değeri de kapsa
)
```

df_copy									
li_karti_var_mi	aktif_uye_mi	tahmini_maas	aynildi_mi(churn)	bakiye_durumu	ortalama_gelir	kredi_skoru_durumu	musteri_kategorisi	musteri_kidem_kategorisi	
1	1	101348.880	1	Hiç Para Yok	33782.960	Orta	Riskli	Kısa Vadeli	
0	1	112542.580	0	Orta	56271.290	Orta	Standart	Kısa Vadeli	
1	0	113931.570	1	Yüksek	12659.063	Orta	Standart	Uzun Vadeli	
0	0	93826.630	0	Hiç Para Yok	46913.315	Orta	Riskli	Kısa Vadeli	
1	1	79084.100	0	Yüksek	26361.367	Yüksek	Riskli	Kısa Vadeli	
1	0	149756.710	1	Yüksek	16639.634	Orta	Standart	Uzun Vadeli	
1	1	10062.800	0	Hiç Para Yok	1257.850	Yüksek	Riskli	Uzun Vadeli	
1	0	119346.880	1	Yüksek	23869.376	Düşük	Riskli	Orta Vadeli	
0	1	74940.500	0	Yüksek	14988.100	Orta	Standart	Orta Vadeli	
1	1	71725.730	0	Yüksek	23908.577	Orta	Standart	Kısa Vadeli	
0	0	80181.120	0	Yüksek	11454.446	Orta	Standart	Orta Vadeli	

1. Bakiye durumu değişkeni: amaç, müşterilerin hesap bakiyelerini belirli aralıklara göre sınıflandırmaktır.

bins ile bu aralıklar tanımlanır:

- 1 ila 0: "Hiç Para Yok"
- 0 ila 50.000: "Düşük"
- 50.000 ila 100.000: "Orta"
- 100.000 ila 200.000: "Yüksek"
- 200.000 ve üzeri: "Çok Yüksek"

labels, her aralığa atanacak kategorik etiketleri tanımlar.

2. Ortalama gelir değişkeni: amaç, müşterinin aylık ortalama gelirini tahmini maaşı üzerinden hesaplamak.

tahmini_maas değişkeni müşterinin toplam kazancını temsil eder.

sirkette_kaldığı_ay_sayısı, müşterinin şirkette kaç ay çalıştığını belirtir.

Toplam maaş, çalışılan ay sayısına bölünerek aylık ortalama gelir hesaplanır.

Bölme işleminde 0'a bölünmeyi önlemek için (sirkette_kaldığı_ay_sayısı + 1) kullanılmıştır.

3. Kredi skoru durumu değişkeni: Amaç, müşterilerin kredi skorunu düşük, orta veya yüksek olarak sınıflandırmak.

- 0 ila 500: "Düşük"
- 500 ila 700: "Orta"
- 700 ila 850: "Yüksek"

4. Müşteri kategorisi değişkeni: amaç, müşterileri "Premium", "Standart" ve "Riskli" olarak sınıflandırmak.

- apply fonksiyonu her satır üzerinde çalışarak yeni bir kategori belirler.
- Premium: bakiye_durumu "Çok Yüksek" ve kredi_skoru_durumu "Yüksek".
- Standart: bakiye_durumu "Yüksek" veya "Orta" ve kredi_skoru_durumu "Orta".
- Riskli: Diğer durumlar.

5. Müşteri kıdem kategorisi değişkeni: Amaç, müşterilerin şirkette kalma süresine göre kıdem kategorisi belirlemek.

- 0 ila 3 ay: "Kısa Vadeli"
- 3 ila 6 ay: "Orta Vadeli"
- 6 ila 10 ay: "Uzun Vadeli"

labels bu aralıklara karşılık gelen etiketlerdir.

right=True sağ sınırın dahil edilmesini sağlar (örneğin, 3 ay "Kısa Vadeli" olarak kabul edilir).

include_lowest=True en düşük değeri (0) dahil eder.

ONE – HOT & LABEL HOT ENCODİNG

Yeni değişkenler üretmiştik. Kategorik ve numerik değişken listemizi güncelleyelim.

```
# Değişkenlerin tiplerine göre ayrılması işlemi
cat_cols, num_cols, cat_but_car = grab_col_names(df_copy)
```

```
Observations: 10000
Variables: 16
cat_cols: 10
num_cols: 6
cat_but_car: 0
num_but_cat: 7
```

```
cat_cols
```

```
['ulke',
 'cinsiyet',
 'musteri_kategorisi',
 'urun_sayısı',
 'kredi_kartı_var_mı',
 'aktif_uye_mi',
 'ayrıldı_mı(churn)',
 'bakiye_durumu',
 'kredi_skoru_durumu',
 'musteri_kidem_kategorisi']
```

Makine öğrenimi algoritmaları genellikle sayısal verilerle çalışır. Ancak, çoğu veri seti kategorik veriler içerir (örneğin, "Cinsiyet", "Ülke"). Bu kategorik verilerin doğrudan modelde kullanılması mümkün olmadığından, bu veriler sayısal bir formata dönüştürülmelidir. Bu amaçla One-Hot Encoding ve Label Encoding gibi teknikler kullanılır.

One-Hot Encoding, her kategori için ayrı bir sütun oluşturarak, kategori varlığını 0 ve 1 ile temsil eder.

Label Encoding, her kategoriye bir sayı atar. Doğal sıralamaya sahip veriler için uygundur.

Bu bilgilere dayanarak Label encoding yapılacak ve one-hot yapılacak değişkenleri belirleyip işlem yaparız.

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
# Label Encoding yapılacak değişkenler
label_cols = ['bakiye_durumu', 'kredi_skoru_durumu', 'musteri_kategorisi', 'musteri_kidem_kategorisi']
le = LabelEncoder()

for col in label_cols:
    df_copy[col] = le.fit_transform(df_copy[col])

# One-Hot Encoding yapılacak değişkenler
df_copy = pd.get_dummies(df_copy, columns=['ulke', 'cinsiyet'], drop_first=False, dtype=int)
```

Label Encoding ve One-Hot Encoding sonrası görünümü:

df_copy.head(10)

lurumu	ortalama_gelir	kredi_skoru_durumu	musteri_kategorisi	musteri_kidem_kategorisi	ulke_France	ulke_Germany	ulke_Spain	cinsiyet_Female	cinsiyet_Male
1	33782.960	1	1	0	1	0	0	1	0
2	56271.290	1	2	0	0	0	1	1	0
3	12659.063	1	2	2	1	0	0	1	0
1	46913.315	1	1	0	1	0	0	1	0
3	26361.367	2	1	0	0	0	1	1	0
3	16639.634	1	2	2	0	0	1	0	1
1	1257.850	2	1	2	1	0	0	0	1
3	23869.376	0	1	1	0	1	0	1	0
3	14988.100	1	2	1	1	0	0	0	1
3	23908.577	1	2	0	1	0	0	0	1

VERİYİ STANDARTLAŞTIRMA İŞLEMİ: Z-SKOR

```
## z-score
from scipy.stats import zscore
# Z-score hesaplanacak sayısal sütunlar
sayisal = ['kredi_skoru', 'yas', 'sirkette_kaldigi_ay_sayisi', 'bakiye',
            'tahmini_maas', 'ortalama_gelir']

# Z-score hesaplama
df_copy[sayisal] = df_copy[sayisal].apply(zscore)
```

- Normalizasyon olmadan, daha büyük ölçekli özellikler daha küçük ölçeklilere baskın gelebilir ve bu da makine öğrenimi modellerinde önyargılı sonuçlara yol açabilir. Bu nedenle standartlaştırma işlemini uyguluyoruz.
- Z-score, verileri standardize ederek ortalaması 0, standart sapması 1 olacak şekilde dönüştürür.
- Veri setimizdeki sayısal değişkenleri (kredi skoru, yaş, şirkette kaldığı ay sayısı, bakiye, tahmini maaş ve ortalama gelir) standartlaştırarak farklı ölçeklerdeki değişkenleri aynı ölçekte değerlendiriyoruz.

Standartlaştırma işlemi sonrası veri setinin görünümü:

df_copy.head(10)

	kredi_skoru	yas	sirkette_kaldigi_ay_sayisi	bakiye	urun_sayisi	kredi_karti_var_mi	aktif_uye_mi	tahmini_maas	ayrildi_mi(churn)	bakiye_durumu
0	-0.326	0.294	-1.042	-1.226	1	1	1	0.022	1	1
1	-0.440	0.198	-1.388	0.117	1	0	1	0.217	0	2
2	-1.537	0.294	1.033	1.333	3	1	0	0.241	1	3
3	0.502	0.007	-1.388	-1.226	2	0	0	-0.109	0	1
4	2.064	0.389	-1.042	0.786	1	1	1	-0.365	0	3
5	-0.057	0.484	1.033	0.597	2	1	0	0.864	1	3
6	1.774	1.056	0.687	-1.226	2	1	1	-1.565	0	1
7	-2.840	-0.946	-0.350	0.618	4	1	0	0.335	1	3
8	-1.547	0.484	-0.350	1.051	2	0	1	-0.437	0	3
9	0.346	-1.137	-1.042	0.931	1	1	1	-0.493	0	3

Z-score sonrası base model oluşturalım:

```
# Bağımlı değişken
y = df_copy["ayrıldı_mı(churn)"]

# Bağımsız değişkenler
X = df_copy.drop(["ayrıldı_mı(churn)"], axis=1) # Hedef değişkeni çıkarıyoruz

models = [
    ('LR', LogisticRegression(random_state=12345)),
    ('KNN', KNeighborsClassifier()),
    ('CART', DecisionTreeClassifier(random_state=12345)),
    ('RF', RandomForestClassifier(random_state=12345)),
    ('SVM', SVC(gamma='auto', random_state=12345)),
    ('XGB', XGBClassifier(random_state=12345)),
    ('LightGBM', LGBMClassifier(random_state=12345)),
    ('CatBoost', CatBoostClassifier(verbose=False, random_state=12345))
]

from sklearn.model_selection import cross_validate

for name, model in models:
    cv_results = cross_validate(
        model,
        X,
        y,
        cv=10, # 10 katlı çapraz doğrulama
        scoring=["accuracy", "f1", "roc_auc", "precision", "recall"]
    )

    print(f"##### {name} #####")
    print(f"Accuracy: {round(cv_results['test_accuracy'].mean(), 4)}")
    print(f"AUC: {round(cv_results['test_roc_auc'].mean(), 4)}")
    print(f"Recall: {round(cv_results['test_recall'].mean(), 4)}")
    print(f"Precision: {round(cv_results['test_precision'].mean(), 4)}")
    print(f"F1: {round(cv_results['test_f1'].mean(), 4)}")
```

LR

Accuracy: 0.8101

Auc: 0.7657

Recall: 0.2126

Precision: 0.5979

F1: 0.3121

KNN

Accuracy: 0.8232

Auc: 0.7559

Recall: 0.3368

Precision: 0.6237

F1: 0.4369

CART

Accuracy: 0.7933

Auc: 0.6917

Recall: 0.5204

Precision: 0.4931

F1: 0.5057

RF

Accuracy: 0.8602

Auc: 0.853

Recall: 0.4595

Precision: 0.7597

F1: 0.5716

SVM

Accuracy: 0.8509

Auc: 0.8416

Recall: 0.3211

Precision: 0.8612

F1: 0.4657

XGB

Accuracy: 0.8545

Auc: 0.8456

Recall: 0.4919

Precision: 0.7064

F1: 0.5783


```
##### LightGBM #####
Accuracy: 0.8621
Auc: 0.8592
Recall: 0.4992
Precision: 0.7407
F1: 0.5952
##### CatBoost #####
Accuracy: 0.8655
Auc: 0.8658
Recall: 0.4963
Precision: 0.7611
F1: 0.5998
```

Bu çıktıya göre yorum yaparsak **CatBoost ve LightGBM**, özellikle dengeli Recall, Precision ve AUC değerleri ile en iyi performansı gösteriyor.

MODEL SEÇİMİ VE TEST HATASI

Modelimizin bağımlı değişkeni yani hedef değişkenimiz ayrıldı mı(churn)'dır.

```
df_model=df_copy.copy()

y = df_model["ayrıldı_mı(churn)"]
X = df_model.drop(["ayrıldı_mı(churn)"], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12345, stratify=y)
```

Bu kodla bağımlı ve bağımsız değişkenleri ayırdık. Ardından veri setini %80 eğitim, %20 test olacak şekilde bölerek model eğitimi için hazır hale getirdik.

Şimdi modellerimizi denemeye geçebiliriz.

1. KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve, auc, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# KNN modelinin tanımlanması
knn_model = KNeighborsClassifier(n_neighbors=5) # K değeri: 5 (isteğe göre değiştirilebilir)

# Modeli eğitme
knn_model.fit(X_train, y_train)

# Tahmin yapma
y_train_pred = knn_model.predict(X_train)
y_test_pred = knn_model.predict(X_test)

# Performans değerlendirme
print("Eğitim Seti Sınıflandırma Raporu:")
print(classification_report(y_train, y_train_pred))

print("\nTest Seti Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred))

# ROC-AUC Skoru
y_train_proba = knn_model.predict_proba(X_train)[:, 1] # Eğitim seti pozitif sınıf olasılıkları
y_test_proba = knn_model.predict_proba(X_test)[:, 1] # Test seti pozitif sınıf olasılıkları

roc_auc_train = roc_auc_score(y_train, y_train_proba)
roc_auc_test = roc_auc_score(y_test, y_test_proba)

print(f"\nEğitim Seti ROC-AUC Skoru: {roc_auc_train}")
print(f"Test Seti ROC-AUC Skoru: {roc_auc_test}")
```



```
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_test_pred)

# Confusion Matrix'i Görselleştirme
cm_display = ConfusionMatrixDisplay(conf_matrix, display_labels=["Churn Yok", "Churn Var"])
cm_display.plot(cmap="viridis")
plt.title("KNN - Confusion Matrix")
plt.show()

# ROC Eğrisi
# Eğitim seti için ROC eğrisi
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
# Test seti için ROC eğrisi
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

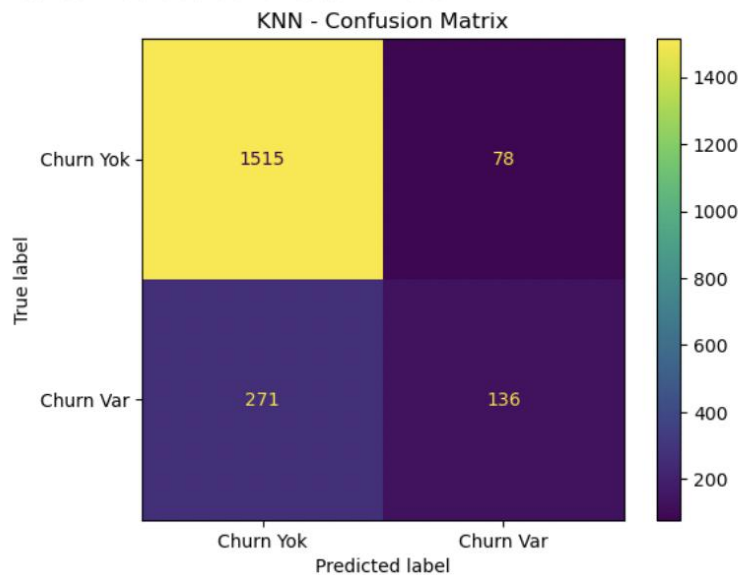
# ROC eğrilerini çizme
plt.figure(figsize=(10, 6))
plt.plot(fpr_train, tpr_train, color="blue", label=f"Eğitim ROC (AUC = {roc_auc_train:.2f})")
plt.plot(fpr_test, tpr_test, color="red", label=f"Test ROC (AUC = {roc_auc_test:.2f})")
plt.plot([0, 1], [0, 1], color="gray", linestyle="--", label="Şans Eğrisi")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Eğrisi - Eğitim ve Test Seti (KNN)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

Çıktı aşağıdaki gibidir.

Eğitim Seti Sınıflandırma Raporu:					
	precision	recall	f1-score	support	
0	0.88	0.97	0.92	6370	
1	0.81	0.46	0.59	1630	
accuracy			0.87	8000	
macro avg	0.84	0.72	0.75	8000	
weighted avg	0.86	0.87	0.85	8000	

Test Seti Sınıflandırma Raporu:					
	precision	recall	f1-score	support	
0	0.85	0.95	0.90	1593	
1	0.64	0.33	0.44	407	
accuracy			0.83	2000	
macro avg	0.74	0.64	0.67	2000	
weighted avg	0.80	0.83	0.80	2000	

Eğitim Seti ROC-AUC Skoru: 0.9138064739817588
Test Seti ROC-AUC Skoru: 0.7698507444270156



1. Accurayc (Doğruluk):

- Eğitim seti accuracy: %87
- Test seti accuracy: %83

Model her iki veri setinde de benzer bir doğruluk oranı göstermiştir. Fakat test setindeki performansın eğitim setine göre biraz daha düşük olması overfitting olabileceğini düşündürüyor.

2.Recall (Duyarlılık) ve Precision (Kesinlik):

• Eğitim seti:

- **Churn yok (0):** Precision (%88) ve Recall (%97) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%81) iyi ancak Recall (%46) düşük. Bu durumda model eğitim setinde gerçek Churn var örneklerinin %47'sini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır.

• Test seti:

- **Churn yok (0):** Precision (%85) ve Recall (%95) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%64) fena değil ancak Recall (%33) çok düşük. Bu durumda model test setinde gerçek churn var örneklerinin %33'ünü doğru tahmin etmiştir. Bu da bizim için düşük bir orandır. Gerçek churn müşterilerinin büyük kısmını kaçırmaktadır.

3.F1-Score:

• Eğitim seti:

- **Churn yok (0):** %92
- **Churn var (1):** %59

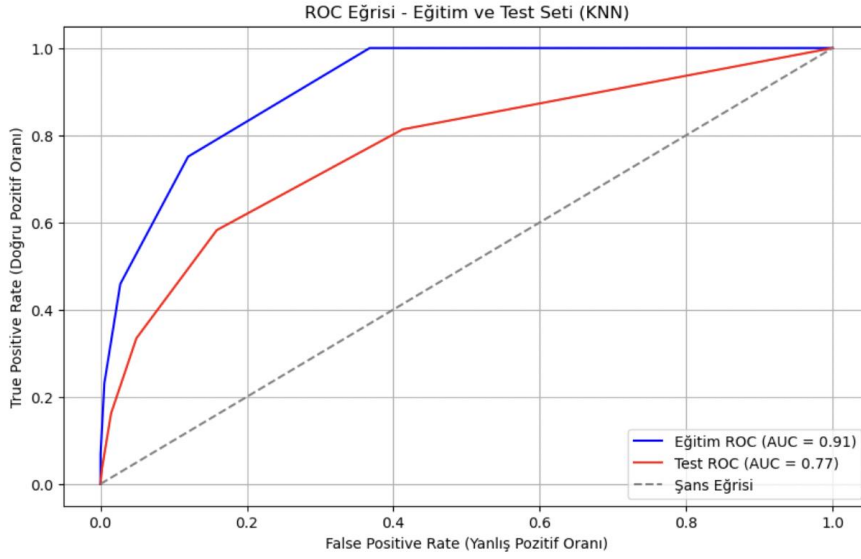
• Test seti:

- **Churn yok (0):** %90
- **Churn var (1):** %44

Model churn yok sınıfında çok başarılı bir performans gösterirken, churn var sınıfındaki düşük recall değeri F1-Skor değerini ciddi şekilde aşağı çekiyor.

4.Confusion Matrix:

- 1515 örnek churn yok doğru tahmin edilmiş.
- 136 örnek churn var doğru tahmin edilmiş.
- 78 kişi churn yok iken churn var olarak tahmin edilmiş.
- 271 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %91
- Test seti: %77

Test kümemiz çok düşük bir performans göstermiştir. Test ve eğitim çizgisi arasındaki fark da fazladır. Overfitting olabileceği artık net bir şekilde görülmüştür.

Kısaca: Eğitim ve test kümesi arasındaki performans farkı overfitting olduğunu düşündürmektedir. Model, **Churn Yok (0)** sınıfında iyi bir performans sergilerken, **Churn Var (1)** sınıfında düşük performans göstermektedir.

2. KARAR AĞAÇLARI

```
# Veriyi eğitim ve test setlerine ayırma
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12345, stratify=y)

# Karar ağacı modelinin tanımlanması
decision_tree_model = DecisionTreeClassifier(random_state=12345, max_depth=3) # max_depth sınırlaması görselleştirme için

# Modeli eğitme
decision_tree_model.fit(X_train, y_train)

# Tahmin yapma
y_train_pred = decision_tree_model.predict(X_train)
y_test_pred = decision_tree_model.predict(X_test)

# Performans değerlendirme
print("Eğitim Seti Sınıflandırma Raporu:")
print(classification_report(y_train, y_train_pred))

print("Test Seti Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred))

# ROC-AUC Skoru
roc_auc_train = roc_auc_score(y_train, decision_tree_model.predict_proba(X_train)[:, 1])
roc_auc_test = roc_auc_score(y_test, decision_tree_model.predict_proba(X_test)[:, 1])
print(f"Eğitim Seti ROC-AUC Skoru: {roc_auc_train}")
print(f"Test Seti ROC-AUC Skoru: {roc_auc_test}")

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_test_pred)
cm_display = metrics.ConfusionMatrixDisplay(conf_matrix, display_labels=["Churn Yok", "Churn Var"])
cm_display.plot(cmap="viridis")
plt.title("Decision Tree - Confusion Matrix")
plt.show()

# ROC Eğrisi için olasılık tahminlerini al
y_train_proba = decision_tree_model.predict_proba(X_train)[:, 1]
y_test_proba = decision_tree_model.predict_proba(X_test)[:, 1]

# False Positive Rate ve True Positive Rate hesaplama
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)
```

```
# ROC Eğrisini Çizme
plt.figure(figsize=(10, 6))
plt.plot(fpr_train, tpr_train, color='blue', lw=2,
        label=f'Eğitim ROC Curve (AUC = {roc_auc_train:.4f})")
plt.plot(fpr_test, tpr_test, color='darkorange', lw=2,
        label=f'Test ROC Curve (AUC = {roc_auc_test:.4f})")
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label="Random Guessing")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC) Curve")
plt.legend(loc="lower right")
plt.grid()
plt.show()

# Karar ağacını görselleştirme
dot_data = export_graphviz(
    decision_tree_model,
    out_file=None,
    feature_names=X.columns,
    class_names=["Churn Yok", "Churn Var"],
    filled=True,
    rounded=True,
    special_characters=True
)

graph = graphviz.Source(dot_data)
graph.render("decision_tree", format="png", cleanup=True)
graph.view() # Karar ağacını gösterir

# Matplotlib ile alternatif görselleştirme
plt.figure(figsize=(16, 10))
tree.plot_tree(
    decision_tree_model,
    feature_names=X.columns,
    class_names=["Churn Yok", "Churn Var"],
    filled=True,
    rounded=True
)
plt.title("Decision Tree - Matplotlib Görselleştirme")
plt.show()
```

Çıktı aşağıdaki gibidir.

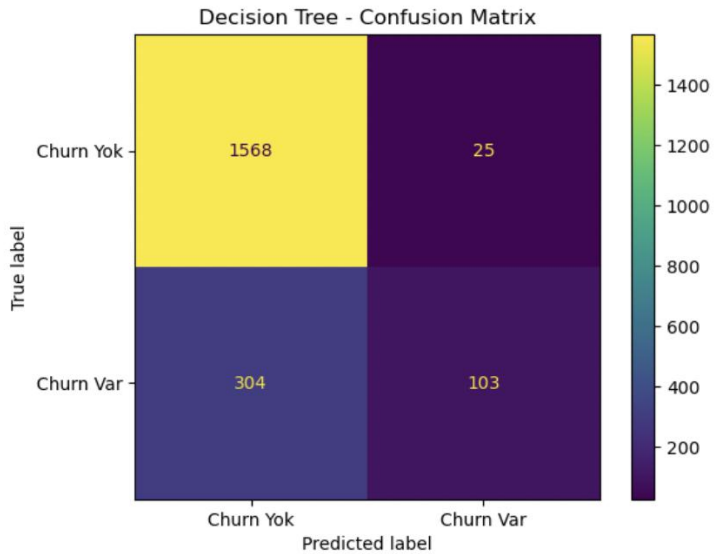
Eğitim Seti Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.84	0.99	0.91	6370
1	0.89	0.25	0.39	1630
accuracy			0.84	8000
macro avg	0.86	0.62	0.65	8000
weighted avg	0.85	0.84	0.80	8000

Test Seti Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.84	0.98	0.91	1593
1	0.80	0.25	0.39	407
accuracy			0.84	2000
macro avg	0.82	0.62	0.65	2000
weighted avg	0.83	0.84	0.80	2000

Eğitim Seti ROC-AUC Skoru: 0.7937855264805307
Test Seti ROC-AUC Skoru: 0.8052852544377967



1. Accurayc (Doğruluk):

- Eğitim seti accuracy: %84
- Test seti accuracy: %84

Model her iki veri setinde de aynı doğruluk oranı göstermiştir.

2.Recall (Duyarlılık) ve Precision (Kesinlik):

• Eğitim seti:

- **Churn yok (0):** Precision (%84) ve Recall (%99) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%89) iyi ancak Recall (%25) düşük. Bu durumda model eğitim setinde gerçek churn var örneklerinin %25'ini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır.

• Test seti:

- **Churn yok (0):** Precision (%84) ve Recall (%98) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%80) fena değil ancak Recall (%25) çok düşük. Bu durumda model test setinde gerçek churn var örneklerinin %25'ini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır. Gerçek churn müşterilerinin büyük kısmını kaçırmaktadır.

3.F1-Score:

• Eğitim seti:

- **Churn yok (0):** %91
- **Churn var (1):** %39

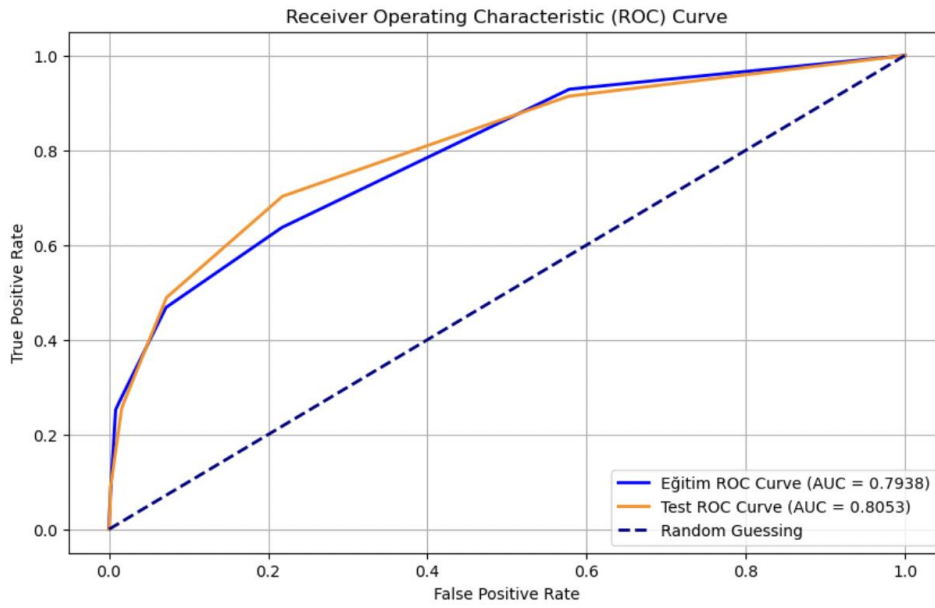
• Test seti:

- **Churn yok (0):** %91
- **Churn var (1):** %39

Model churn yok sınıfında çok başarılı bir performans gösterirken, churn var sınıfındaki düşük recall değeri F1-Skor değerini ciddi şekilde aşağı çekiyor.

4.Confusion Matrix:

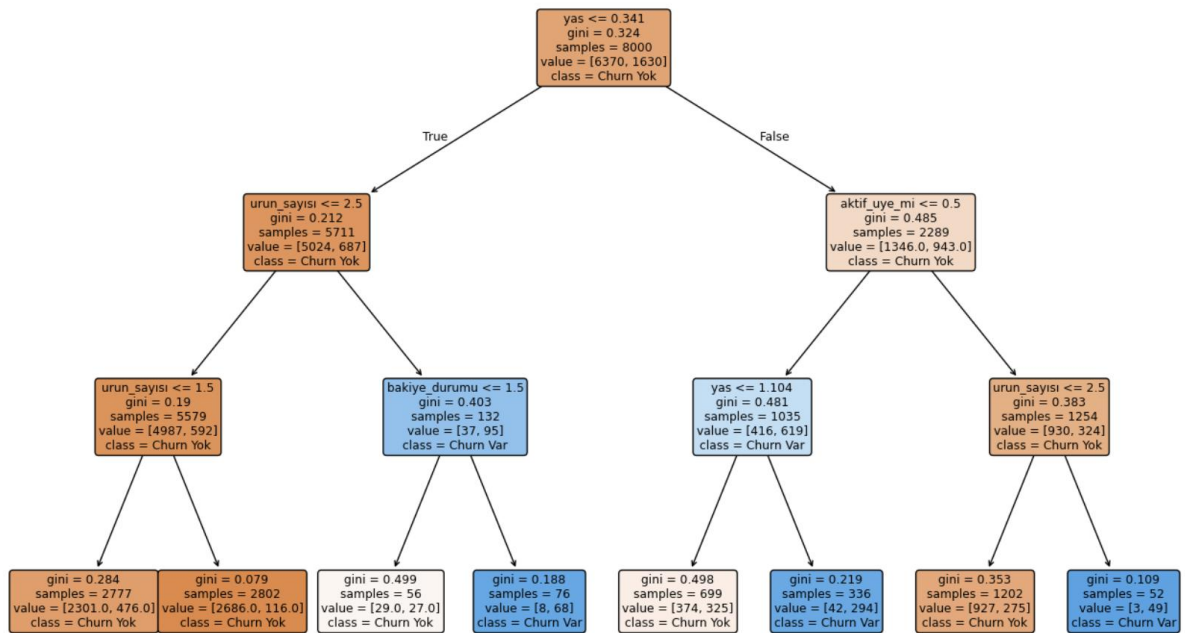
- 1568 örnek churn yok doğru tahmin edilmiş.
- 103 örnek churn var doğru tahmin edilmiş.
- 25 kişi churn yok iken churn var olarak tahmin edilmiş.
- 304 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %79
- Test seti: %80

Test ve eğitim kümemiz çok yakın bir performans göstermiştir.



Kısaca: Model, **Churn Yok (0)** sınıfında iyi bir performans sergilerken, **Churn Var (1)** sınıfında düşük performans göstermektedir.

3. LIGHTGBM

```
# LightGBM modelinin tanımlanması
lgbm_model = LGBMClassifier(random_state=12345)

# Modeli eğitme
lgbm_model.fit(X_train, y_train)

# Tahmin yapma
y_train_pred = lgbm_model.predict(X_train)
y_test_pred = lgbm_model.predict(X_test)

# Performans değerlendirme
print("Eğitim Seti Sınıflandırma Raporu:")
print(classification_report(y_train, y_train_pred))

print("Test Seti Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred))

# ROC-AUC Skoru
roc_auc_train = roc_auc_score(y_train, lgbm_model.predict_proba(X_train)[:, 1])
roc_auc_test = roc_auc_score(y_test, lgbm_model.predict_proba(X_test)[:, 1])
print(f"Eğitim Seti ROC-AUC Skoru: {roc_auc_train}")
print(f"Test Seti ROC-AUC Skoru: {roc_auc_test}")

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_test_pred)
cm_display = metrics.ConfusionMatrixDisplay(conf_matrix, display_labels=["Churn Yok", "Churn Var"])
cm_display.plot(cmap="viridis")
plt.title("LightGBM - Confusion Matrix")
plt.show()

# ROC Eğrisi
y_train_proba = lgbm_model.predict_proba(X_train)[:, 1]
y_test_proba = lgbm_model.predict_proba(X_test)[:, 1]

# False Positive Rate ve True Positive Rate hesaplama
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC Eğrisini Çizme
plt.figure(figsize=(10, 6))
plt.plot(fpr_train, tpr_train, color='blue', lw=2, label=f"Eğitim ROC Curve (AUC = {roc_auc_train:.4f})")
plt.plot(fpr_test, tpr_test, color='darkorange', lw=2, label=f"Test ROC Curve (AUC = {roc_auc_test:.4f})")
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label="Random Guessing")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC) Curve")
plt.legend(loc="lower right")
plt.grid()
plt.show()

from sklearn.metrics import mean_squared_error
import numpy as np

# Test seti için RMSE hesaplama
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_proba))
print(f"Test Seti RMSE: {rmse_test}")

# Eğitim seti için RMSE hesaplama
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_proba))
print(f"Eğitim Seti RMSE: {rmse_train}")
```

Çıktımız aşağıda verilmiştir.

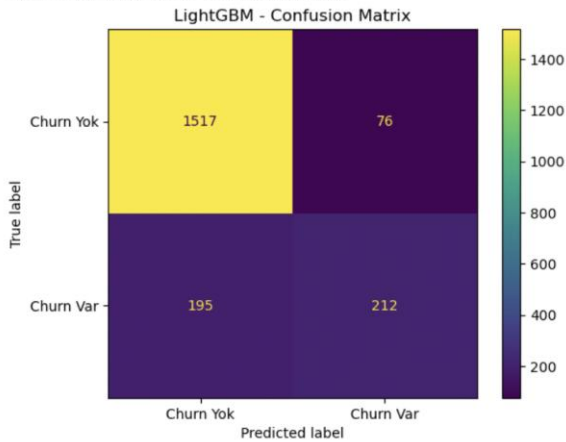
```
[LightGBM] [Info] Number of positive: 1630, number of negative: 6370
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.000755 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 1131
[LightGBM] [Info] Number of data points in the train set: 8000, number of used features: 18
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.203750 -> initscore=-1.363019
[LightGBM] [Info] Start training from score -1.363019
Eğitim Seti Sınıflandırma Raporu:
precision    recall    f1-score   support
0           0.92      0.98      0.95     6370
1           0.91      0.65      0.76     1630

accuracy      0.92     8000
macro avg     0.91     8000
weighted avg  0.92     8000

Test Seti Sınıflandırma Raporu:
precision    recall    f1-score   support
0           0.89      0.95      0.92     1593
1           0.74      0.52      0.61      407

accuracy      0.86     2000
macro avg     0.81     2000
weighted avg  0.86     2000

Eğitim Seti ROC-AUC Skoru: 0.9686133235738845
Test Seti ROC-AUC Skoru: 0.8610737085313356
```



1. Accurayc (Doğruluk):

- Eğitim seti accuracy: %92
- Test seti accuracy: %86

Model her iki veri setinde de benzer bir doğruluk oranı göstermiştir. Fakat test setindeki performansın eğitim setine göre daha düşük olması overfitting olabileceğini düşündürüyor.

2.Recall (Duyarlılık) ve Precision (Kesinlik):

• Eğitim seti:

- **Churn yok (0):** Precision (%92) ve Recall (%98) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%91) iyi ancak Recall (%65) düşük. Bu durumda model eğitim setinde gerçek churn var örneklerinin %65'ini doğru tahmin etmiştir.

• Test seti:

- **Churn yok (0):** Precision (%89) ve Recall (%95) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%74) fena değil ancak Recall (%52) çok düşük. Bu durumda model test setinde gerçek churn var örneklerinin %52'ini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır. Gerçek churn müşterilerinin büyük kısmını kaçırmaktadır.

3.F1-Score:

- Eğitim seti:

- Churn yok (0): %95
- Churn var (1): %76

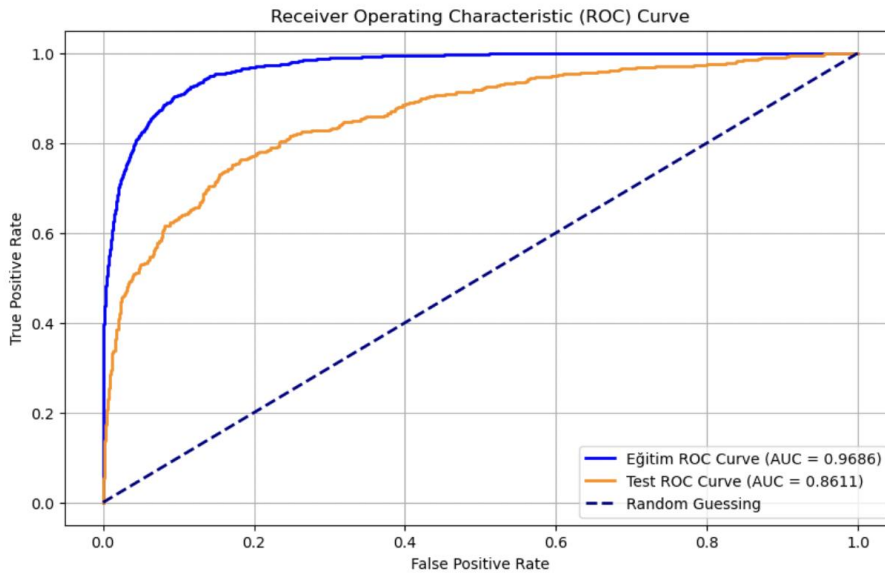
- Test seti:

- Churn yok (0): %92
- Churn var (1): %61

Model churn yok sınıfında çok başarılı bir performans gösterirken, churn var sınıfındaki düşük recall değeri F1-Skor değerini aşağı çekiyor.

4.Confusion Matrix:

- 1517 örnek churn yok doğru tahmin edilmiş.
- 212 örnek churn var doğru tahmin edilmiş.
- 76 kişi churn yok iken churn var olarak tahmin edilmiş.
- 195 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %96

- Test seti: %86

Test kümemiz çok düşük bir performans göstermiştir. Test ve eğitim çizgisi arasındaki fark da fazladır. Overfitting olabileceği artık net bir şekilde görülmüştür.

4. LOJİSTİK REGRESYON

```
# Lojistik Regresyon modelinin tanımlanması
lr_model = LogisticRegression(random_state=12345, max_iter=1000)

# Modeli eğitme
lr_model.fit(X_train, y_train)

# Tahmin yapma
y_train_pred = lr_model.predict(X_train)
y_test_pred = lr_model.predict(X_test)

# Performans değerlendirme
print("Eğitim Seti Sınıflandırma Raporu:")
print(classification_report(y_train, y_train_pred))

print("Test Seti Sınıflandırma Raporu:")
print(classification_report(y_test, y_test_pred))

# ROC-AUC Skoru
y_train_proba = lr_model.predict_proba(X_train)[:, 1]
y_test_proba = lr_model.predict_proba(X_test)[:, 1]

roc_auc_train = roc_auc_score(y_train, y_train_proba)
roc_auc_test = roc_auc_score(y_test, y_test_proba)

print(f"Eğitim Seti ROC-AUC Skoru: {roc_auc_train}")
print(f"Test Seti ROC-AUC Skoru: {roc_auc_test}")

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_test_pred)

# Confusion Matrix'i görselleştirme
cm_display = metrics.ConfusionMatrixDisplay(conf_matrix, display_labels=["Churn Yok", "Churn Var"])
cm_display.plot(cmap="viridis")
plt.title("Logistic Regression - Confusion Matrix")
plt.show()

# ROC Eğrisi
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

plt.figure(figsize=(10, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC Curve (AUC = {roc_auc_train:.4f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC Curve (AUC = {roc_auc_test:.4f})", color="darkorange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random Guessing")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC) Curve - Logistic Regression")
plt.legend(loc="lower right")
plt.grid()
plt.show()

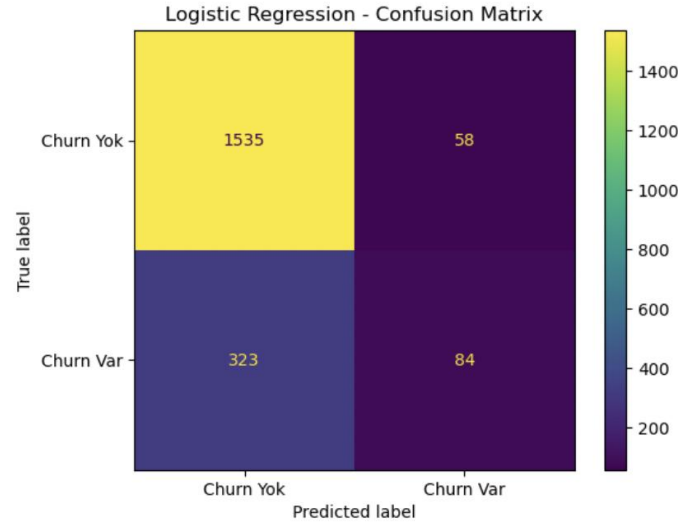
# RMSE Hesaplama
rmse_test = np.sqrt(mean_squared_error(y_test, y_test_proba))
print(f"Test Seti RMSE: {rmse_test}")

rmse_train = np.sqrt(mean_squared_error(y_train, y_train_proba))
print(f"Eğitim Seti RMSE: {rmse_train}")
```

Eğitim Seti Sınıflandırma Raporu:					
	precision	recall	f1-score	support	
0	0.83	0.97	0.89	6370	
1	0.62	0.21	0.32	1630	
accuracy			0.81	8000	
macro avg	0.72	0.59	0.60	8000	
weighted avg	0.78	0.81	0.77	8000	

Test Seti Sınıflandırma Raporu:					
	precision	recall	f1-score	support	
0	0.83	0.96	0.89	1593	
1	0.59	0.21	0.31	407	
accuracy			0.81	2000	
macro avg	0.71	0.58	0.60	2000	
weighted avg	0.78	0.81	0.77	2000	

Eğitim Seti ROC-AUC Skoru: 0.7641183268965915
Test Seti ROC-AUC Skoru: 0.776494522572341



1. Accurayc (Doğruluk):

- Eğitim seti accuracy: %81
- Test seti accuracy: %81

Model her iki veri setinde de aynı doğruluk oranı göstermiştir.

2.Recall (Duyarlılık) ve Precision (Kesinlik):

• Eğitim seti:

- **Churn yok (0):** Precision (%83) ve Recall (%97) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%62) iyi ancak Recall (%21) düşük. Bu durumda model eğitim setinde gerçek churn var örneklerinin %21'ini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır.

• Test seti:

- **Churn yok (0):** Precision (%83) ve Recall (%96) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%59) normal ancak Recall (%21) çok düşük. Bu durumda model test setinde gerçek churn var örneklerinin %21'ini doğru tahmin etmiştir. Bu da bizim için düşük bir orandır. Gerçek churn müşterilerinin büyük kısmını kaçırmaktadır.

3.F1-Score:

- Eğitim seti:

- Churn yok (0): %89
- Churn var (1): %32

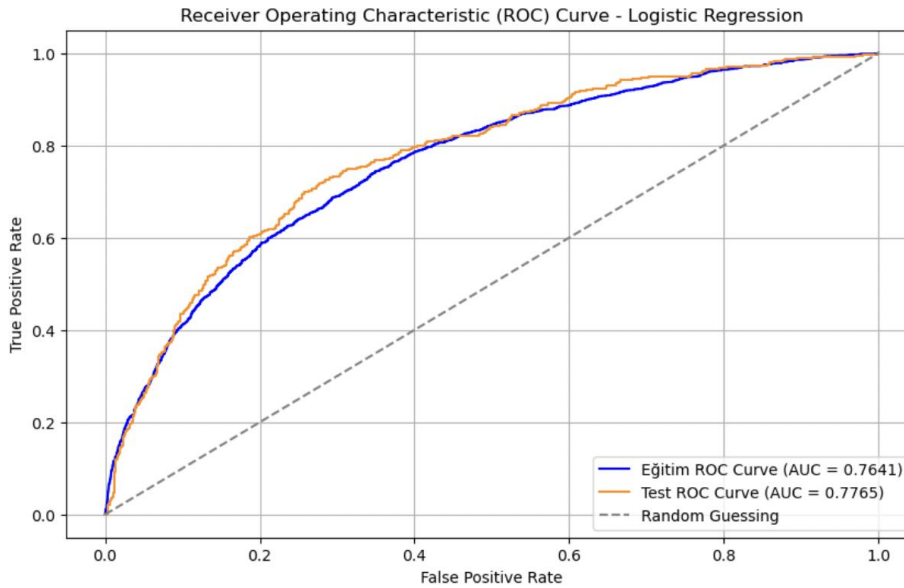
- Test seti:

- Churn yok (0): %89
- Churn var (1): %31

Model churn yok sınıfında çok başarılı bir performans gösterirken, churn var sınıfındaki düşük recall değeri F1-Skor değerini ciddi şekilde aşağı çekiyor.

4.Confusion Matrix:

- 1535 örnek churn yok doğru tahmin edilmiş.
- 84 örnek churn var doğru tahmin edilmiş.
- 58 kişi churn yok iken churn var olarak tahmin edilmiş.
- 323 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %76

- Test seti: %77

Test ve eğitim kümemiz yakın performans göstermiştir. Overfitting görülmemiştir fakat modelimiz churn yok grubunu daha iyi tahmin edip churn var grubunu daha kötü tahmin etmiştir.

Kısaca, modellerimizde churn yok grubunu daha iyi tahmin edip churn var grubunu daha kötü tahmin ettik. Bu iki grubun dengesiz dağılımından dolayı bu sorun ortaya çıkmıştır. Smote yaparak model performanslarımızı inceleyelim.

SMOTE UYGULAMA

- Smote dengesiz veri setlerinde azınlık sınıfını dengelemek için kullanılan bir yöntemdir. Bizim verimizde de dağılım dengesizdir.
- Veri setimizde **“Churn Var”** azınlık sınıfı olduğundan model çoğunluk sınıfına (churn yok) daha fazla ağırlık verdi. **Bu durum, azınlık sınıfını doğru tahmin etme başarısını düşürdü.** Bu nedenle smote uygulayıp model performanslarımızı inceleyeceğiz.

```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
# df_copy'den df_copy_smote adında bir kopya oluştur
df_copy_smote = df_copy.copy()
# Bağımlı ve bağımsız değişkenlerin ayrılması
y = df_copy_smote["ayrıldı_mı(churn)"] # Bağımlı değişken
X = df_copy_smote.drop(["ayrıldı_mı(churn)"], axis=1) # Bağımsız değişkenler

from imblearn.over_sampling import SMOTE
from collections import Counter

# SMOTE uygulama
smt = SMOTE(random_state=12345)
X_res, y_res = smt.fit_resample(X, y) # fit_sample yerine fit_resample kullanılmalı

print(f"Orijinal veri seti dağılımı: {Counter(y)}")
print(f"Resampled veri seti dağılımı: {Counter(y_res)}")

Orijinal veri seti dağılımı: Counter({0: 7963, 1: 2037})
Resampled veri seti dağılımı: Counter({1: 7963, 0: 7963})
```

Daha net göstermek için smote öncesi ve sonrası pasta grafiği çiziyoruz.

```
# SMOTE öncesi sınıf dağılımını hesapla
original_class_distribution = y_train.value_counts()

# SMOTE sonrası sınıf dağılımını hesapla
smote_class_distribution = pd.Series(y_train_smote).value_counts()

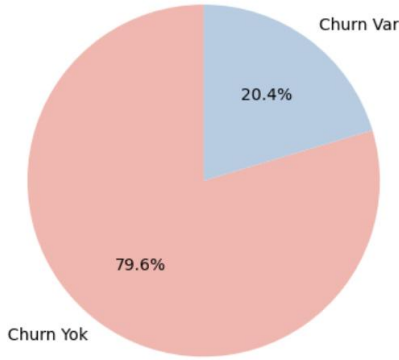
# Yan yana iki pie grafiği çizdirme
fig, axes = plt.subplots(1, 2, figsize=(9, 5))

# SMOTE öncesi pie grafiği
axes[0].pie(original_class_distribution,
            labels=['Churn Yok', 'Churn Var'],
            autopct='%1.1f%%',
            startangle=90,
            colors=plt.cm.Pastel1.colors)
axes[0].set_title("SMOTE Öncesi Eğitim Verisinde Churn Dağılımı")

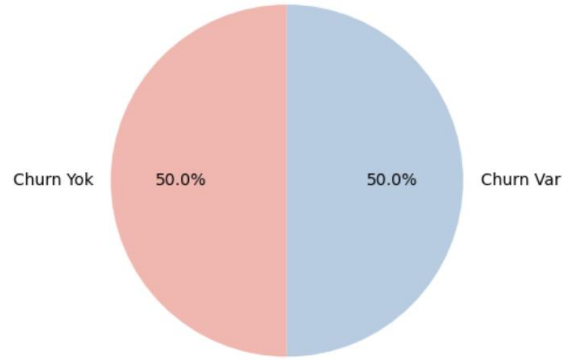
# SMOTE sonrası pie grafiği
axes[1].pie(smote_class_distribution,
            labels=['Churn Yok', 'Churn Var'],
            autopct='%1.1f%%',
            startangle=90,
            colors=plt.cm.Pastel1.colors)
axes[1].set_title("SMOTE Sonrası Eğitim Verisinde Churn Dağılımı")

# Grafiklerin gösterimi
plt.tight_layout()
plt.show()
```

SMOTE Öncesi Eğitim Verisinde Churn Dağılımı



SMOTE Sonrası Eğitim Verisinde Churn Dağılımı



SMOTE Öncesi Grafik:

- **Churn yok (0)** sınıfı çoğunluğu oluşturarak %79,6 oranında bir dağılıma sahiptir.
- **Churn var (1)** sınıfı ise %20,4 ile azınlığı oluşturmaktadır.

Bu dengesiz sınıf dağılımı, modelin azınlık sınıfını tahmin etmekte zorlandığını yukarıda yaptığımız modellerde görmüştük.

SMOTE Sonrası Grafik:

- **Churn yok (0) ve Churn var (1)** sınıflarını eşitleyerek %50-%50 oranına getirdik.

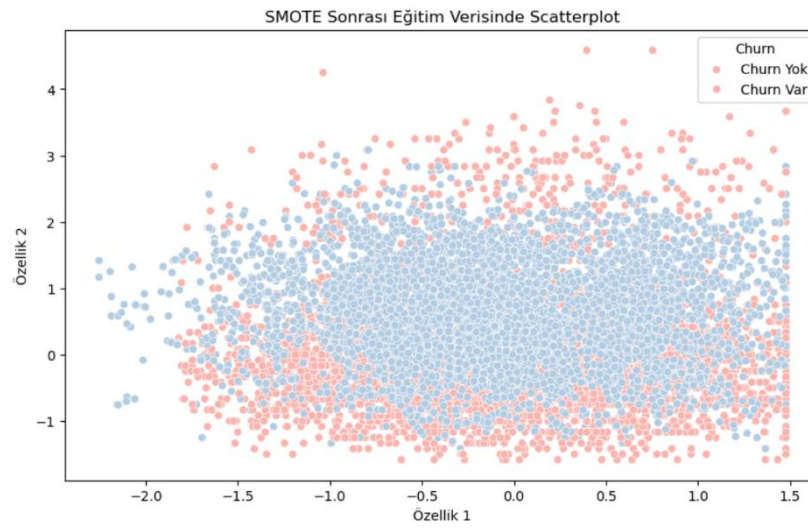
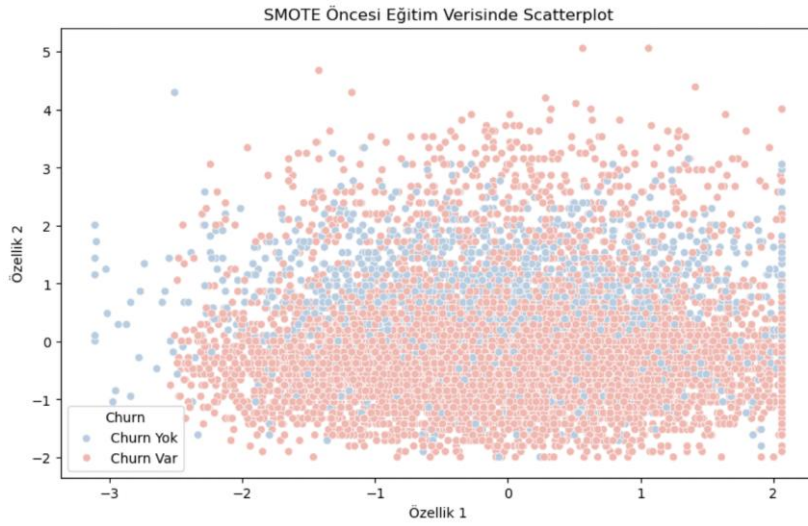
Bu işlem modelin her iki sınıfa da eşit ağırlık vermesini sağlar. Bu şekilde azınlık sınıf olan **Churn var**'ın daha doğru tahmin edilmesine katkıda bulunur.

Bir de scatter plot ile görselleştirme yapalım:

```
import matplotlib.pyplot as plt
import seaborn as sns

# SMOTE öncesi scatterplot (df_copy için)
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X.iloc[:, 0], # İlk özellik
               y=X.iloc[:, 1], # İkinci özellik
               hue=y,          # Hedef değişken
               palette=plt.cm.Pastel1.colors)
plt.title("SMOTE Öncesi Eğitim Verisinde Scatterplot")
plt.xlabel("Özellik 1")
plt.ylabel("Özellik 2")
plt.legend(title="Churn", labels=["Churn Yok", "Churn Var"])
plt.show()

# SMOTE sonrası scatterplot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_train_smote.iloc[:, 0], # İlk özellik
               y=X_train_smote.iloc[:, 1], # İkinci özellik
               hue=y_train_smote,          # Hedef değişken
               palette=plt.cm.Pastel1.colors)
plt.title("SMOTE Sonrası Eğitim Verisinde Scatterplot")
plt.xlabel("Özellik 1")
plt.ylabel("Özellik 2")
plt.legend(title="Churn", labels=["Churn Yok", "Churn Var"])
plt.show()
```

Smote öncesi scatter plotta dengesiz sınıf dağılımı belirgin bir şekilde görülüyor.

Smote sonrasında bu sorun giderilmiştir. Artık modellemeye geçebiliriz.

SMOTE UYGULANMIŞ ŞEKİLDE MODELLERE BAKALIM:

1)LIGHTGBM

```
from sklearn.metrics import classification_report

#split işlemi
X_train, X_test, y_train, y_test = train_test_split(X_res, y_res,
                                                    test_size=0.20,
                                                    random_state=12345)

# lgbm model kurulumu
lgbm_model=LGBMClassifier(random_state=12345).fit(X_train,y_train)
y_pred = lgbm_model.predict(X_test)

# validasyon hatası, accuracy skoru, confusion matrix
cv_results = cross_val_score(lgbm_model, X_train,y_train, cv = 10, scoring= "accuracy")

print("cross_val_score(train):", cv_results.mean())

cv_results = cross_val_score(lgbm_model, X_test,y_test, cv = 10, scoring= "accuracy")
print("cross_val_score(test):", cv_results.mean())

y_train_pred = lgbm_model.predict(X_train)
print("accuracy_score(train):",accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):",accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix/np.sum(cf_matrix), annot=True,
            fmt='.2%', cmap='Blues');

from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Eğitim ve test için tahmin olasılıkları
y_train_proba = lgbm_model.predict_proba(X_train)[: , 1]
y_test_proba = lgbm_model.predict_proba(X_test)[: , 1]

# ROC eğrisi metrikleri
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()

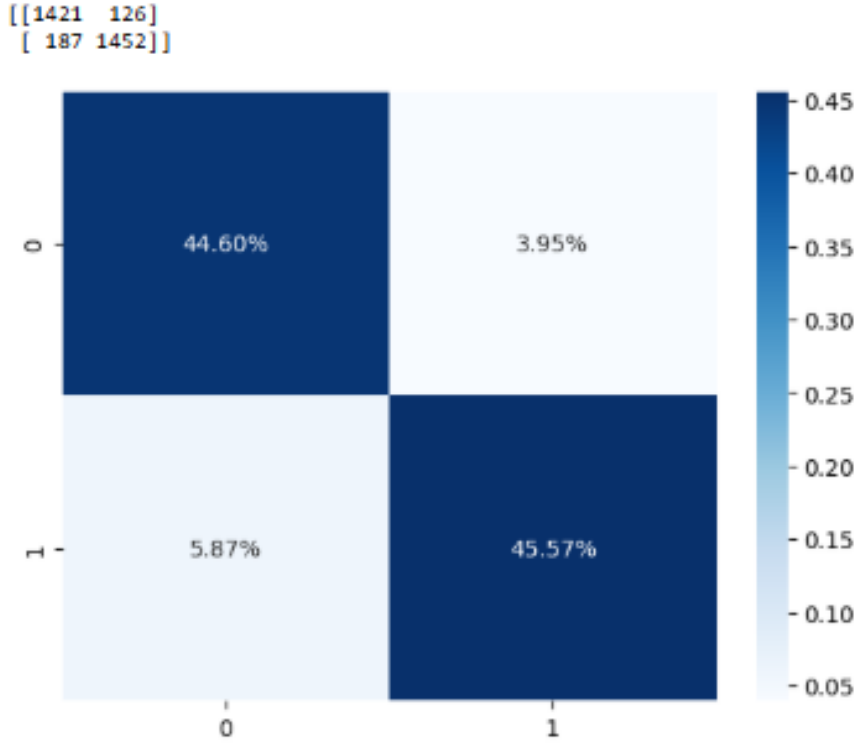
from sklearn.metrics import mean_squared_error
import numpy as np

# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
cross_val_score(test): 0.8741300447546381
accuracy_score(train): 0.932967032967033
accuracy_score(test): 0.9017576898932831
```

	precision	recall	f1-score	support
0	0.88	0.92	0.90	1547
1	0.92	0.89	0.90	1639
accuracy			0.90	3186
macro avg	0.90	0.90	0.90	3186
weighted avg	0.90	0.90	0.90	3186



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Test setindeki çapraz doğrulama doğruluğu: %87,41

Bu değer, çapraz doğrulama sırasında test seti üzerinde elde edilen ortalama başarıdır. %87,41 doğruluk oranı, modelin iyi bir performans sergilediğini gösterir.

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %93,29
- Test seti accuracy: %90,17

Eğitim ve test seti üzerindeki doğruluk oranları yüksek bir doğruluk oranları yüksektir, modelin eğitim ve test verisini iyi öğrendiğini gösterir.

Not: Eğitim setinde doğruluğun test setine göre daha yüksek olması, modelin aşırı öğrenme (overfitting) eğiliminde olabileceğini düşündürebilir. Ancak burada fark çok büyük değil, bu yüzden modelin genelleme performansı iyi görünüyor.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%88) ve Recall (%92) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%92) ve Recall (%89) yüksek, model bu sınıfta da başarılıdır.

Model, her iki sınıfta da nispeten dengeli bir duyarlılık göstermiştir.

4.F1-Score:

- **Churn yok (0):** %90
- **Churn var (1):** %90

Model hem churn yok (0) hem de churn var (1) için performans neredeyse eşit ve dengeli bir performans göstermiştir

6.Support (Destek):

- **Churn yok (0):** 1547 örnek
- **Churn var (1):** 1639 örnek

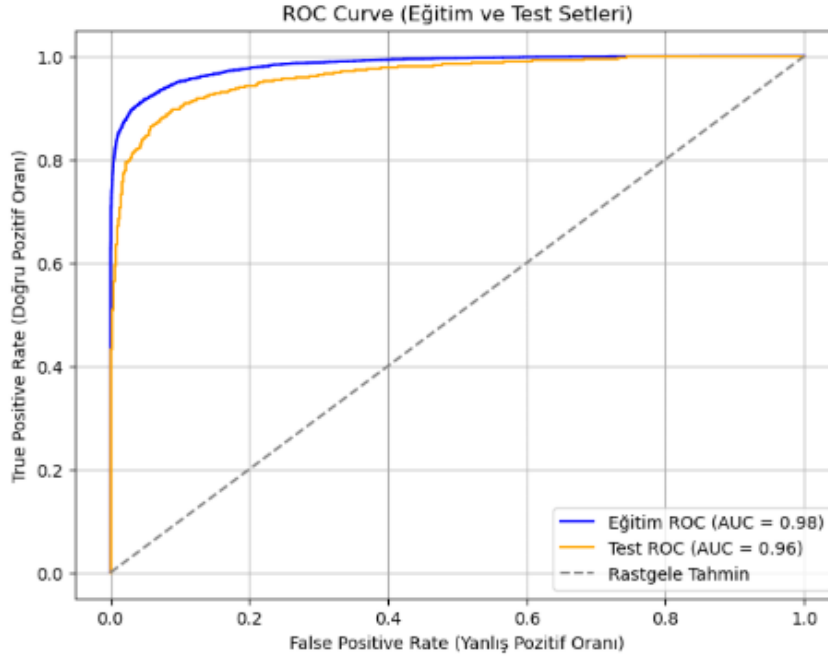
Veri seti nispeten dengeli (1547'ye 1639) olduğu için sınıf dengesizliği kaynaklı sorunlar görülüyor.

7.Genel Performans:

- Accuracy (Doğruluk): 0.90 (Modelin tüm sınıfları birlikte doğru tahmin oranı.)
- Macro Avg: Tüm sınıfların performans ortalaması (precision, recall, f1-score).
- Weighted Avg: Sınıfların destek sayısına göre ağırlıklı ortalaması.

8.Confusion Matrix:

- 1421 örnek churn yok olarak doğru tahmin edilmiş.
- 1452 örnek churn var olarak doğru tahmin edilmiş.
- 126 kişi churn yok iken churn var olarak tahmin edilmiş.
- 187 kişi churn var iken churn yok olarak tahmin edilmiş



Eğitim Seti RMSE: 0.2589
Test Seti RMSE: 0.3134

ROC Eğrisi:

- Eğitim seti: %98
- Test seti: %96

Eğitim setindeki AUC değeri test setinden daha yüksek olsa da bu fark çok büyük değildir. Modelin hem eğitim hem de test setinde iyi ve benzer bir performans gösterdiği söylenebilir.

RMSE Değerleri:

- Eğitim seti: 0,4234
- Test seti: 0,4241

RMSE değerleri arasında fark vardır ancak bu fark overfitting seviyesinin kabul edilebilir düzeyde olduğunu göstermektedir.

Model, ayırım gücü açısından oldukça başarılıdır ve farklı sınıfları ayırt etme yeteneği yüksektir.

1) RANDOM FOREST

```
# RandomForest Modeli
rf_model = RandomForestClassifier(random_state=12345)
rf_model.fit(X_train, y_train)

# Tahminler
y_train_pred = rf_model.predict(X_train)
y_test_pred = rf_model.predict(X_test)

# Cross-Validation Accuracy
cv_results_train = cross_val_score(rf_model, X_train, y_train, cv=10, scoring="accuracy")
cv_results_test = cross_val_score(rf_model, X_test, y_test, cv=10, scoring="accuracy")

print(f"Cross-Validation Accuracy (Train): {cv_results_train.mean():.4f}")
print(f"Cross-Validation Accuracy (Test): {cv_results_test.mean():.4f}")

# Accuracy Scores
print(f"Accuracy Score (Train): {accuracy_score(y_train, y_train_pred):.4f}")
print(f"Accuracy Score (Test): {accuracy_score(y_test, y_test_pred):.4f}")

# Classification Report
print("Classification Report (Test):")
print(classification_report(y_test, y_test_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_test_pred)
print("Confusion Matrix:")
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.title("Random Forest - Confusion Matrix (Test)")
plt.show()

# ROC Eğrisi
y_train_proba = rf_model.predict_proba(X_train)[:, 1]
y_test_proba = rf_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()

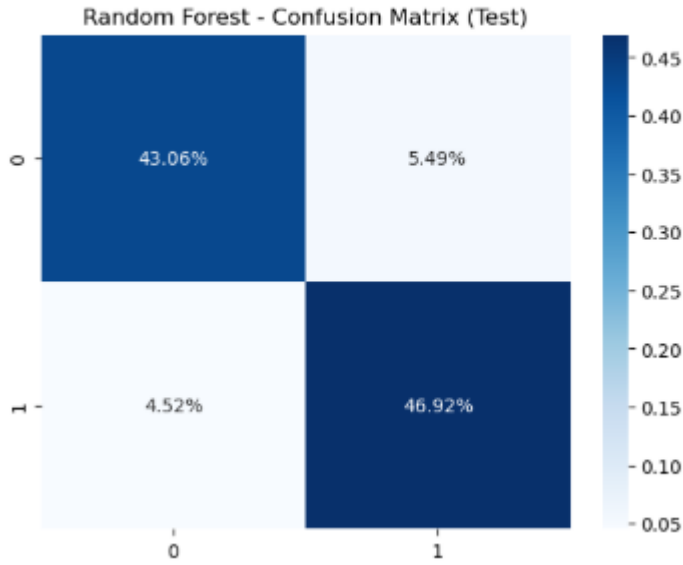
# RMSE Hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
Cross-Validation Accuracy (Train): 0.8940
Cross-Validation Accuracy (Test): 0.8418
Accuracy Score (Train): 1.0000
Accuracy Score (Test): 0.8999
Classification Report (Test):
```

	precision	recall	f1-score	support
0	0.91	0.89	0.90	1547
1	0.90	0.91	0.90	1639
accuracy			0.90	3186
macro avg	0.90	0.90	0.90	3186
weighted avg	0.90	0.90	0.90	3186

Confusion Matrix:
[[1372 175]
[144 1495]]



1. Accurayc (Doğruluk):

- Eğitim seti accuracy: %100
- Test seti accuracy: %89

Bu yüksek doğruluk oranı **overfitting** belirtisi olabilir.

2.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%91) ve Recall (%89) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%90) ve Recall (%91) yüksek, model bu sınıfta da başarılıdır.

3.F1-Score:

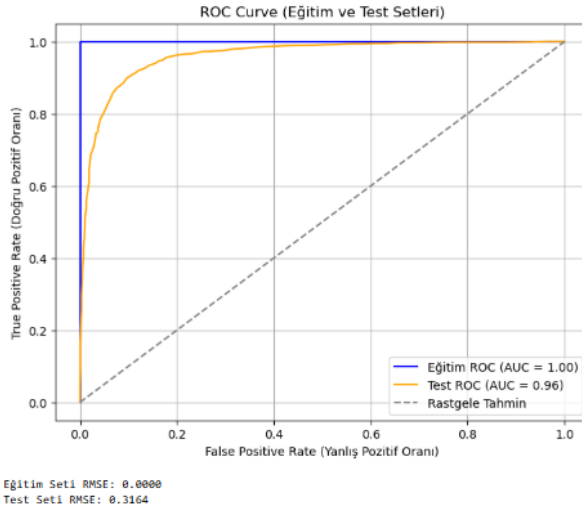
- **Churn yok (0):** %90
- **Churn var (1):** %90

Model her iki sınıf için dengeli bir performans gösteriyor.

Model hem churn olan hem de olmayan müşterileri tespit etmekte başarılıdır.

4.Confusion Matrix:

- 1372 örnek churn yok olarak doğru tahmin edilmiş.
- 1495 örnek churn var olarak doğru tahmin edilmiş.
- 175 kişi churn yok iken churn var olarak tahmin edilmiş.
- 144 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %100
- Test seti: %96

Model, eğitim setinde mükemmel bir performans göstermiştir. Bu, modelin eğitim verisine çok iyi uyum sağladığını, hatta fazla uyum sağladığını, yani **overfitting** riski olduğunu gösteriyor.

RMSE Değerleri:

- Eğitim seti: 0,00
- Test seti: 0,3164

3)XGBOOST

```
# XGBoost modelinin kurulumu ve eğitimi
xgb_model = XGBClassifier(random_state=12345, use_label_encoder=False, eval_metric="logloss")
xgb_model.fit(X_train, y_train)

# Test tahminleri
y_pred = xgb_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(xgb_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(xgb_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = xgb_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# Eğitim ve test için tahmin olasılıkları
y_train_proba = xgb_model.predict_proba(X_train)[:, 1]
y_test_proba = xgb_model.predict_proba(X_test)[:, 1]

# ROC eğrisi metrikleri
fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

```

cross_val_score(train): 0.8967817896389325
cross_val_score(test): 0.8672315214605391
accuracy_score(train): 0.9652276295133438
accuracy_score(test): 0.8983050847457628

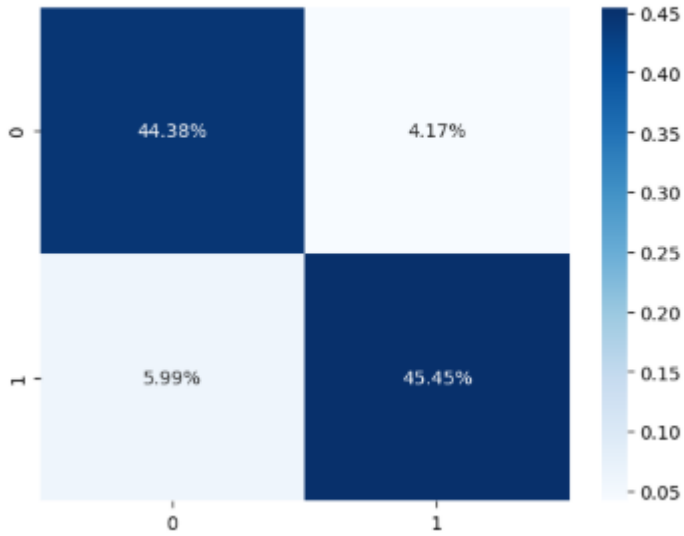
```

	precision	recall	f1-score	support
0	0.88	0.91	0.90	1547
1	0.92	0.88	0.90	1639
accuracy			0.90	3186
macro avg	0.90	0.90	0.90	3186
weighted avg	0.90	0.90	0.90	3186

```

[[1414 133]
 [ 191 1448]]

```



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %89,67
- Test setindeki çapraz doğrulama doğruluğu: %86,72

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %96
- Test seti accuracy: %89

Bu yüksek doğruluk oranı **overfitting** belirtisi olabilir.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%88) ve Recall (%91) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%92) ve Recall (%88) yüksek, model bu sınıfta da başarılıdır.

4.F1-Score:

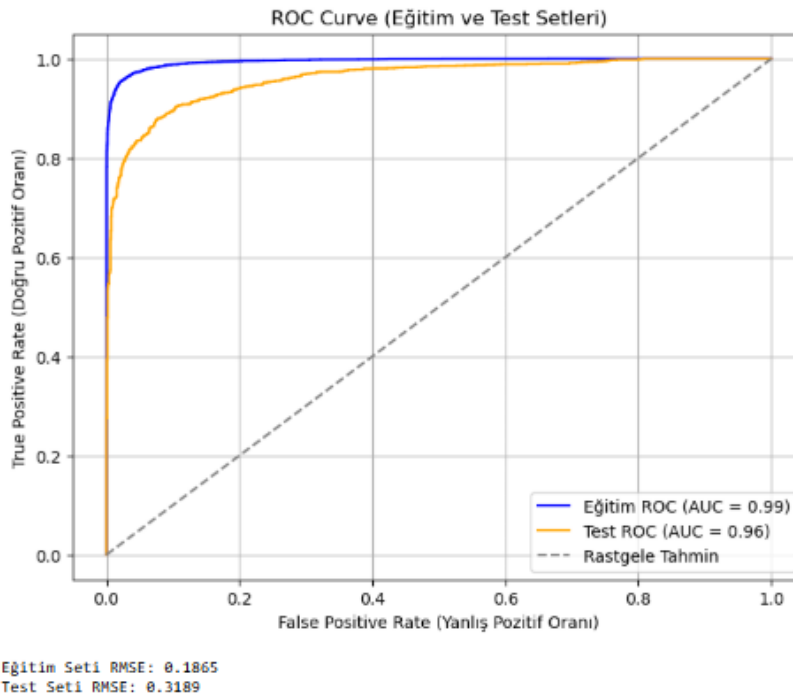
- **Churn yok (0):** %90
- **Churn var (1):** %90

Model her iki sınıf için dengeli bir performans gösteriyor.

Model hem churn olan hem de olmayan müşterileri tespit etmekte başarılıdır.

5.Confusion Matrix:

- 1414 örnek churn yok olarak doğru tahmin edilmiş.
- 1448 örnek churn var olarak doğru tahmin edilmiş.
- 133 kişi churn yok iken churn var olarak tahmin edilmiş.
- 191 kişi churn var iken churn yok olarak tahmin edilmiş



ROC Eğrisi:

- Eğitim seti: %99
- Test seti: %96

Model, yüksek eğitim performansı (0.99) ve test performansındaki küçük düşüşle **çok hafif overfitting** göstermektedir.

RMSE Değerleri:

- Eğitim seti: 0,1865
- Test seti: 0,3189

AUC değerleri ve düşük RMSE sonuçları, modelin iyi bir genelleme yaptığını ve veriyi doğru bir şekilde sınıflandırdığını gösteriyor.

4)ADABOOSTING

```
# AdaBoost modelinin kurulumu ve eğitimi
ada_model = AdaBoostClassifier(random_state=12345)
ada_model.fit(X_train, y_train)

# Test tahminleri
y_pred = ada_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(ada_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(ada_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = ada_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = ada_model.predict_proba(X_train)[:, 1]
y_test_proba = ada_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

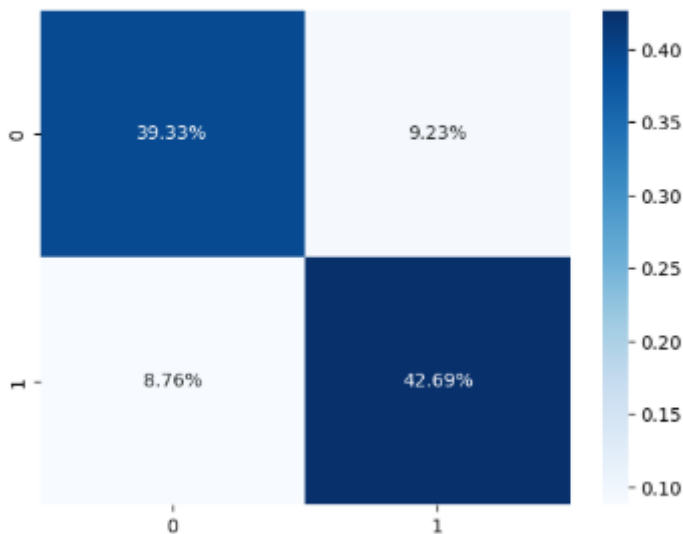
```
cross_val_score(train): 0.818995290423862
cross_val_score(test): 0.8066501054789927
accuracy_score(train): 0.8207221350078493
accuracy_score(test): 0.82015065913371

      precision    recall  f1-score   support

     0       0.82       0.81       0.81       1547
     1       0.82       0.83       0.83       1639

 accuracy          0.82
 macro avg       0.82       0.82       0.82       3186
weighted avg       0.82       0.82       0.82       3186
```

```
[[1253 294]
 [ 279 1360]]
```



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %81,9
- Test setindeki çapraz doğrulama doğruluğu: %80,66

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %82
- Test seti accuracy: %82

Bu, modelin genel olarak dengeli bir performans sergilediğini gösterir

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%82) ve Recall (%81) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%82) ve Recall (%83) yüksek, model bu sınıfta da başarılıdır.

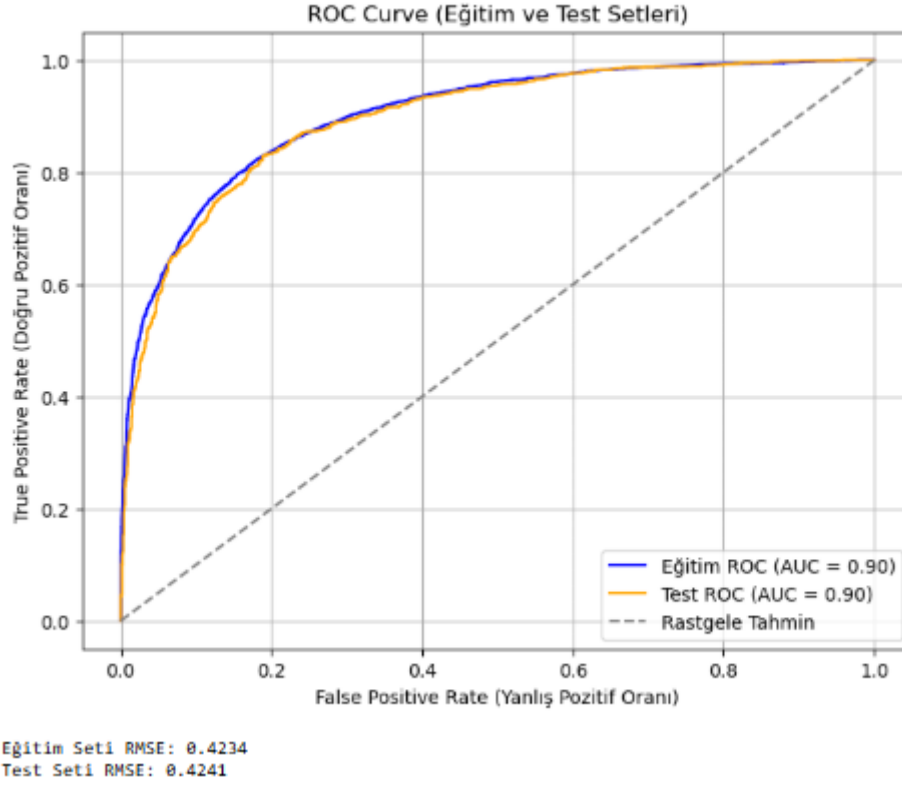
4.F1-Score:

- **Churn yok (0):** %81
- **Churn var (1):** %83

F1-score değerleri sınıflar arasında benzer. Bu modelin sınıf dengesizliklerini iyi ele aldığını gösterir.

5.Confusion Matrix:

- 1253 örnek churn yok olarak doğru tahmin edilmiş.
- 1360 örnek churn var olarak doğru tahmin edilmiş.
- 294 kişi churn yok iken churn var olarak tahmin edilmiş.
- 279 kişi churn var iken churn yok olarak tahmin edilmiş.



ROC Eğrisi:

- Eğitim seti: %90
- Test seti: %90

Bu değerler modelin hem eğitim hem de test setlerinde dengeli bir performans gösterdiğini ve sınıflandırma yeteneğinin iyi olduğunu gösteriyor.

Eğitim ve test eğrileri birbirine oldukça yakın ve bu durum overfitting riskinin düşük olduğunu işaret ediyor. Model hem eğitim hem de test verilerinde benzer bir performans sergiliyor.

RMSE Değerleri:

- Eğitim seti: 0,4234
- Test seti: 0,4241

5) CATBOOST

```
# CatBoost modelinin kurulumu ve eğitimi
catboost_model = CatBoostClassifier(iterations=1000, learning_rate=0.1, depth=6, random_state=12345, cat_features=[], verbose=20)
catboost_model.fit(X_train, y_train)

# Test tahminleri
y_pred = catboost_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(catboost_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(catboost_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = catboost_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = catboost_model.predict_proba(X_train)[:, 1]
y_test_proba = catboost_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()

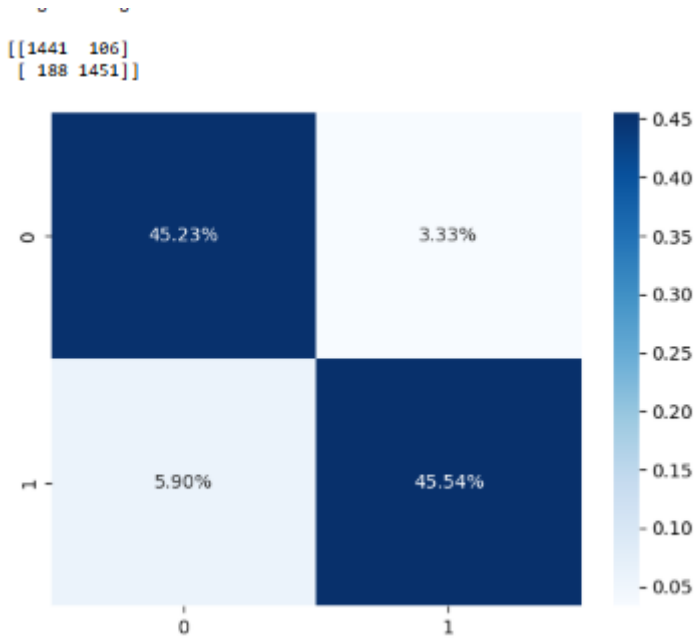
# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
-----
cross_val_score(test): 0.8731846769582619
accuracy_score(train): 0.9832810047095761
accuracy_score(test): 0.9077212806026366
      precision    recall  f1-score   support

     0       0.88       0.93       0.91       1547
     1       0.93       0.89       0.91       1639

 accuracy          0.91          0.91          0.91       3186
 macro avg         0.91          0.91          0.91       3186
 weighted avg         0.91          0.91          0.91       3186
```



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Test setindeki çapraz doğrulama doğruluğu: %87,31

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %98
- Test seti accuracy: %90

Bu yüksek doğruluk oranı **overfitting** belirtisi olabilir.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%88) ve Recall (%93) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%93) ve Recall (%89) yüksek, model bu sınıfta da başarılıdır.

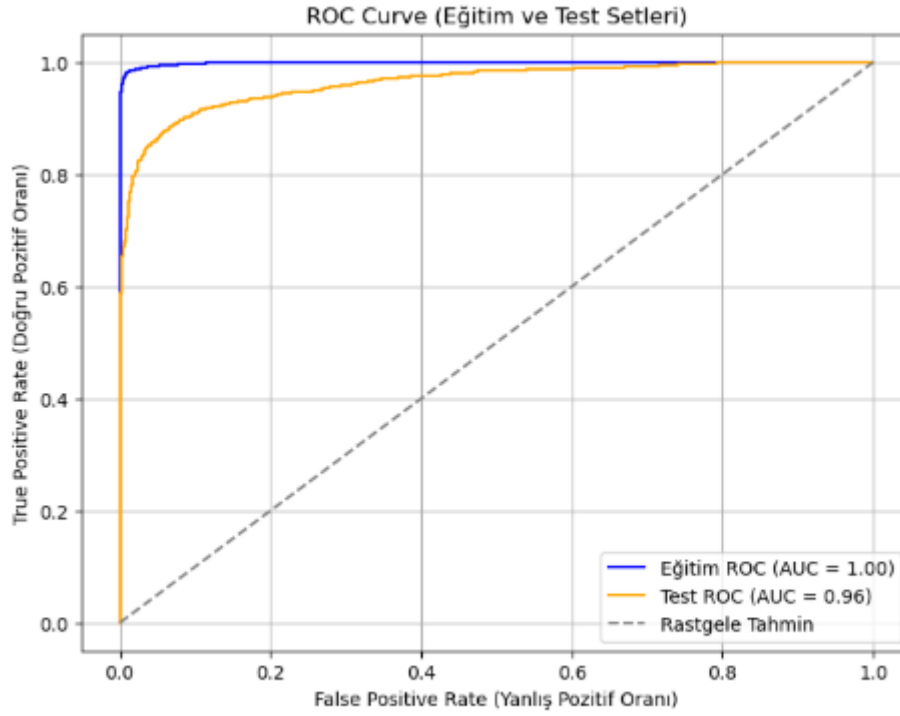
4.F1-Score:

- **Churn yok (0):** %91
- **Churn var (1):** %91

F1-score değerleri sınıflar arasında benzer. Bu modelin sınıf dengesizliklerini iyi ele aldığını gösterir.

5.Confusion Matrix:

- 1441 örnek churn yok olarak doğru tahmin edilmiş.
- 1451 örnek churn var olarak doğru tahmin edilmiş.
- 106 kişi churn yok iken churn var olarak tahmin edilmiş.
- 188 kişi churn var iken churn yok olarak tahmin edilmiş.



Eğitim Seti RMSE: 0.1293
Test Seti RMSE: 0.3038

ROC Eğrisi:

- Eğitim seti: %100
- Test seti: %96

Model, eğitim setinde mükemmel bir performans göstermiştir. Bu, modelin eğitim verisine çok iyi uyum sağladığını, hatta fazla uyum sağladığını, yani **overfitting** riski olduğunu gösteriyor.

RMSE Değerleri:

- Eğitim seti: 0,1293
- Test seti: 0,3038

6)GRADİENT BOOSTING

```
# Gradient Boosting modelinin kurulumu ve eğitimi
gb_model = GradientBoostingClassifier(random_state=12345)
gb_model.fit(X_train, y_train)

# Test tahminleri
y_pred = gb_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(gb_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(gb_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = gb_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = gb_model.predict_proba(X_train)[:, 1]
y_test_proba = gb_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

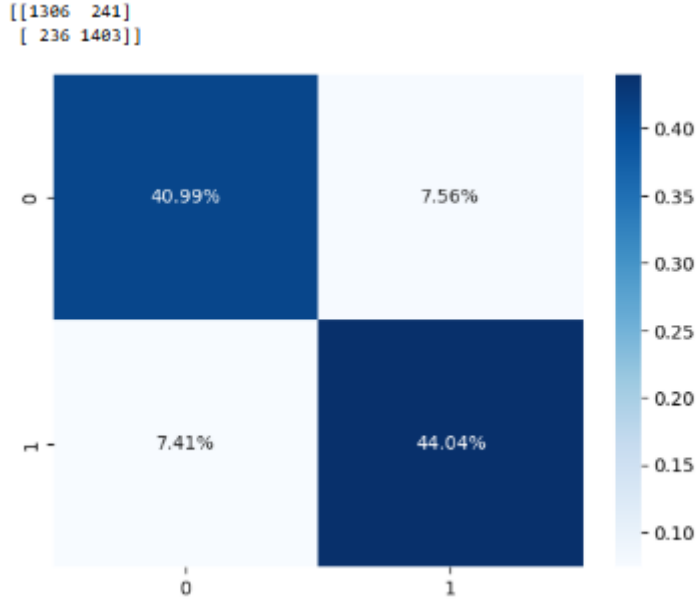
# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()

# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
cross_val_score(train): 0.8510989010989011
cross_val_score(test): 0.8427475798978726
accuracy_score(train): 0.8624018838304552
accuracy_score(test): 0.8502824858757062
```

	precision	recall	f1-score	support
0	0.85	0.84	0.85	1547
1	0.85	0.86	0.85	1639
accuracy			0.85	3186
macro avg	0.85	0.85	0.85	3186
weighted avg	0.85	0.85	0.85	3186



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %85
- Test setindeki çapraz doğrulama doğruluğu: %84

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %86
- Test seti accuracy: %85

Bu, modelin genel olarak dengeli bir performans sergilediğini gösterir

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%85) ve Recall (%84) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%85) ve Recall (%86) yüksek, model bu sınıfta da başarılıdır.

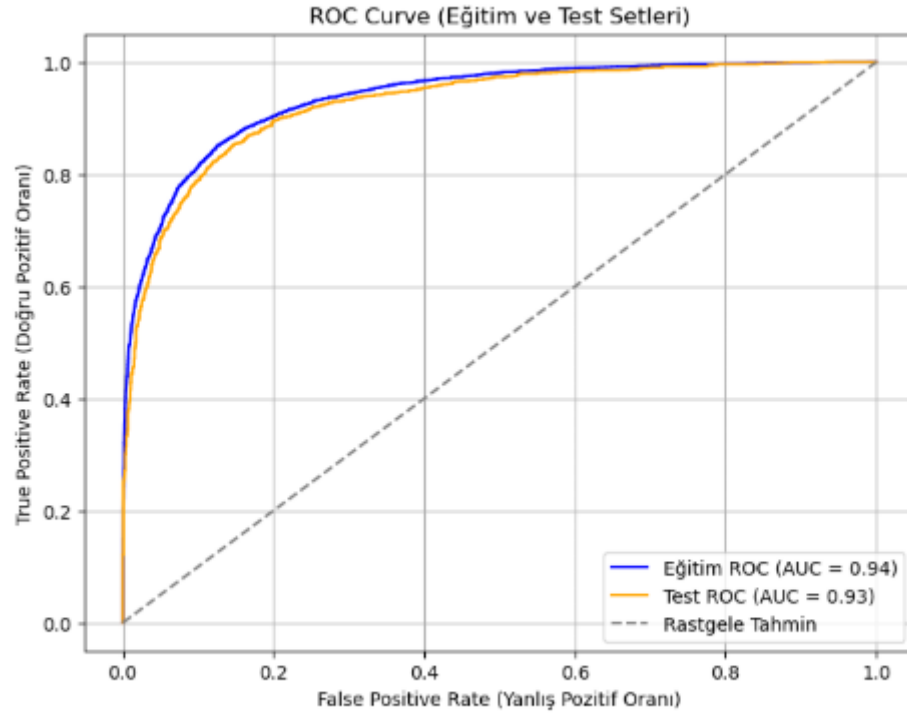
4.F1-Score:

- **Churn yok (0):** %85
- **Churn var (1):** %85

F1-score değerleri sınıflar arasında benzer. Bu modelin sınıf dengesizliklerini iyi ele aldığını gösterir.

5.Confusion Matrix:

- 1306 örnek churn yok olarak doğru tahmin edilmiş.
- 1403 örnek churn var olarak doğru tahmin edilmiş.
- 241 kişi churn yok iken churn var olarak tahmin edilmiş.
- 236 kişi churn var iken churn yok olarak tahmin edilmiş.



Eğitim Seti RMSE: 0.3709
Test Seti RMSE: 0.3869

ROC Eğrisi:

- Eğitim seti: %94
- Test seti: %93

Bu değerler modelin hem eğitim hem de test setlerinde dengeli bir performans gösterdiğini ve sınıflandırma yeteneğinin iyi olduğunu gösteriyor.

Eğitim ve test eğrileri birbirine oldukça yakın ve bu durum overfitting riskinin düşük olduğunu işaret ediyor. Model hem eğitim hem de test verilerinde benzer bir performans sergiliyor.

RMSE Değerleri:

- Eğitim seti: 0,3709
- Test seti: 0,3869

7) NAVİE BAYES

```
# Gaussian Naive Bayes modelinin kurulumu ve eğitimi
gnb_model = GaussianNB()
gnb_model.fit(X_train, y_train)

# Test tahminleri
y_pred = gnb_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(gnb_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(gnb_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = gnb_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = gnb_model.predict_proba(X_train)[:, 1]
y_test_proba = gnb_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

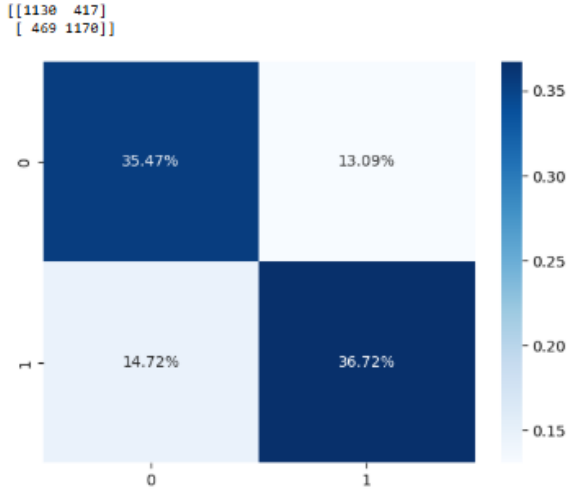
# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

```
# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
cross_val_score(train): 0.7204866562009419
cross_val_score(test): 0.7149987184795252
accuracy_score(train): 0.7207221350078493
accuracy_score(test): 0.7219083490269931
```

	precision	recall	f1-score	support
0	0.71	0.73	0.72	1547
1	0.74	0.71	0.73	1639
accuracy			0.72	3186
macro avg	0.72	0.72	0.72	3186
weighted avg	0.72	0.72	0.72	3186



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %72
- Test setindeki çapraz doğrulama doğruluğu: %71

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %72
- Test seti accuracy: %72

Bu, modelin genel olarak dengeli bir performans sergilediğini, **overfitting** yapmadığını gösterir. Ancak doğruluk oranı nispeten düşüktür.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%71) ve Recall (%73), model bu sınıfta ortalama performans sergilemiştir.
- **Churn var (1):** Precision (%74) ve Recall (%71) model bu sınıfta ortalama performans sergilemiştir.

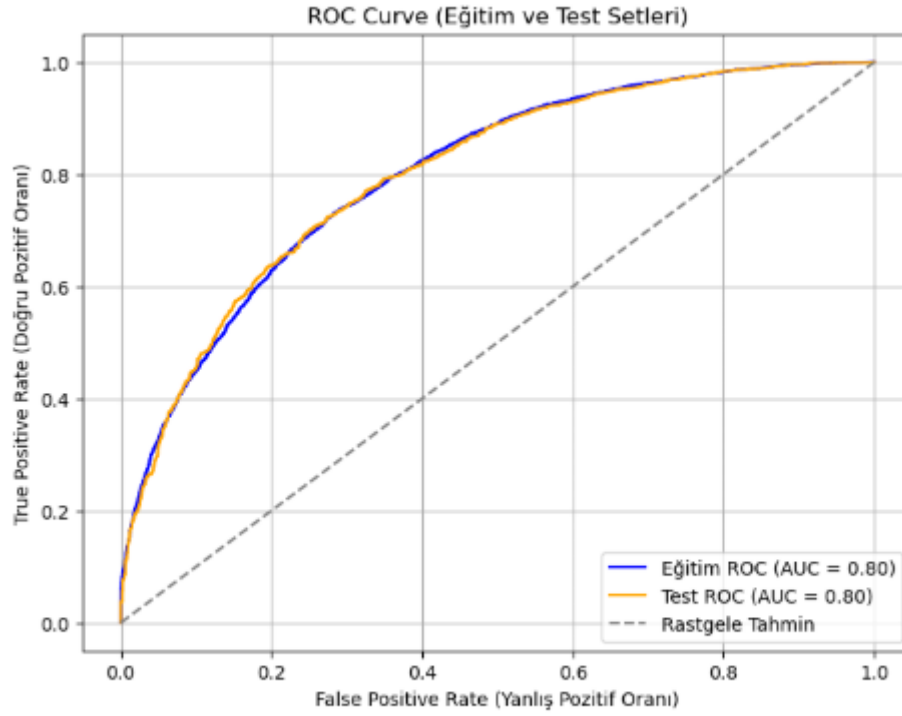
4.F1-Score:

- **Churn yok (0):** %72
- **Churn var (1):** %73

F1-score değerleri sınıflar arasında benzer.

5.Confusion Matrix:

- 1130 örnek churn yok olarak doğru tahmin edilmiş.
- 1170 örnek churn var olarak doğru tahmin edilmiş.
- 417 kişi churn yok iken churn var olarak tahmin edilmiş.
- 469 kişi churn var iken churn yok olarak tahmin edilmiş.



Eğitim Seti RMSE: 0.5285
Test Seti RMSE: 0.5273

ROC Eğrisi:

- Eğitim seti: %80
- Test seti: %80

Hem eğitim hem de test seti eğrilerinin birlikte görülmesi, **overfitting olmadığını** ve modelin farklı veri kümelerinde tutarlı performans gösterdiğini ifade eder.

RMSE Değerleri:

- Eğitim seti: 0,5285
- Test seti: 0,5273

Kısaca, genel doğruluk ve diğer metrikler dengelidir. Ancak modelin genel performansı düşük ila orta düzeyde kabul edilebilir.

8) KNN

```
# KNN modelinin kurulumu ve eğitimi
knn_model = KNeighborsClassifier(n_neighbors=5) # Burada n_neighbors parametresi ayarlanabilir
knn_model.fit(X_train, y_train)

# Test tahminleri
y_pred = knn_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(knn_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(knn_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = knn_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = knn_model.predict_proba(X_train)[:, 1]
y_test_proba = knn_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

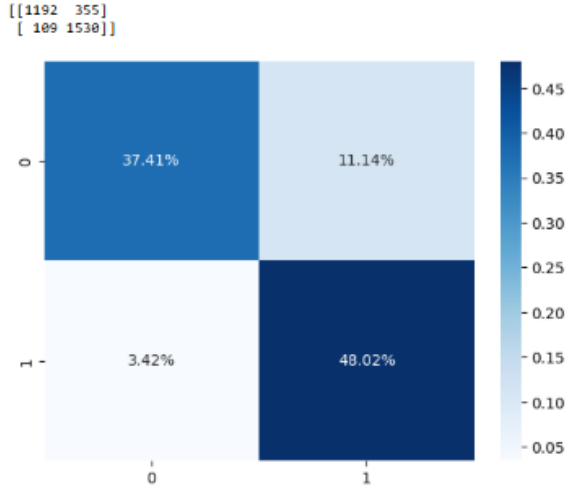
# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()

# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")
```

```
cross_val_score(train): 0.8542386185243327
cross_val_score(test): 0.76805070887794011
accuracy_score(train): 0.9086342229199372
accuracy_score(test): 0.8543628374136849
```

	precision	recall	f1-score	support
0	0.92	0.77	0.84	1547
1	0.81	0.93	0.87	1639
accuracy			0.85	3186
macro avg	0.86	0.85	0.85	3186
weighted avg	0.86	0.85	0.85	3186



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %85
- Test setindeki çapraz doğrulama doğruluğu: %76

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %90
- Test seti accuracy: %85

Eğitim doğruluğunun test doğruluğundan belirgin şekilde daha yüksek olması, **overfitting** ihtimalini gösterebilir. Eğitim setindeki performans yüksekken, test setindeki performans daha düşüktür.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%92) ve Recall (%77) yüksek, model bu sınıfta başarılıdır.
- **Churn var (1):** Precision (%81) ve Recall (%93) yüksek, model bu sınıfta da başarılıdır.

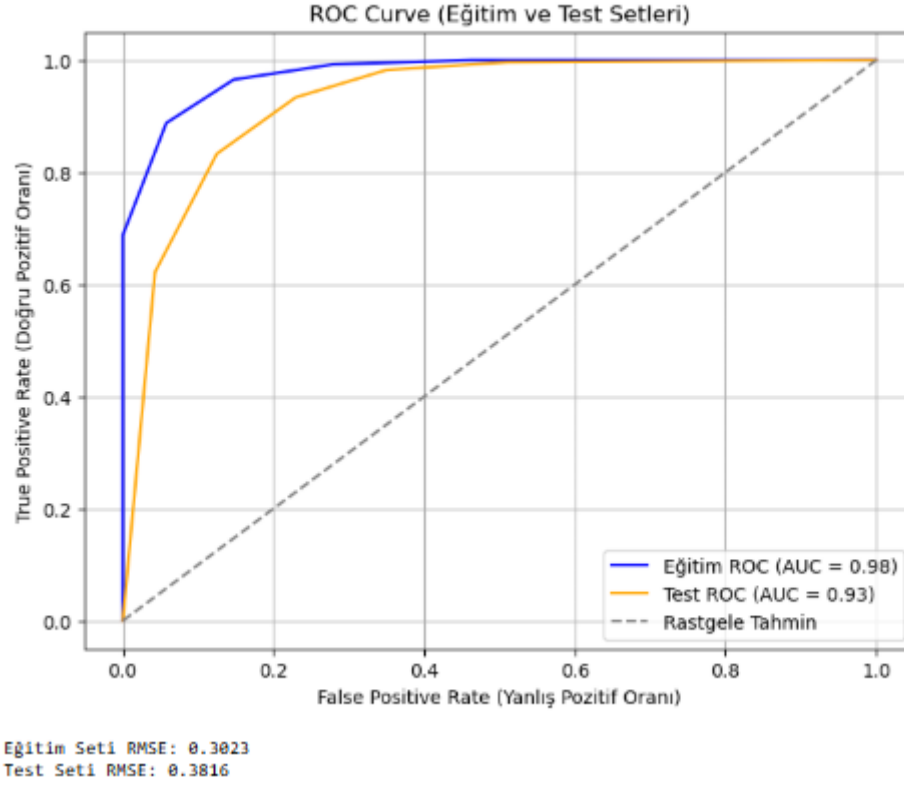
4.F1-Score:

- **Churn yok (0):** %84
- **Churn var (1):** %87

Model churn var sınıfında (1) daha başarılı, ancak churn yok sınıfında (0) bazı hatalar yapıyor.

5.Confusion Matrix:

- 1192 örnek churn yok olarak doğru tahmin edilmiş.
- 1530 örnek churn var olarak doğru tahmin edilmiş.
- 355 kişi churn yok iken churn var olarak tahmin edilmiş.
- 109 kişi churn var iken churn yok olarak tahmin edilmiş.



ROC Eğrisi:

- Eğitim seti: %98
- Test seti: %93

Model, yüksek eğitim performansı (0.98) ve test performansındaki küçük düşüşle **hafif overfitting** olabileceğini göstermektedir.

RMSE Değerleri:

- Eğitim seti: 0,3023
- Test seti: 0,3816

Eğitim ve test setleri arasındaki RMSE farkı, hafif overfitting olduğunu ve modelin hem eğitim hem de test verisinde benzer bir başarıyla çalıştığını göstermektedir.

AUC değerleri ve düşük RMSE sonuçları, modelin iyi bir genelleme yaptığını ve veriyi doğru bir şekilde sınıflandırdığını gösteriyor.

9) KARAR AĞAÇLARI

```
# Karar Ağacı modelinin kurulumu ve eğitimi
dt_model = DecisionTreeClassifier(random_state=12345, max_depth=3) # max_depth ile ağacın derinliğini sınırlıyoruz
dt_model.fit(X_train, y_train)

# Test tahminleri
y_pred = dt_model.predict(X_test)

# Cross-validation sonuçları (Eğitim Seti)
cv_results_train = cross_val_score(dt_model, X_train, y_train, cv=10, scoring="accuracy")
print("cross_val_score(train):", cv_results_train.mean())

# Cross-validation sonuçları (Test Seti)
cv_results_test = cross_val_score(dt_model, X_test, y_test, cv=10, scoring="accuracy")
print("cross_val_score(test):", cv_results_test.mean())

# Accuracy score
y_train_pred = dt_model.predict(X_train)
print("accuracy_score(train):", accuracy_score(y_train, y_train_pred))
print("accuracy_score(test):", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Confusion Matrix
cf_matrix = confusion_matrix(y_test, y_pred)
print(cf_matrix)
sns.heatmap(cf_matrix / np.sum(cf_matrix), annot=True, fmt='.2%', cmap='Blues')
plt.show()

# ROC eğrisi metrikleri
y_train_proba = dt_model.predict_proba(X_train)[:, 1]
y_test_proba = dt_model.predict_proba(X_test)[:, 1]

fpr_train, tpr_train, _ = roc_curve(y_train, y_train_proba)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_proba)

# ROC eğrisi çizimi
plt.figure(figsize=(8, 6))
plt.plot(fpr_train, tpr_train, label=f"Eğitim ROC (AUC = {roc_auc_score(y_train, y_train_proba):.2f})", color="blue")
plt.plot(fpr_test, tpr_test, label=f"Test ROC (AUC = {roc_auc_score(y_test, y_test_proba):.2f})", color="orange")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Rastgele Tahmin")
plt.xlabel("False Positive Rate (Yanlış Pozitif Oranı)")
plt.ylabel("True Positive Rate (Doğru Pozitif Oranı)")
plt.title("ROC Curve (Eğitim ve Test Setleri)")
plt.legend(loc="lower right")
plt.grid()
plt.show()
```

```
# RMSE hesaplama
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

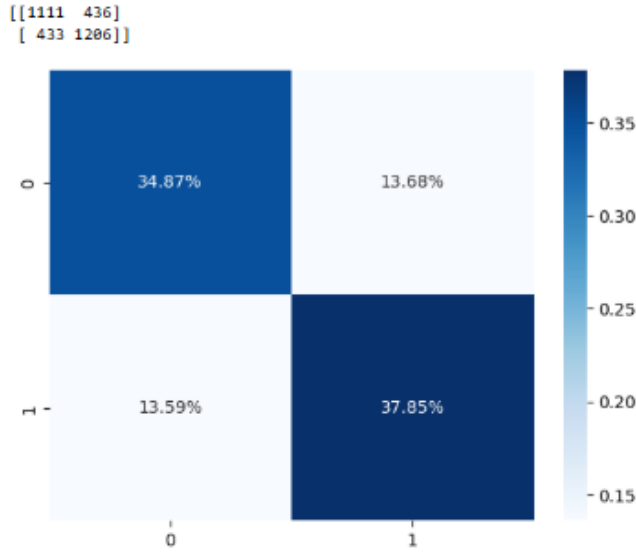
print(f"Eğitim Seti RMSE: {train_rmse:.4f}")
print(f"Test Seti RMSE: {test_rmse:.4f}")

# Karar ağacını Graphviz ile görselleştirme
dot_data = export_graphviz(
    dt_model,
    out_file=None,
    feature_names=X.columns,
    class_names=["Churn Yok", "Churn Var"],
    filled=True,
    rounded=True,
    special_characters=True
)

graph = graphviz.Source(dot_data)
graph.render("decision_tree", format="png", cleanup=True) # Karar ağacını bir PNG dosyasına kaydeder
graph.view() # Karar ağacını gösterir
```

```
cross_val_score(train): 0.7430926216640502
cross_val_score(test): 0.731944362295696
accuracy_score(train): 0.7430141287284144
accuracy_score(test): 0.7272441933458883
```

	precision	recall	f1-score	support
0	0.72	0.72	0.72	1547
1	0.73	0.74	0.74	1639
accuracy			0.73	3186
macro avg	0.73	0.73	0.73	3186
weighted avg	0.73	0.73	0.73	3186



1.Çapraz Doğrulama ve Doğruluk Skorları:

- Eğitim setindeki çapraz doğrulama doğruluğu: %74
- Test setindeki çapraz doğrulama doğruluğu: %73

2. Accurayc (Doğruluk):

- Eğitim seti accuracy: %74
- Test seti accuracy: %72

Bu, modelin genel olarak dengeli bir performans sergilediğini gösterir.

3.Recall (Duyarlılık) ve Precision (Kesinlik):

- **Churn yok (0):** Precision (%72) ve Recall (%72) model bu sınıfta ortalama performans sergilemiştir.
- **Churn var (1):** Precision (%73) ve Recall (%74) model bu sınıfta ortalama performans sergilemiştir.

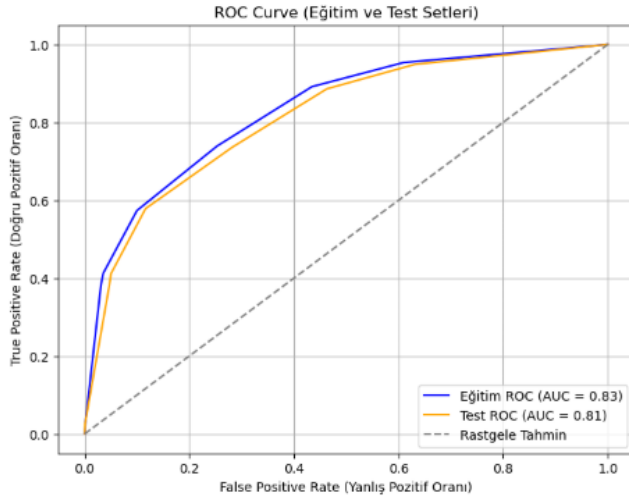
4.F1-Score:

- **Churn yok (0):** %72
- **Churn var (1):** %74

F1-score değerleri sınıflar arasında benzerdir.

5.Confusion Matrix:

- 1111 örnek churn yok olarak doğru tahmin edilmiş.
- 1206 örnek churn var olarak doğru tahmin edilmiş.
- 436 kişi churn yok iken churn var olarak tahmin edilmiş.
- 433 kişi churn var iken churn yok olarak tahmin edilmiş.



Eğitim Seti RMSE: 0.5069
Test Seti RMSE: 0.5223

ROC Eğrisi:

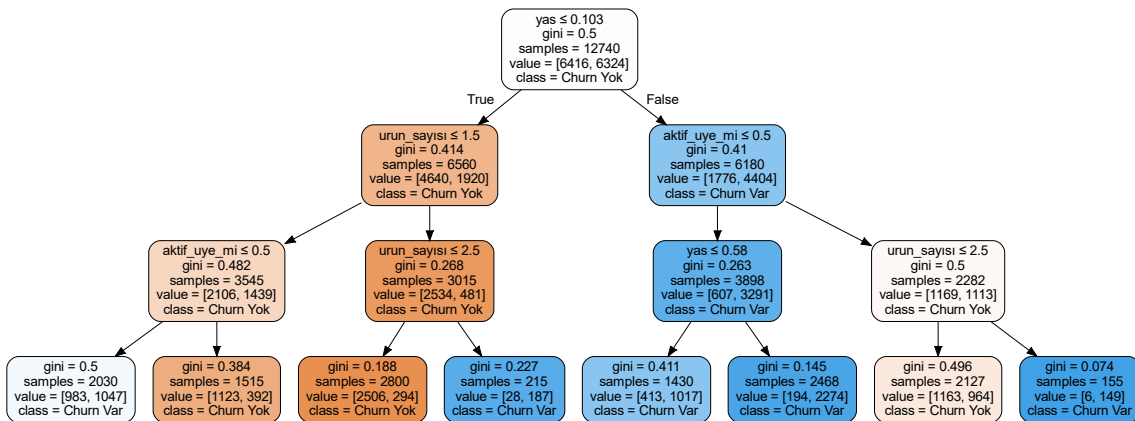
- Eğitim seti: %83
- Test seti: %81

Eğitim ve test setindeki AUC değerleri birbirine oldukça yakındır. Bu durum, modelin genelleme kapasitesinin iyi olduğunu ve overfitting belirtisi göstermediğini ifade eder.

RMSE Değerleri:

- Eğitim seti: 0,5069
- Test seti: 0,5223

Kısaca, genel doğruluk ve diğer metrikler dengelidir. Ancak modelin genel performansı orta düzeyde kabul edilebilir.



MODEL SEÇİMİ

Modellerin birçoğunda aşırı öğrenme (overfitting) var. O modelleri eledikten sonra en küçük hataya sahip olan ve model performansı yüksek olan “lightGBM” modelini veri setimizi en iyi tahmin edecek model olarak tercih ederiz.

Teste uygulama:

```
# Test kümesindeki tahminler
y_test_predictions = lgbm_model.predict(X_test) # Sınıf tahminleri
y_test_probabilities = lgbm_model.predict_proba(X_test)[:, 1] # Churn olasılıkları (pozitif sınıf)

# Sadece gerekli sütunları içeren DataFrame oluşturma
test_results = pd.DataFrame({
    'Gerçek': y_test.values,
    'Tahmin': y_test_predictions,
    'Churn Olasılığı': y_test_probabilities
})

# İlk birkaç satırı gösterme
test_results
```

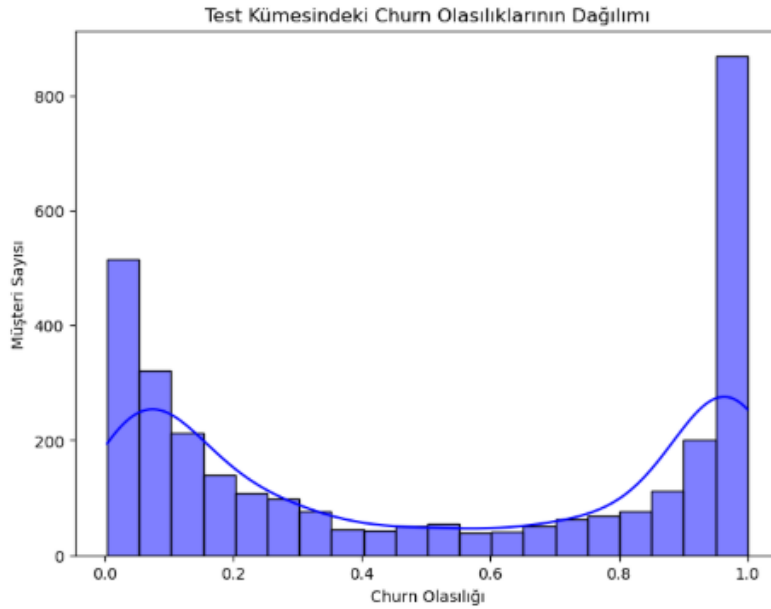
	Gerçek	Tahmin	Churn Olasılığı
0	1	1	0.939
1	1	1	0.996
2	0	0	0.051
3	1	1	0.992
4	1	1	0.963
5	0	0	0.031
6	0	0	0.068
7	1	1	0.996
8	0	1	0.970
9	1	0	0.377
10	0	0	0.069

Model, test kümesindeki her bir müşteri için churn (1) veya churn değil (0) tahminlerini verecektir.

- **Gerçek (y_test):** Test setindeki gerçek churn durumu.
- **Tahmin:** Modelin churn tahmini.
- **Churn Olasılığı:** Modelin churn olasılığı tahmini

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.histplot(y_test_probabilities, bins=20, kde=True, color="blue")
plt.title("Test Kümesindeki Churn Olasılıklarının Dağılımı")
plt.xlabel("Churn Olasılığı")
plt.ylabel("Müşteri Sayısı")
plt.show()
```



Churn olasılığı 0,0 ve 1,0 civarında yüksek yoğunluk gözlemleniyor. Bu, modelin birçok müşteriyi yüksek güvenle churn etmeyecek (0,0) veya churn edecek (1,0) şekilde sınıflandırdığını gösteriyor.

0,4 ile 0,6 arasında churn olasılıkları oldukça az. Bu durum, modelin kararında kararsız kaldığı müşteri sayısının düşük olduğunu ifade edebilir.

Churn olasılıklarının uç değerlere (0 ve 1) yığılması, modelin churn tahminlerinde oldukça iddialı olduğunu gösterebilir.

SHAP İLE ÖZELLİK ÖNEM GRAFİĞİ VE YORUMLANMASI

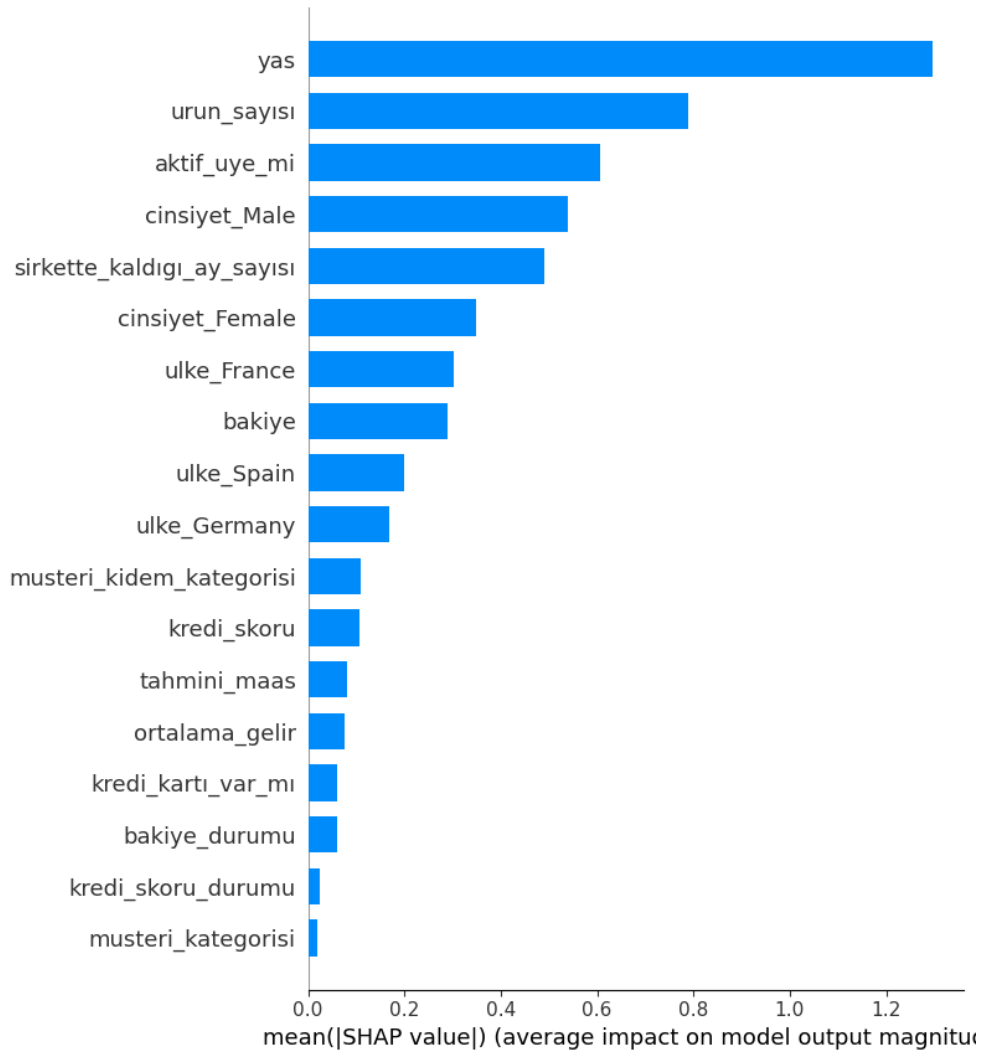
SHAP analizi, bir makine öğrenimi modelinin (burda LightGBM) tahminleri üzerindeki değişkenlerin etkisini değerlendirir.

Özellikle, SHAP değerleri, her bir özelliğin modelin kararına yaptığı katkıyı hesaplar.

```
#!/pip install shap

import shap

shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, plot_type="bar")
```



Önemli Özellikler:

1. Yaş (yas):

- En yüksek etkiye sahip özellik. Müşteri yaşı, modelin churn tahmininde belirleyici bir faktör.

2. Ürün Sayısı (urun_sayisi):

- İkinci sırada. Müşterilerin kullandığı ürünlerin sayısı churn tahmininde güçlü bir faktör.
- Daha az ürün kullanan müşterilerin şirketi terk etme ihtimali daha yüksek olabilir.

3. Aktif Üyelik (aktif_uye_mi):

- Müşterinin aktif üyelik durumu, churn tahmininde üçüncü sırada. Aktif olmayan müşterilerin churn etme olasılığı daha yüksek olabilir.

4. Cinsiyet (cinsiyet_Male ve cinsiyet_Female):

- Cinsiyet de churn üzerinde etkili. Erkek ya da kadın müşteriler arasında farklı churn davranışları gözlemlenmiş olabilir.

5. Şirkette Kaldığı Ay Sayısı (sirkette_kaldigi_ay_sayisi):

- Şirkette daha uzun süre kalan müşteriler genelde daha düşük churn riskine sahip olabilir.

6. Ülke (ulke_France, ulke_Spain, ulke_Germany):

- Müşterinin yaşadığı ülke de churn tahmininde etkili. Bu, ülkeler arasındaki farklı müşteri davranışlarından kaynaklanabilir.

7. Bakiye (bakiye):

- Müşterilerin hesap bakiyeleri churn tahmininde önemli bir rol oynuyor. Daha düşük bakiyeye sahip müşteriler daha yüksek churn riski taşıyor olabilir.

Daha Az Etkili Özellikler:

- Müşteri Kıdem Kategorisi (musteri_kidem_kategorisi), Kredi Skoru (kredi_skoru), Tahmini Maaş (tahmini_maas), Ortalama Gelir (ortalama_gelir):
 - Bu özellikler churn tahmininde daha az etkili olmuştur.
- Kredi Kartı Var mı (kredi_karti_var_mi), Bakiye Durumu (bakiye_durumu), Kredi Skoru Durumu (kredi_skoru_durumu):
 - Daha düşük önem seviyesine sahiptir. Bu, modelin bu özelliklere daha az bağımlı olduğunu gösterir.

Elde edilen bu bilgiler, müşteri davranışlarını anlamak ve churn riskini azaltmak için iş stratejileri geliştirmekte kullanılabilir. Örneğin:

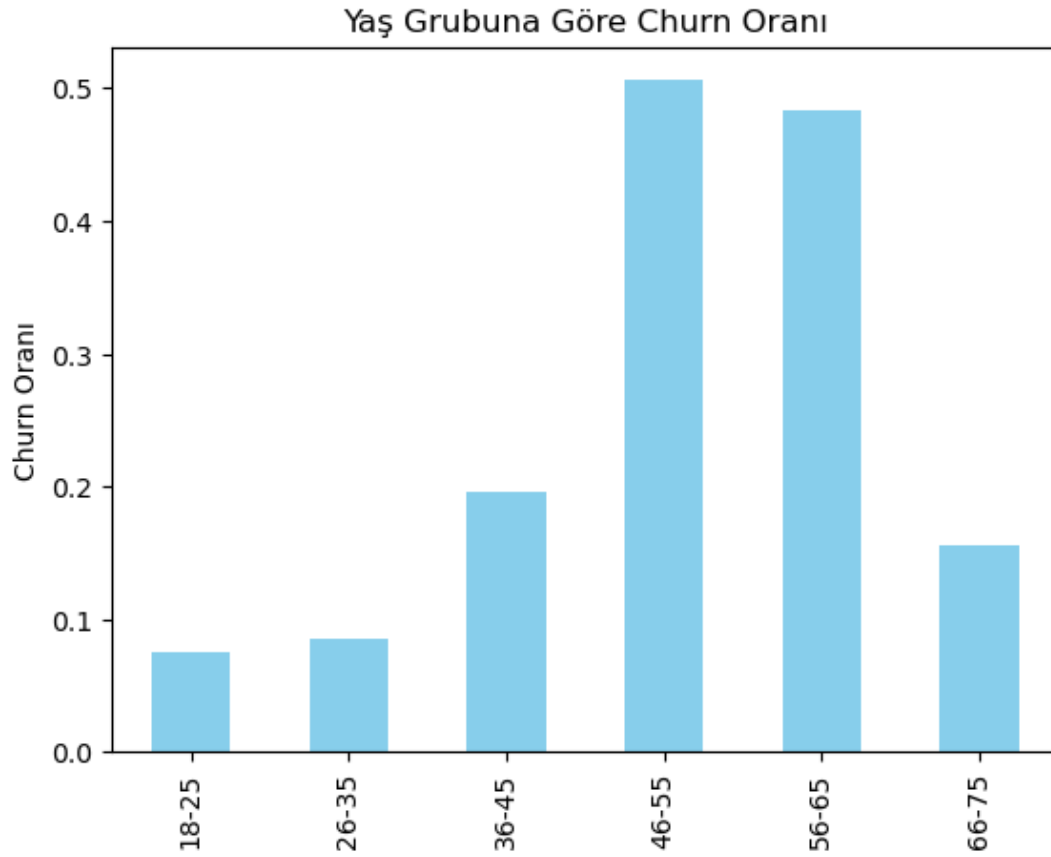
- Daha genç müşterilere özel teklifler sunmak.
- Daha az ürün kullanan müşterilere çapraz satış stratejileri geliştirmek.
- Aktif olmayan müşterileri yeniden kazanmaya yönelik kampanyalar düzenlemek.

Şirketi terk eden müşterilerin çoğu hangi yaşta?

```
# Yaşı gruplara ayırma
df['yas_grubu'] = pd.cut(df['yas'], bins=[18, 25, 35, 45, 55, 65, 75],
                        labels=['18-25', '26-35', '36-45', '46-55', '56-65', '66-75'])

# Her yaş grubu için churn oranını hesaplama
yas_grubu_churn = df.groupby('yas_grubu')['ayrıldı_mı(churn)'].mean()
print(yas_grubu_churn)

# Bar grafiği ile görselleştirme
import matplotlib.pyplot as plt
yas_grubu_churn.plot(kind='bar', color='skyblue', title='Yaş Grubuna Göre Churn Oranı')
plt.xlabel('Yaş Grubu')
plt.ylabel('Churn Oranı')
plt.show()
```



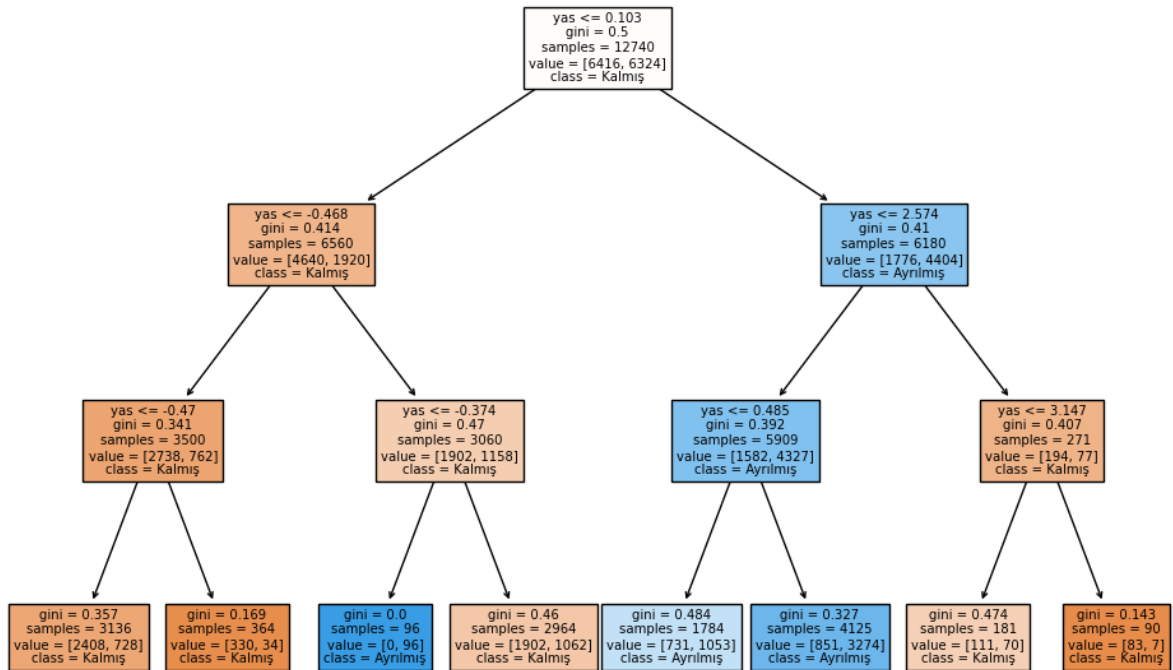
Şirketi en çok terk eden müşterilerin yaş grubu 46-55 ve 56-65 aralığı imiş.

```
# Yaş ve diğer özelliklere göre churn oranı
segment_churn = df.groupby(['yas_grubu', 'aktif_uye_mi'])['ayrıldı_mı(churn)'].mean()
print(segment_churn)
```

```
yas_grubu  aktif_uye_mi
18-25      0            0.115
          1            0.039
26-35      0            0.108
          1            0.063
36-45      0            0.242
          1            0.146
46-55      0            0.636
          1            0.372
56-65      0            0.939
          1            0.255
66-75      0            0.655
          1            0.079
Name: ayrıldı_mı(churn), dtype: float64
```

Yaş aralığı 56-65 olan ve aktif üye olmayanlar şirketi en çok terk eden müşteriler.

Karar ağacı ile inceleyelim:



Bu karar ağacı, yaş değişkeni üzerinden müşterilerin churn (ayrılma) ya da kalma durumlarını tahmin etmek için oluşturulmuştur. Model, yaş değişkenine göre veri kümesini dallandırarak sınıflar arasında en iyi ayrımı yapmaya çalışmaktadır.

Genel Yapı ve Önemli Dallanmalar:

1. Kök Düğüm (Root Node):

- Karar ağacının başlangıç noktasıdır ve tüm veri kümesi burada değerlendirilmektedir (12,740 örnek).
- “ $yas \leq 0.103$ ” koşulu, en önemli ayırım noktasıdır. Bu koşul, yaş değişkeninin churn üzerinde etkili olduğunu göstermektedir.
- Örneklerin büyük kısmı bu düğümde “kalmış” sınıfına (6,416) aittir; 6,324 örnek ise “ayrılmış” sınıfındadır.

2. Alt Gruplar:

- Sol Dal ($yas \leq 0.103$): Bu grup genelde daha genç bireyleri temsil etmektedir ve “kalmış” sınıfı baskındır.
- Sol dalda, yaş küçüldükçe churn olasılığı azalmaktadır. Örneğin:
- “ $yas \leq -0.47$ ” ile ayrılan alt grup, %87 oranında “kalmış” sınıfındadır.
- Daha derinlemesine ayrıldığında, bazı gruplar tamamen “kalmış” sınıfına aittir (ör. Gini = 0.0).
- Sağ Dal ($yas > 0.103$): Bu grup genelde daha ileri yaştaki bireyleri temsil etmektedir ve churn olasılığı artmaktadır.
- Sağ dalda, özellikle “ $yas > 2.574$ ” koşulu ile ayrılan alt grup churn sınıfında baskındır. Örneğin:
- “ $yas > 0.485$ ” koşulu sağlandığında, örneklerin çoğunluğu (%73) churn sınıfına aittir.

3. Yaprak Düğümler (Leaf Nodes):

- Model, her yaprak düğümde churn ve kalma sınıflarının dağılımını gösterir.
- Örneğin:
- Sol alt köşedeki düğümde 3,364 örneğin 2,408’i “kalmış”, 728’i “ayrılmış” olarak sınıflandırılmıştır.
- Sağ alt köşede ise 3,274 churn, 851 kalma örneği bulunmaktadır.

Yorum ve Model Performansı:

- Yaş değişkeni, churn durumunu tahmin etmekte güçlü bir etkindir. Özellikle yaş arttıkça churn olasılığı yükselmektedir.

- Model, churn ve kalma sınıflarını iyi ayırtmaktadır. Ancak, her düğümde homojenlik tam sağlanmamıştır (Gini > 0 olan düğümler).
- Yaprak düğümlerde sınıf dağılımı net bir şekilde sunulmuş ve churn sınıfının daha küçük alt gruplar halinde temsil edildiği gözlemlenmiştir.

Sonuç: Bu model, müşterilerin yaş değişkenine dayalı churn olasılığını değerlendirmede etkili bir yol sunmaktadır. Ancak, churn sınıfı sayıca daha az olduğu için model performansı daha homojen veri ile artırılabilir veya diğer değişkenlerin eklenmesiyle zenginleştirilebilir.

Test tahmin sonuçlarını excel dosyasına kaydedelim.

```
test_results.to_excel("test_results.xlsx", index=False)
test_results.to_excel(r"C:\Users\HANDENUR\Downloads\test_results.xlsx", index=False)
```

KAYNAKÇA

İST405 Veri Madenciliği Ders Notları

<https://plotly.com/python/discrete-color/>

<https://www.geeksforgeeks.org/python-operators/>

<https://www.data-to-viz.com/>

<https://medium.com/deep-learning-turkiye/karar-a%C4%9Fa%C3%A7lar%C4%B1-makine-%C3%B6%C4%9Frenmesi-serisi-3-a03f3ff00ba5>

<https://www.geeksforgeeks.org/z-score-normalization-definition-and-examples/>

https://mustafabayhan.com/tr/post/dengesiz-dagilmis-veri-ile-calisma-oversampling-yontemi/#google_vignette

<https://ahmetokanarik.medium.com/smote-synthetic-minority-oversampling-technique-c52d4fbec47e>